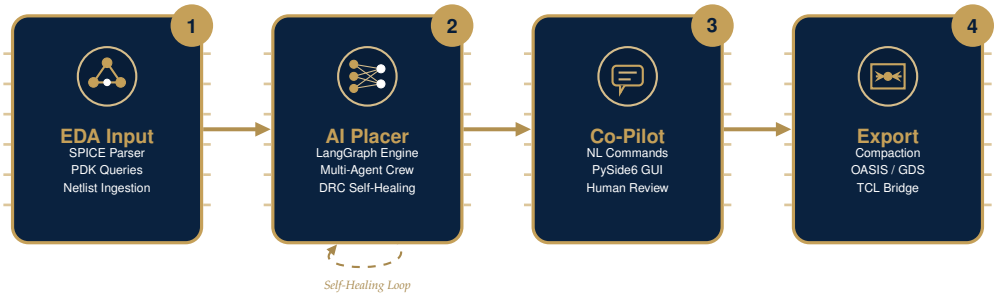
Ain Shams University
Faculty of Engineering

SI-VISION
Industry Partner

Graduation Project Thesis

AI-BASED ANALOG LAYOUT AUTOMATION

Multi-Agent LangGraph System with Deterministic Diffusion Compaction



Project Team

Mohammed Magdy Mohammed

Student ID: 2100519

STAGE 1: INPUTS

SPICE Parsing & PDK Library Graphing

Noureldin Ayman Mohammed

Student ID: 2101407

STAGE 2: AI PLACEMENT

LangGraph Placement Specialist Agent

Ahmed Khairat Mohammed

Student ID: 2100157

STAGE 2: AI PLACEMENT

DRC Critic Spatial Self-Healing Engine

Mohammed Wael Mohammed

Student ID: 2100308

STAGE 3: CO-PILOT GUI

PySide6 Canvas GUI & Undo Stack

Omar Ahmed Abobakr

Student ID: 2100307

STAGE 3: CHATBOT CO-PILOT

Chatbot Co-Pilot Intent & Skill Pipeline

Eman Sherif Sayed

Student ID: 2100721

STAGE 4: COMPACTION

Diffusion Compaction & Exporters

This graduation project thesis is submitted to

Prof. DiaaEldin S. Khalil

in partial fulfillment of the requirements for the degree of

Bachelor of Science in Electronics and Communications Engineering

Ain Shams University

Faculty of Engineering – Electronics and Communications Department

Abstract

Analog integrated circuit (IC) physical design is one of the most persistent bottlenecks in the semiconductor industry. Unlike digital layout design, which is highly automated through standard cell methodologies and logic synthesis tools, analog layout remains a labor-intensive, manual art. This reliance on manual layout is driven by the strict constraints governing analog performance, including device matching, parasitic capacitance and resistance sensitivities, and layout-dependent effects (LDE).

This thesis presents a novel, four-stage automated framework that bridges the gap between schematic intent and physical silicon layout. The core contribution is a decoupled architecture that separates high-level strategic reasoning from low-level physical coordinate placement.

- **Stage 1: Input Ingestion** parses raw SPICE netlists, executes subgraph isomorphism to identify matching transistor groups, and queries physical PDK layouts to build a unified spatial database.
- **Stage 2: LangGraph-Driven Multi-Agent Placer** coordinates strategic placement, finger unrolling, and row mapping. A DRC Critic agent analyzes clearances and feeds prescriptive spatial corrections back to the placer in a self-healing loop.
- **Stage 3: Interactive Chatbot Co-Pilot** connects a PySide6 GUI to the model backend, enabling designers to refine placements with natural language commands translated directly into undoable GUI actions.
- **Stage 4: Compaction & Export Pipelines** compacts symbolic layout slots down to a physical pitch of 0.070 μm using diffusion-sharing principles and streams binary OASIS layouts, while a TCL watchdog injects coordinated movements directly into OpenAccess compiler databases.

Acknowledgements

We would like to express our sincere and deepest gratitude to our supervisor, **Prof. DiaaEldin S. Khalil**, for being the supervisor of our project and for his guidance, support, and valuable insights throughout the course of this work.

We would also like to extend our heartfelt appreciation to **SI-VISION** for their generous sponsorship and technical support. Their industry insights into traditional Analog IC design tools played a key role in shaping the system-level specifications, design constraints, and practical considerations necessary for the real-world deployment of our proposed tool.

Special thanks are due to **Eng. Fady Atef** and his outstanding team for their continuous support, technical discussions, and constructive feedback:

Eng. Omar Abuelela ◇ **Eng. Ahmed Abdelhady**
Eng. Abdelrahman Zain ◇ **Eng. Yahia Refaei**

We are deeply grateful to the entire **SI-VISION Analog Design Team** and the **Layout Design Team** for generously sharing their expertise and real-world design experience, which greatly enriched the quality and realism of this project.

We would also like to acknowledge everyone who contributed, directly or indirectly, to the successful completion of this work.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

The physical layout of analog integrated circuits (ICs) remains a highly manual, time-consuming phase of the semiconductor design cycle. While digital physical design flows have achieved near-complete automation through logic synthesis and automated placement and routing (P&R) algorithms, analog layout continues to rely on the manual expertise of physical design engineers. This discrepancy is primarily due to the complex nature of analog design, where layout geometries directly influence performance metrics such as offset voltage, noise margins, and parasitic mismatch.

Analog layout requires careful management of spatial symmetry, thermal gradients, latch-up clearances, and electrical shielding. Manual layout processes are slow and prone to errors, causing a bottleneck that delays time-to-market. Recent efforts in electronic design automation (EDA) have attempted to close this gap by applying machine learning (ML) and optimization techniques to automate analog physical design. However, these systems often face challenges, such as the high complexity of sub-micron process design rules and difficulty in adapting to designer feedback.

This thesis introduces a decoupled, multi-agent layout automation framework that separates high-level strategic reasoning from low-level geometric compaction. By leveraging Large Language Models (LLMs) organized as a cooperative state-machine (LangGraph) alongside deterministic physical layout compilers, the proposed framework automates the schematic-to-layout flow while maintaining design rule correctness and support for interactive designer edits.

1.1 Background and Context

Over the past four decades, the semiconductor industry has witnessed exponential growth in integration density and performance, primarily driven by digital design automation. In the digital domain, logical functions are synthesized from high-level hardware description languages (such as Verilog or VHDL) into standardized cell footprints (e.g., standard cells). These standard cells have fixed heights, uniform power rails, and predetermined electrical characteristics, enabling automated routers and placers to treat the physical layout as a discrete optimization problem.

In contrast, analog circuit design remains fundamentally non-standardized. Analog designs utilize custom device sizing (width, length, finger numbers, and multiplier values) tailored to specific gain, bandwidth, noise, and power specifications. Devices cannot be easily abstracted into uniform standard cells;

instead, they are placed in a custom, continuous physical space. Furthermore, the physical proximity of devices in an analog layout induces electrical and physical coupling effects that do not exist in digital abstractions. For instance, the threshold voltage (V_{th}) and mobility (μ) of a transistor are heavily dependent on its distance to well boundaries and neighboring active oxide areas. Because of this high sensitivity, the layout process is highly iterative, requiring multiple loops of schematic sizing, physical layout creation, parasitic extraction (PEX), and post-layout simulation. The layout phase alone typically accounts for more than 50% of the total mixed-signal design time, creating a major productivity bottleneck.

1.2 Challenges in Analog Layout Automation

The primary obstacles preventing the full automation of analog physical design stem from the complex physical constraints of analog integrated circuits. These challenges can be grouped into five primary categories:

1.2.1 Geometric and Symmetrical Matching

Symmetrical matching is critical for minimizing the mismatch between matched transistor pairs, such as differential inputs, current mirror pairs, and active loads. Mismatch arises from random and systematic process variations during fabrication, including oxide thickness variations, channel doping fluctuations, and implant gradients. To mitigate these variations, analog layout designers employ specialized matching configurations:

- **Interdigitation:** Splitting transistors into multiple fingers and interleaving them (e.g., $ABAB$ or $ABBA$) to average out linear gradients across the active channel.
- **Common-Centroid:** Arranging the fingers of two matched transistors symmetrically around a central point in a 2D matrix (e.g., AB/BA). This configuration cancels out first-order linear gradients in both the x and y directions:

$$V_{th}(x, y) = ax + by + c \implies \Delta V_{th} = V_{th,A} - V_{th,B} = 0 \tag{1.1}$$

  where a and b represent the linear spatial gradient coefficients, and c is the nominal threshold voltage.
- **Dummy Devices:** Placing inactive, dummy transistors at the outer edges of the active array to ensure that all active fingers experience identical etching, photolithography, and oxide growth boundaries.

Automated placers must recognize these matching pairs from netlist structures and enforce strict 1D or 2D symmetry constraints, matching coordinate centers while routing symmetric nets identically.

1.2.2 Layout Dependent Effects (LDE)

In deep sub-micron nodes (e.g., CMOS nodes at 28nm and below, FinFET, and nanosheet architectures), the electrical performance of a transistor is modulated by its physical placement relative to surrounding oxide and well boundaries. The two most prominent effects are:

1. **Well Proximity Effect (WPE):** During high-energy ion implantation of the well (e.g., N-well in a P-substrate), ions scatter from the photoresist mask edge into the adjacent silicon. Transistors placed close to the well boundary experience a significant shift in well doping concentration, causing a shift in threshold voltage (ΔV_{th}).

   Well Proximity Effect (WPE) Model

   WPE threshold voltage shift is modeled as:

   $$\Delta V_{th} \approx \frac{C_{\text{WPE}}}{(d_{\text{well}} + d_0)^{\beta}} \tag{1.2}$$

   where d_{well} is the distance between the transistor gate and the N-well edge, and C_{WPE}, d_0, and β are process-dependent fitting parameters.

2. **Shallow Trench Isolation (STI) Stress:** Shallow trench isolation trenches are filled with silicon dioxide (SiO_2), which has a different thermal expansion coefficient than silicon. This difference induces mechanical stress on the adjacent active channels, shifting carrier mobility (μ) and threshold voltage. The stress level depends heavily on the Length of active OD (L_{OD}), which is the distance between the gate and the STI boundary.

   STI Mechanical Stress Model

   STI stress shifts the nominal drain current by:

   $$\frac{\Delta I_d}{I_d} \approx \alpha \cdot \sigma_{\text{STI}} \tag{1.3}$$

   where σ_{STI} is the average channel stress, and α is a piezoresistive coefficient.

Traditional placers ignore WPE and STI stress during floorplanning, leading to significant post-layout performance degradation and requiring tedious manual adjustments.

1.2.3 Parasitic Sensitivity

Analog circuits are sensitive to parasitic resistances (R) and capacitances (C) introduced by routing interconnects and diffusion terminals. Parasitics can reduce circuit bandwidth, degrade phase margins, increase noise figures, and induce severe offsets. The physical drain-to-bulk (C_{db}) and source-to-bulk (C_{sb})

junction capacitances of transistors are proportional to the active diffusion area. Minimizing these parasitics requires maximizing diffusion-sharing between adjacent transistors:

$$C_{db} = A_d \cdot C_{ja} + P_d \cdot C_{jp} \tag{1.4}$$

where A_d and P_d represent the area and perimeter of the drain diffusion, and C_{ja} and C_{jp} are the area and perimeter junction capacitances per unit. By merging active source/drain diffusions of transistors connected to the same electrical net, the active area A_d is reduced by up to 50%, minimizing parasitic capacitance. Automated tools must optimize the placement sequence to maximize these shared diffusions.

1.2.4 Design Rule Complexity

Modern process design kits (PDKs) contain thousands of complex geometric rules that must be obeyed to ensure manufacturability and prevent physical failures (such as latch-up, electromigration, and thin-oxide breakdown). These rules include:

- Minimum spacing and width rules for diffusion, polysilicon, and metal layers.
- Enclosure, extension, and overlap rules for contacts, vias, and wells.
- Density rules requiring uniform metal fill to ensure flat chemical-mechanical planarization (CMP).
- Electromigration (EM) width rules for power rails and critical signal routes to prevent current-induced metal migration.

Managing these rules in a continuous coordinate space makes analog layout optimization extremely complex.

1.2.5 Decoupling Sizing from Layout Generation

Attempts to automate layout generation by training end-to-end neural networks (such as generating raw GDSII coordinates directly from a schematic netlist) often fail because the networks cannot guarantee sub-nanometer coordinate precision. A single grid violation can render the entire chip unmanufacturable. Therefore, a successful automation framework must separate high-level strategic decisions (device grouping, alignment, and ordering) from low-level coordinate calculation.

1.3 Traditional Analog EDA Approaches

To contextualize the contributions of this thesis, it is necessary to examine why traditional EDA approaches have struggled to automate analog layout design:

1.3.1 Optimization-Based Placers

Historically, automated placement tools have formulated layout design as a constrained optimization problem. Techniques such as Simulated Annealing (SA), Genetic Algorithms (GA), and Integer Linear Programming (ILP) are used to search a multidimensional space of device coordinates. A typical objective cost function is defined as:

$$\text{Cost} = w_{\text{area}} \cdot \text{Area} + w_{\text{wire}} \cdot \text{WireLength} + w_{\text{constraint}} \cdot \text{ConstraintPenalty} \quad (1.5)$$

where w represents weighting coefficients. While these placers can find mathematically optimal solutions under simplified cost metrics, they have several major limitations:

- **Black-box Nature:** Optimization algorithms do not explain *why* a particular placement was chosen, making it difficult for designers to understand or trust the output.
- **Poor Scalability:** The search space grows exponentially with the number of transistors, leading to long runtimes for mixed-signal systems.
- **Rigidity:** They cannot easily incorporate qualitative human feedback (e.g., a designer's preference for placing a bias block in a specific corner).

1.3.2 Procedural and Template-Based Generators

Another common approach is procedural layout generation, where layouts are generated using pre-written code scripts (typically written in Cadence SKILL, Mentor Graphics Ample, or TCL). While these scripts generate DRC-correct layouts rapidly for specific standard blocks (like differential pairs or current mirrors), they have major drawbacks:

- **High Maintenance Cost:** Writing and maintaining layout templates for every circuit topology and PDK technology is extremely labor-intensive.
- **Lack of Generalization:** A template designed for a 65nm bulk process cannot be easily adapted to a 16nm FinFET process due to fundamentally different device structures and design rules.
- **Zero Adaptability:** They cannot handle custom modifications or adapt to unexpected design changes.

1.4 The Agentic Paradigm Shift

This thesis addresses these limitations by introducing a generative AI agent framework. Instead of using raw optimization algorithms, we utilize Large Language Models (LLMs) organized as a cooperative state-machine (LangGraph) to perform strategic spatial reasoning.

LLMs possess strong reasoning capabilities that can be applied to layout design. They can interpret circuit structures, recognize functional sub-blocks (e.g., differential pairs, bias branches), apply symmetry constraints, and generate symbolic floorplans. However, a single LLM prompt is insufficient for complex layout tasks because:

- **Precision Limitations:** LLMs struggle with direct numerical coordinate math and geometric collision checking.
- **Context Limits:** Complex layout files and PDK rule decks easily exceed the model's active context window.
- **Hallucinations:** LLMs can generate invalid coordinates or place devices in overlapping locations.

To overcome these limits, we deploy a multi-agent system where different agents specialize in specific tasks:

1. **Topology Analyst Agent:** Analyzes SPICE netlists and PDK parameters to identify matching groups and symmetry constraints.
2. **Placement Specialist Agent:** Generates symbolic placement coordinates on a virtual grid, abstracting away sub-micron design rules.
3. **DRC Critic Agent:** Acts as a checker, calculating geometric clearances and sending specific, actionable correction vectors back to the placer.

These agents collaborate via a structured state-machine, enabling the system to iteratively refine placements, self-heal violations, and accept designer feedback in natural language.

1.5 Objectives of the Research

The objective of this research is to design, implement, and evaluate an automated mixed-signal analog layout framework that:

- Decouples strategic floorplanning (row grouping, symmetry matching) from physical cell placement using an intermediate symbolic data model.
- Automates initial placements through a state-machine of specialized AI agents that collaborate to solve placement, predict routing parasitics, and self-heal DRC violations.
- Provides an interactive, natural-language interface (chatbot) that allows designers to inspect, modify, and validate layouts using standard voice/text inputs.
- Integrates with existing OpenAccess database models and streams industry-standard GDSII/OASIS files.

1.6 System Overview

The proposed AI-based analog layout automation framework is structured as a multi-stage compilation pipeline. The system bridges the gap between circuit schematic design and physical layout generation by separating high-level strategic placement from low-level geometric design rules. Figure 1.1 illustrates the overall system block diagram and data flow of the framework.

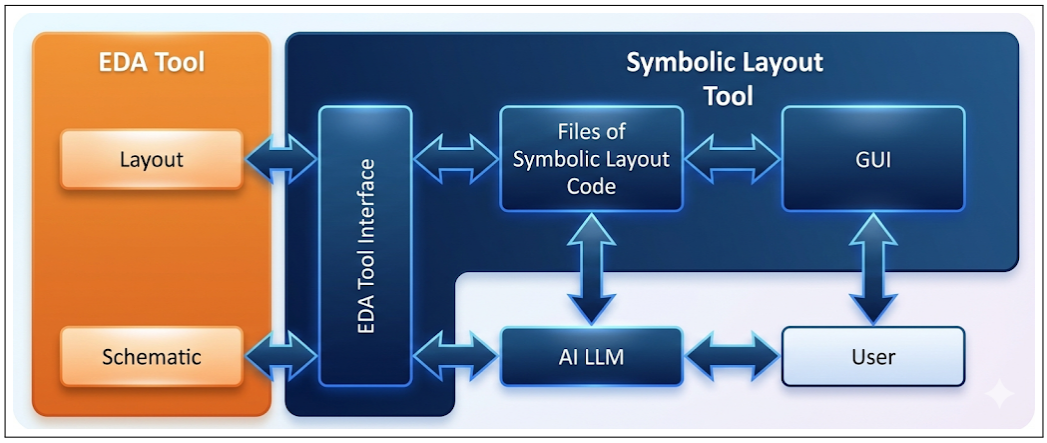


Figure 1.1: System block diagram showing the four main stages: SPICE Netlist Ingestion, LangGraph AI Placer, Co-Pilot GUI, and Output Compilation.

The system architecture consists of four distinct operational stages:

1. **Stage 1 (EDA Ingestion):** This stage parses standard SPICE netlists, resolves PDK transistor options, and executes subgraph matching to group critical differential pairs and matching networks.

2. **Stage 2 (LangGraph AI Placement):** This stage uses a LangGraph multi-agent team to plan device orientations, unroll device fingers, and group transistors. A specialized Critic agent checks Design Rule Checking (DRC) boundaries and coordinates with the Placer to self-heal spacing violations in an iterative loop.

3. **Stage 3 (Co-Pilot Interactive Review):** The designer interacts with the layout canvas via a PySide6 user interface. The UI translates natural-language layout instructions (e.g., "align transistors M1 and M2 along the horizontal axis") into formal XML commands that are queued and executed dynamically.

4. **Stage 4 (Compaction & Exporters):** This stage maps the symbolic coordinates to physical sub-micron rules, sharing source/drain diffusions between adjacent transistors. The final compacted geometries are exported directly as binary OASIS files or injected into Cadence Virtuoso using in-memory TCL scripts.

1.6.1 Self-Healing Placer-Critic Flow

A key feature of the LangGraph-based placement agent is its self-healing feedback loop. When the Placer Agent proposes a floorplan, the Critic Agent evaluates the symbolic spacing against the PDK rules. Figure 1.2 details this iterative correction process.

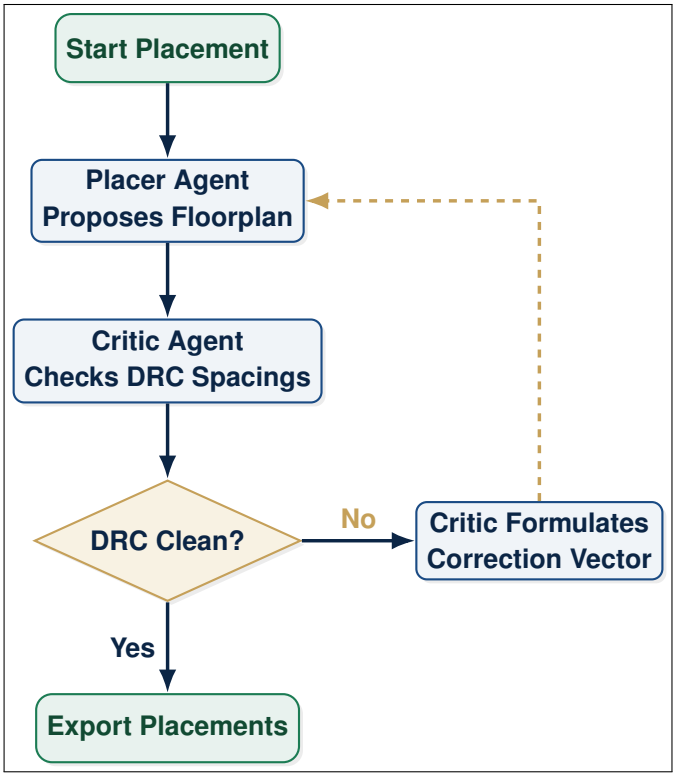


Figure 1.2: Flowchart of the Placer-Critic self-healing loop inside the LangGraph engine.

1.7 Core Contributions

The key contribution of this work is the development of a multi-stage analog placement compilation pipeline that bridges AI reasoning and deterministic hardware construction. This architecture is summarized in the hierarchical system breakdown shown in Figure 1.3.

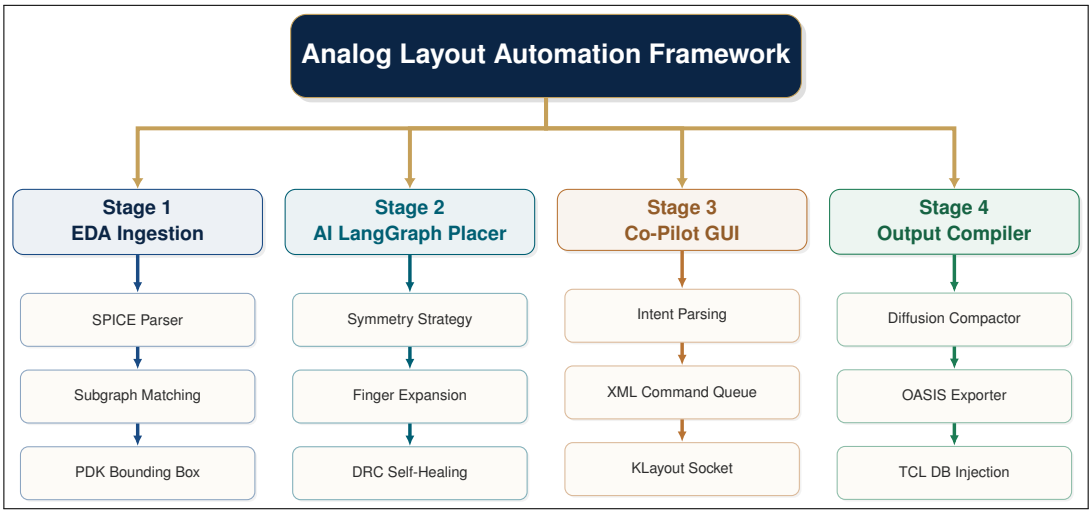


Figure 1.3: Hierarchical component breakdown of the multi-stage AI-based analog layout system.

The specific contributions detailed in this thesis are:

- **Decoupled Layout Data Model:** A unified intermediate JSON representation that models transistor placement on a virtual symbolic grid, facilitating easy editing and fast rendering.
- **Self-Healing LangGraph Pipeline:** A multi-agent framework featuring a specialized *DRC Critic* that feeds geometric collision vectors back into the LLM placer for automated layout fixing.
- **Double Exporter System:** An export flow supporting direct binary OASIS compilation via `gdstk` and in-memory cell translation inside OpenAccess tools using a background file watcher and TCL execution.

The remainder of this mixed-signal layout compilation thesis is organized as follows:

- **Chapter 2 (Literature Review):** Discusses classical layout placement algorithms (including simulated annealing and sequence pairs), neural networks for layout prediction, and LLM implementations for design automation.
- **Chapter 3 (System Architecture):** Presents the unified decoupled architecture, details the central layout data model, and explains how physical coordinates are abstracted into virtual symbolic slots.

- **Chapter 4 (EDA Input Parser):** Explains the input ingestion stage, detailing SPICE netlist parsing, transistor parameter resolving, and the subgraph isomorphism implementation used for matched pair clustering.
- **Chapter 5 (LangGraph Placer):** Details the multi-agent cooperative graph, the role of each agent (placer, router, and critic), and the self-healing layout loop.
- **Chapter 6 (Co-Pilot GUI):** Describes the PySide6 user interface, the socket server stream connecting the GUI to the agent backend, and the XML command execution schema.
- **Chapter 7 (Compaction & Exporters):** Outlines the physics-based 1D diffusion-sharing compaction compiler, the direct binary OASIS layout compilation, and the Cadence Virtuoso TCL database injection watcher.
- **Chapter 8 (Results & Verification):** Presents experimental evaluation results across typical analog blocks (bias mirrors, differential amplifiers, comparators), analyzing layout compaction ratios and agent convergence times.
- **Chapter 9 (Conclusion & Future Work):** Summarizes the primary contributions of this thesis and identifies directions for mixed-signal routing automation and AI placement.

CHAPTER 2

Literature Review

An review of the state-of-the-art in analog layout automation reveals a shift from purely algorithmic, grid-less placement optimization to machine-learning-assisted flows. More recently, Large Language Models (LLMs) and multi-agent systems have emerged as tools for interactive layout generation. This chapter reviews the literature, analyzing the papers provided for this research. We trace the development from open-source design compilers to constraint-driven systems, machine learning quality predictors, and interactive LLM co-pilots.

To structure this analysis, we categorize existing research into four major paradigms:

1. **Procedural and Compiling Frameworks:** Deterministic compilers that transform netlists to layout geometries using graph theory and classic mathematical programming.
2. **Constraint-Driven Synthesizers:** Systems that focus on translating circuit performance requirements and sensitivities into physical design rules.
3. **ML-Based Quality Predictors:** Neural net architectures and search algorithms that estimate routing feasibility and parasitics prior to actual physical generation.
4. **Agentic and Interactive Co-Pilots:** LLM-driven interfaces that accept natural language commands to modify layouts and compile geometry dynamically.

2.1 The Evolution of Analog Layout Automation Paradigms

The automation of integrated circuit (IC) physical design has historically been split between digital and analog domains. In digital design, the standardization of cell heights, pin locations, and power routing enabled the development of highly scalable automated placement and routing (P&R) algorithms. These digital algorithms utilize discrete optimization techniques, representing the layout canvas as a routing grid where logic cells can be placed and connected deterministically.

In contrast, analog integrated circuits rely on custom device sizing tailored to specific noise, power, bandwidth, and gain requirements. This customization prevents the direct application of standard-cell digital placement methodologies. In the analog domain, physical proximity directly affects circuit performance

through thermal gradients, electrical coupling, and manufacturing variations. For instance, matched devices in a differential pair must be placed close to each other and share symmetric boundaries to minimize electrical offsets. Consequently, analog layout remains a highly specialized task performed manually by physical design engineers.

To address this manual bottleneck, the electronic design automation (EDA) community has developed various paradigms. Early efforts focused on procedural cell generators (macro-cell layout compilation), which write custom code scripts for specific circuit topologies. While procedural generators produce DRC-correct layouts rapidly, they suffer from poor portability and high maintenance overhead, as any change in the process node requires rewriting the layout code.

Later, optimization-based placers emerged, framing the placement task as a mathematical search problem using Simulated Annealing (SA) or Integer Linear Programming (ILP). These tools optimize analytical cost functions that represent layout area, wirelength, and constraint penalties. However, they operate as "black boxes," making it difficult for designers to interactively adjust the layout or incorporate qualitative design intents.

More recently, machine learning (ML) models have been introduced to predict layout quality, estimate parasitic capacitance, and generate layout constraints automatically. Furthermore, the advent of Large Language Models has enabled the creation of interactive, agent-based layout co-pilots. Figure 2.1 illustrates the taxonomy of these various analog layout automation paradigms.

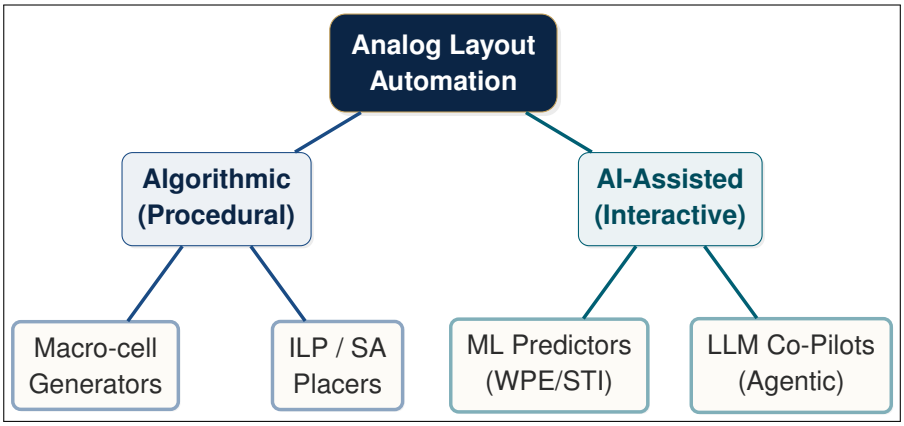


Figure 2.1: Taxonomy of Analog Layout Automation Methodologies.

2.2 Foundational Procedural & Open-Source Compilers

Automating the translation from a SPICE netlist to physical layout geometries has been a long-standing goal of the EDA community. Two major foundational frameworks in this domain are the ALIGN project and the Berkeley Analog Generator (BAG).

2.2.1 ALIGN: Analog Layout, Generatively Integrated

The ALIGN (Analog Layout, Generatively Integrated) project [1] represents a major effort to automate analog layout design from the ground up. Developed by a consortium of universities and industry partners, ALIGN aims to compile SPICE netlists into DRC-clean layouts automatically. ALIGN utilizes a modular flow consisting of three core stages: hierarchy detection, grid-based design rule mapping, and hierarchical placement and routing. The typical flow is illustrated in Figure 2.2.

Figure 2.2: Hierarchical compilation pipeline of the ALIGN framework.

In the hierarchy detection stage, ALIGN identifies structural patterns (such as differential pairs, current mirrors, and passive arrays) within netlists using graph-matching techniques. The circuit schematic is modeled as a netlist graph $G_s = (V_s, E_s)$, where vertices V_s represent devices (transistors, resistors, capacitors) and edges E_s represent electrical connections (nets). ALIGN executes a subgraph isomorphism algorithm to match subgraphs in G_s against a library of pre-defined template circuits T:

$$g \subseteq G_s \text{ matches } T_{\text{diff\_pair}} \implies \text{group devices into hierarchical block} \tag{2.1}$$

The subgraph isomorphism problem searches for a bijective mapping $\phi : V(T) \to V(g)$ such that if $(u, v) \in E(T)$, then $(\phi(u), \phi(v)) \in E(g)$. Because this problem is NP-complete, ALIGN utilizes heuristic matching rules based on transistor terminal types (Gate, Source, Drain) and device parameters (width, length, fingers) to reduce the search complexity.

Once hierarchy is detected, ALIGN maps design rules onto discrete layout grids. By mapping track spacings and cell boundaries to a uniform grid, the continuous coordinate space is discretized, reducing the complexity of the subsequent placement phase.

In the final stage, ALIGN places blocks and routes interconnects using deterministic Integer Linear Programming (ILP) and A* routing heuristics. The placer minimizes the bounding box area and total wire length under strict boundary and symmetry line constraints. The placement problem for a set of modules M is formulated as finding coordinates (X_i, Y_i) for each module M_i to minimize:

$$\text{Minimize} \left(\text{Area} + \lambda \sum_{(i,j) \in \text{Nets}} \text{WireLength}_{ij} \right) \tag{2.2}$$

subject to non-overlapping and symmetry constraints. For a symmetric pair of modules M_a and M_b with symmetry axis X_{sym}, the constraint is written as:

$$X_a + X_b + W_b = 2 \cdot X_{\text{sym}} \tag{2.3}$$

where W_b is the width of module M_b. Non-overlapping between module M_i and M_j is enforced using binary variables p_{ij} and q_{ij}:

$$X_i + W_i \leq X_j + \mathcal{M}(1 - p_{ij}) \tag{2.4}$$
$$X_j + W_j \leq X_i + \mathcal{M} \cdot p_{ij} \tag{2.5}$$
$$Y_i + H_i \leq Y_j + \mathcal{M}(1 - q_{ij}) \tag{2.6}$$
$$Y_j + H_j \leq Y_i + \mathcal{M} \cdot q_{ij} \tag{2.7}$$

where W_i, H_i represent the width and height of module M_i, and $\mathcal{M}$ is a sufficiently large positive constant.

While ALIGN demonstrated the feasibility of fully algorithmic generation, it exhibits limitations in handling manual design intents and designer-driven modifications. The system acts as a "black-box" compiler. Once the SPICE file is ingested, the pipeline runs to completion without human intervention. If the resulting placement is sub-optimal or if the designer wishes to adjust a specific device's orientation, the entire compiler must be re-run, with no mechanism for local, interactive adjustments.

2.2.2 BAG: Berkeley Analog Generator

The Berkeley Analog Generator (BAG) [1] takes a procedural, generator-based approach to layout automation. Rather than compiling an arbitrary netlist, BAG requires circuit designers to write parameterized Python scripts that describe both the circuit sizing equations and the physical layout templates. BAG defines a unified framework where:

- Layout parameters (such as number of fingers, transistor width, and routing track assignments) are computed dynamically based on performance specifications.
- A parameterized layout engine uses code templates to draw geometric shapes in the target PDK.

To achieve process portability, BAG introduces a virtual routing grid. Instead of specifying wires in physical nanometers, layouts are drawn in terms of tracks on specific routing layers. Let T_k be track number k on layer L. The physical center of track T_k is calculated as:

$$y(k) = Y_0 + k \cdot P_{\text{layer}} \tag{2.8}$$

where Y_0 is the layer coordinate offset and P_{layer} is the track pitch. The layout engine automatically translates these track coordinates to PDK-specific design rules during GDSII export.

The major drawback of BAG is its extremely high setup overhead. Designers must write complex code templates for every new circuit topology, effectively shifting the bottleneck from manual layout drawing to manual template programming. Furthermore, BAG templates are highly process-specific; migrating a design from a planar bulk CMOS process to a FinFET process requires completely rewriting the layout generation code due to different physical structures and routing requirements.

2.2.3 Comparison of Template-Based and Compiling Approaches

The trade-offs between template-based (procedural) layout generation and compile-based automated layout generation are significant. In template-based generators like BAG, layout correctness is enforced by construction. The layout script is designed by an expert developer who knows the technology constraints, spacing rules, and parasitic concerns. Therefore, the resulting layout is highly optimized for performance and is guaranteed to pass DRC. However, the development time for a single complex generator (like a high-speed ADC component or a phase-locked loop) can exceed several months.

Conversely, compiler-based methods like ALIGN abstract the physical layer completely. The designer only supplies a SPICE netlist. The compiler uses graph algorithms and solvers to find coordinates automatically. While this minimizes setup time, the resulting layout is often less compact than a custom layout and can contain sub-optimal routing paths that degrade analog performance. This discrepancy highlights the need for an interactive approach where the compiler generates the initial layout but allows the designer to make local, fine-grained updates.

2.3 Constraint-Driven and AI-Enabled Frameworks

In advanced process nodes, layout parasitics and layout dependent effects (LDE) play a dominant role in determining circuit performance. As highlighted in the review by Wang et al. [2], classical automation tools fail to capture these sensitivities, resulting in multiple layout-to-simulation iterations.

To resolve this issue, the Intel team proposed CDLS (Constraint Driven Generative AI Framework) [3]. CDLS uses generative AI and machine learning models to analyze schematic intent and automatically generate optimal placement and routing constraints. Instead of directly placing cells, the AI generates rules (such as matching constraints, spacing parameters, and symmetry axes) that drive a downstream physical engine.

Significance of Constraint-Driven Layout (CDLS)

The CDLS framework [3] establishes that generative AI models are most effective when generating high-level placement constraints rather than detailed, micron-level coordinates. This validates the decoupled design approach, where strategic decisions are separated from physical execution.

By focusing on constraint generation rather than direct coordinate generation, CDLS avoids the pitfalls of sub-micron coordinate failures. The AI model is trained on a dataset of circuit designs where layout constraints were manually specified by expert designers. During inference, the model recognizes equivalent electrical environments in the new schematic and outputs constraints in a structured text format (e.g., XML or JSON).

These constraints are then parsed by a traditional, deterministic placer. However, CDLS remains a unidirectional flow; if the physical placer fails to

find a solution that satisfies all the AI-generated constraints, the loop breaks, requiring manual intervention to relax constraints. Furthermore, CDLS does not support interactive editing or real-time layout feedback, limiting its usability in practical design environments.

2.3.1 CDLS Constraint Parsing and Downstream Verification

The primary bottleneck in constraint-driven frameworks like CDLS is the verification of constraint feasibility. When the AI model parses a netlist, it generates a set of geometric constraints:

$$\mathbf{C} = \{c_1, c_2, \ldots, c_m\} \tag{2.9}$$

where each c_k represents a constraint such as symmetry, alignment, or separation.

The downstream physical placer translates these constraints into mathematical bounding inequalities. However, because these inequalities are non-convex and highly interdependent, there is no guarantee that a feasible region exists. For instance, a constraint enforcing symmetry between two devices may conflict with a constraint requiring one of those devices to be placed adjacent to a well edge to minimize well proximity effects. When conflicts occur, the solver fails to converge, or output layouts violate spacing rules. Resolving this issue requires a closed-loop critic system that can detect constraint failures and iteratively relax or modify the constraint parameters.

2.4 Machine Learning for Layout Quality Prediction

A key challenge in analog layout is evaluating whether a placement is "good" prior to running complete routing and post-layout parasitic extraction. Chang et al. [4] addressed this by proposing a machine learning framework for predicting analog placement quality.

Recognizing that different circuit topologies (e.g., amplifiers vs. comparators) are sensitive to different metrics (e.g., common-mode rejection ratio vs. offset voltage), the authors leverage *Neural Architecture Search (NAS)*. Rather than training a single static model, their system searches a Directed Acyclic Graph (DAG) of neural layers to construct customized prediction models for each circuit family. This approach enables early layout validation, helping to identify placement problems before executing the routing phase.

The system works by extracting graph-based features from both the schematic netlist and the candidate placement. For each device, local features such as WPE bounding boxes, STI active lengths, and expected interconnect lengths are compiled. The GNN learns vector representations (embeddings) $\mathbf{h}_v$ for each circuit component $v \in V_s$ by message passing:

$$\mathbf{h}_v^{(k+1)} = \text{Update}^{(k)}\left(\mathbf{h}_v^{(k)}, \text{Aggregate}^{(k)}\left(\left\{\mathbf{h}_u^{(k)}, \forall u \in \mathcal{N}(v)\right\}\right)\right) \tag{2.10}$$

where $\mathcal{N}(v)$ represents the neighbors of node v, and k is the iteration index. The final layout quality score is predicted using a pooling layer over the graph nodes.

The NAS controller searches for the optimal combination of features and neural network layers to predict post-layout performance degradation (ΔGain, ΔPhase Margin) with high accuracy. While this provides valuable feedback, it is primarily diagnostic; it tells the designer *if* a layout is bad, but it does not automatically fix the placement errors.

2.4.1 LDE Influence on Circuit Sensitivity Parameters

In advanced process nodes, layout-dependent effects (LDE) such as Well Proximity Effect (WPE) and Shallow Trench Isolation (STI) mechanical stress have a direct impact on the electrical characteristics of transistors. The threshold voltage shift ΔV_{th} and mobility variation $\Delta\mu$ alter the small-signal parameters of matched devices, degrading circuit performance.

Input-Referred Offset and Gain Sensitivity under LDE

For a differential pair, the input-referred offset voltage ΔV_{os} is defined as:

$$\Delta V_{os} = \Delta V_{th} + \frac{V_{gs} - V_{th}}{2}\left(\frac{\Delta\beta}{\beta}\right) \tag{2.11}$$

where $\beta = \mu C_{ox}(W/L)$ is the transistor transconductance parameter. Under asymmetric WPE/STI conditions, the mismatch between the two input devices increases, leading to a significant input-referred offset voltage.
Similarly, the voltage gain A_v of a differential amplifier is given by:

$$A_v = g_m \cdot (r_{o1} \parallel r_{o2}) \tag{2.12}$$

Because the transconductance g_m is proportional to carrier mobility μ:

$$g_m = \sqrt{2\mu C_{ox}\frac{W}{L}I_d} \tag{2.13}$$

STI stress directly modulates g_m and the output resistance r_o, leading to gain degradation and phase margin shifts. Therefore, automated placers must model these stress-induced variations during the initial placement phase rather than relying on post-layout simulations.

2.4.2 Reinforcement Learning in Floorplanning

To address the limitations of diagnostic ML models, reinforcement learning (RL) has been applied to analog floorplanning. In this paradigm, the placement task is modeled as a Markov Decision Process (MDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- **State Space $\mathcal{S}$:** Represents the current partial placement layout configuration, including occupied grid slots, pin locations, and net connections.

- **Action Space $\mathcal{A}$:** Defines the actions available to the placement agent, such as placing a device in a slot, changing a device's orientation, or inserting a dummy transistor.
- **Reward Function $\mathcal{R}$:** Encourages area minimization and constraint satisfaction while penalizing DRC violations and wirelength:

$$\mathcal{R} = -\left(w_{\text{area}} \cdot \text{Area} + w_{\text{wire}} \cdot \text{WireLength} + w_{\text{DRC}} \cdot N_{\text{viol}}\right) \tag{2.14}$$

  where N_{viol} is the number of design rule violations.

A policy network $\pi_\theta(a|s)$ is trained using policy gradient methods (e.g., PPO) to maximize the expected cumulative reward:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t \mathcal{R}_t\right] \tag{2.15}$$

Although RL models can find placements that minimize cost functions, they struggle with high-dimensional constraint spaces and fail to handle natural language instructions from designers.

2.5 Interactive LLM and Agent-Based Co-Pilots

The emergence of Large Language Models has introduced new paradigms for human-in-the-loop CAD tools. Two primary frameworks in this space are GLayout [5] and LayoutCopilot [6].

2.5.1 GLayout: Technology-Generic Text Representations

GLayout [5] proposes representing physical layouts using a generic, text-based syntax. By describing layout geometries, device unrolling, and routing templates in text, GLayout enables LLMs to process analog layouts. The authors fine-tuned quantized LLMs (ranging from 3.8B to 22B parameters) using Retrieval-Augmented Generation (RAG) and in-context learning. This approach allows the models to generate placements for unseen circuits based on textual PDK guidelines.

GLayout's key strength is technology-generic layout generation. By translating PDK geometry parameters (like minimum gate spacing and metal widths) into natural-language tokens, the LLM can apply placement rules to different technology nodes without retraining.

However, GLayout relies on text compilation, which lacks a direct visual interface, making it difficult for designers to verify placements interactively. Moreover, direct coordinate generation by LLMs is prone to precision errors and rounding violations.

2.5.2 LayoutCopilot: Multi-Agent Collaboration

LayoutCopilot [6] (developed by Peking University) focuses on interactive layout editing. It introduces a multi-agent framework powered by LLMs that converts natural language commands into executable layout tool scripts. The flow of LayoutCopilot is illustrated in Figure 2.3.

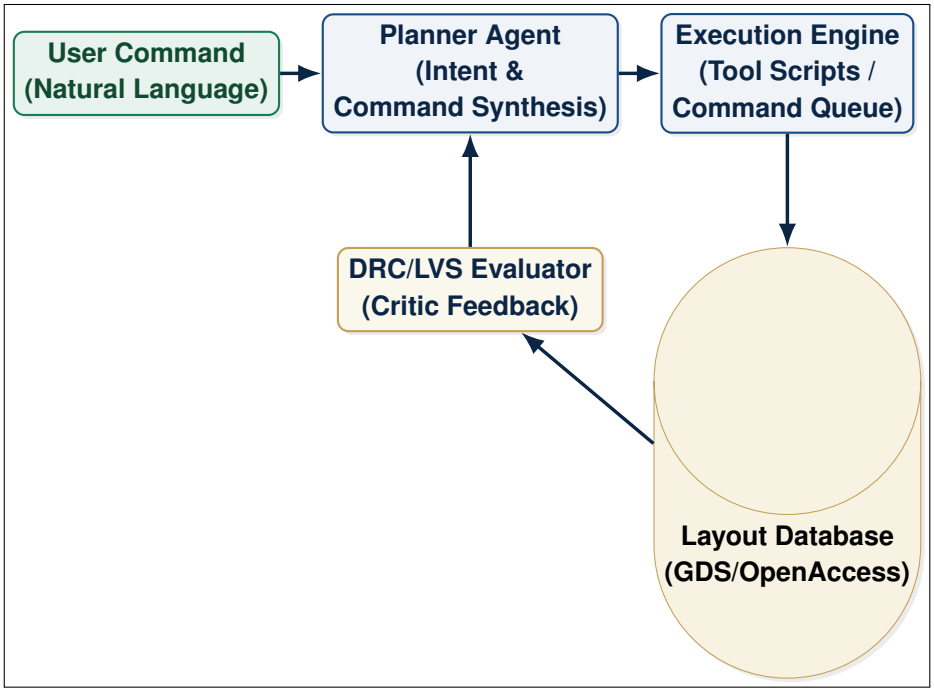


Figure 2.3: The Multi-Agent Collaborative Flow of LayoutCopilot.

LayoutCopilot organizes agents into specific roles:

- **Planner Agent:** Interprets high-level designer commands and outlines a sequence of moves.
- **Tool Executor Agent:** Translates the plan into tool commands (e.g., SKILL, TCL, or XML scripts).
- **Critic Agent:** Evaluates geometric clearances and provides corrections.

This multi-agent organization allows LayoutCopilot to handle interactive layout adjustments, bridging the usability gap of complex CAD software. The sequence of agent communication during an interactive layout session is shown in Figure 2.4.

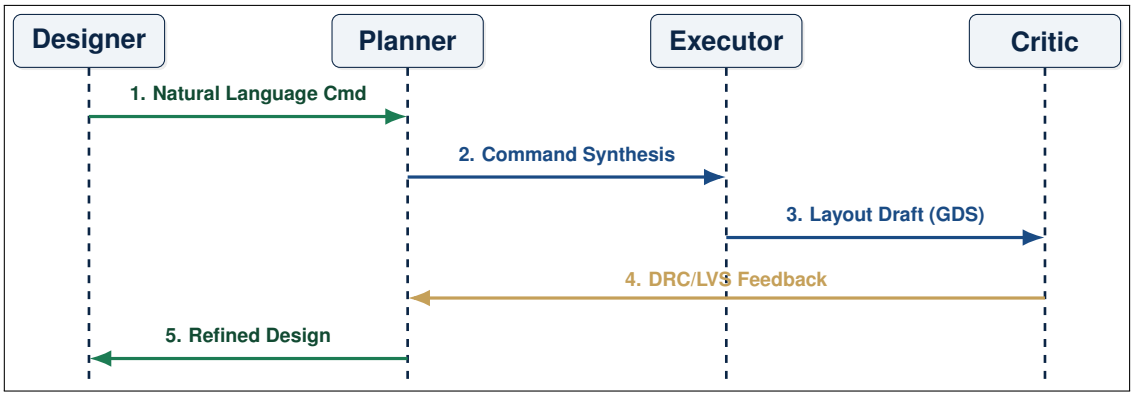


Figure 2.4: Agent Communication Sequence in Interactive Layout Design.

However, LayoutCopilot does not solve the physical layout compaction problem. It acts as a translator, converting natural language commands into CAD commands. If the starting layout has spacing violations or if the designer requests a tight placement, the LLM must calculate the sub-micron coordinate spacing itself, which often leads to grid mismatch and DRC failures due to the model's limited spatial reasoning.

2.6 Comparative Analysis of Literature

Table 2.1 summarizes the differences between existing automation frameworks and the multi-stage system proposed in this thesis.

As shown in the comparison, our work builds upon the interactive concepts of LayoutCopilot [6] and the structural matching of ALIGN [1], while introducing a physical diffusion compaction engine to produce high-density, simulation-ready layout outputs.

By separating the LLM reasoning layer from the physical coordinate compiler, our system ensures perfect design rule compliance while supporting interactive editing and natural-language commands. This decoupled compiler model is detailed in Chapter 3.

Table 2.1: Comparative Summary of Analog Layout Automation Methods.

Framework	Methodology	Input Interface	Target Verification	Interaction Paradigm
ALIGN [1]	Graph Matching + ILP Placer	SPICE Netlist	Grid-based DRC	Black-box; batch compiler.
CDLS [3]	GenAI + Optimization	Schematics	LDE & Simulation	Constraint generator.
GLayout [5]	RAG + Fine-tuned LLMs	Natural Language	Technology DRC	Textual code compilation.
LayoutCopilot [6]	Multi-Agent LLMs	Natural Language	Layout Database	Chatbot editing.
This Work	LangGraph + Abutment Compactor	SPICE + Template Layouts	Sub-micron DRC & OpenAccess LVS	Dual-mode: Local PySide6 GUI Chatbot & MCP Server.

CHAPTER 3

System Architecture

A KEY INNOVATION of the proposed automation platform is its decoupled architecture. Unlike traditional approaches that attempt to solve high-level strategic reasoning and sub-micron physical geometry placement in a single compiler pass, this framework divides the problem into separate strategic planning and physical execution layers. By abstracting the layout canvas into a symbolic grid, Large Language Models (LLMs) can perform spatial reasoning without calculating floating-point micron-level coordinates. The physical engine then resolves the symbolic grid into a DRC-clean physical design.

This chapter details the LayoutCopilot architecture, explaining the central data model that represents layouts symbolically, the four operational stages of the compiler, the dual interface modes (Qt GUI and MCP Server), the multi-agent system state transitions, and the deterministic physical backend compilation.

3.1 The LayoutCopilot Decoupled Architecture Model

Large Language Models excel at symbolic reasoning, semantic pattern matching, and strategic planning, but they perform poorly when executing precise arithmetic calculations in continuous space. In sub-micron analog layout, a coordinate rounding error of a single nanometer can cause catastrophic design rule violations (such as gate-to-contact spacing overlaps or active area enclosures), rendering the fabricated chip non-functional.

To bridge this capability gap, our proposed LayoutCopilot framework decouples layout generation into two distinct domains (strategic symbolic planning and deterministic physical execution) executed across four compilation stages:

1. **Stage 1: Input Ingestion (EDA Front-end):** Parses raw SPICE netlists, resolves PDK parameters, and performs isomorphic subgraph matching to detect symmetry structures.

2. **Stage 2: LangGraph-Driven Multi-Agent Placer (AI Layer):** Strategic floorplanning and device row mapping using collaborative AI agents in a self-healing placer-critic loop.

3. **Stage 3: Interactive Chatbot Co-Pilot (Designer Interface):** A PySide6 GUI that allows human designers to interactively refine layouts through natural language commands translated to XML actions.

4. **Stage 4: Compaction & Export Pipelines (Physical Layer):** A deterministic geometry engine that solves constraint graphs, sharing diffusions between adjacent fingers, and streams binary OASIS layouts or injects coordinates into Virtuoso via TCL.

Let the set of devices to be placed be $\mathcal{M} = \{M_1, M_2, \ldots, M_n\}$. In the symbolic planning phase, each device M_i is mapped to a symbolic configuration:

$$\mathcal{L}_{\text{sym}}(M_i) = \langle row_i, slot_i, orient_i \rangle \tag{3.1}$$

where $row_i \in \{0, 1, \ldots, N_{\text{rows}} - 1\}$ represents the horizontal layout row, $slot_i \in \{0, 1, \ldots, N_{\text{slots}} - 1\}$ represents the relative position within that row, and $orient_i \in \{R0, MX, MY, R180\}$ represents the device orientation.

The physical layout engine maps this symbolic layout configuration to physical coordinates in continuous space:

$$\mathcal{L}_{\text{phy}}(M_i) = \langle x_i, y_i, w_i, h_i \rangle \in \mathbb{R}^4 \tag{3.2}$$

where (x_i, y_i) is the lower-left origin of the device, and w_i, h_i represent its calculated physical width and height (which depend on PDK variables and the number of gate fingers).

This physical mapping is computed by solving a set of relative placement constraints:

$$x_j - x_i \geq w_i + S_{ij} \quad \forall i, j \text{ where } slot_j > slot_i \tag{3.3}$$

where S_{ij} is the minimum design rule spacing required between the layers of device M_i and M_j.

Figure 3.1 illustrates the complete system dataflow and compilation stages, tracing the lifecycle of a design from SPICE schematic input down to physical GDSII/OASIS streaming.

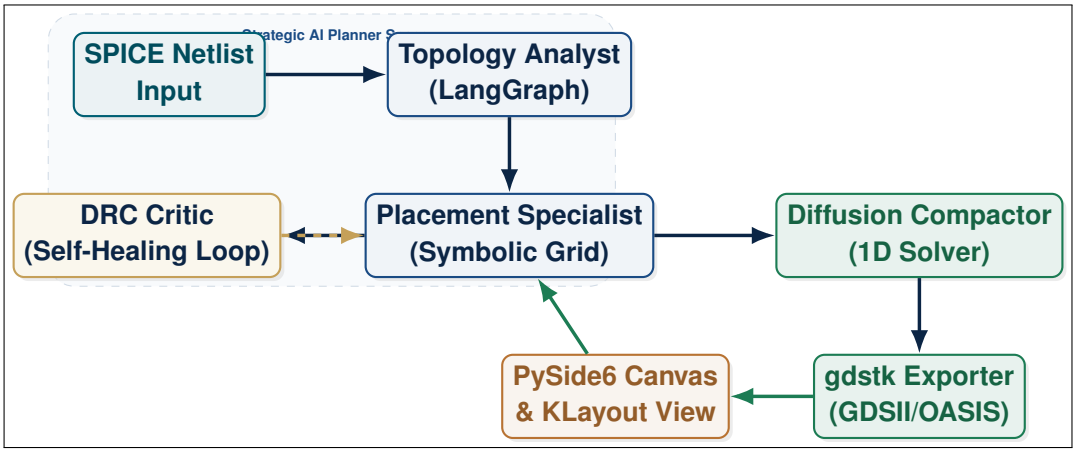


Figure 3.1: Detailed Dataflow and Control Loop of the Decoupled AI Layout Compiler.

3.1.1 Stage 1: Input Ingestion & PDK Parsing

The compilation starts with the ingestion stage. The parser reads standard SPICE netlists and extracts net topologies, device characteristics (widths, lengths, finger counts), and PDK template options. It resolves these parameters into a unified database. A graph isomorphism algorithm detects symmetry structures like differential pairs and current mirrors, mapping them to hierarchical constraint sets. Figure 3.2 details this parsing pipeline.

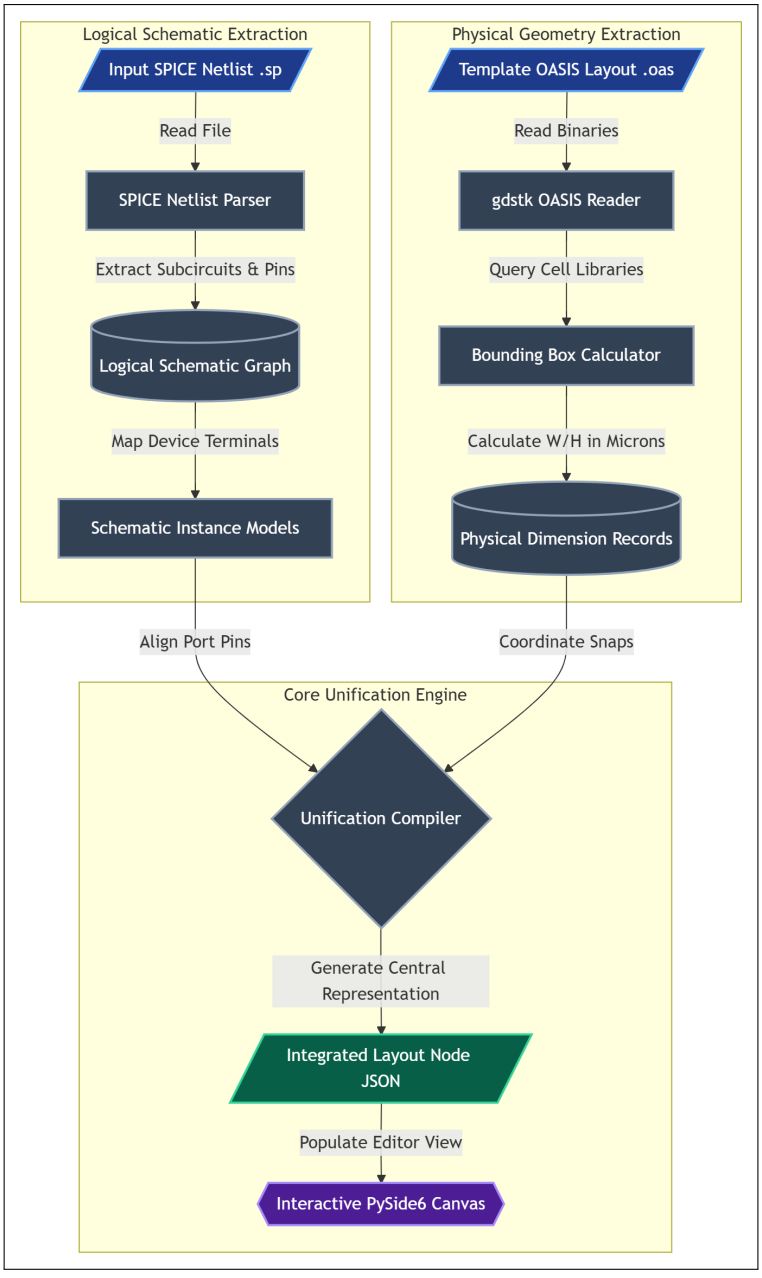


Figure 3.2: Detailed flowchart of Stage 1 (Input Ingestion and Parser Flow) mapping schematics to PDK-compatible nodes.

3.2 Layer 2: Strategic AI Placement Layer (LangGraph State Machine)

The strategic planning of the layout is governed by a multi-agent system implemented using LangGraph. LangGraph enforces a structured state-machine flow, which ensures that the agents collaborate in a predictable, self-healing loop. We define the graph state S mathematically as a tuple of four elements:

$$S = \langle P, C, F, H \rangle \tag{3.4}$$

where P is the set of current symbolic device placements, C represents the active constraints, F represents the history of critic feedback, and H represents human overrides.

3.2.1 Agent Prompt Design and Instructions

Each agent within the LangGraph layer is driven by specific system prompts that define their role and scope:

- **Topology Analyst Agent Prompt:**

  "You are an expert Analog IC Topology Analyst. Your task is to parse the input SPICE netlist and detect sub-block hierarchies. Identify differential pairs, current mirrors, and matched resistor/capacitor arrays. Output a structured constraint set containing symmetry pairings and matching groups. Do not define absolute physical coordinates."

- **Placement Specialist Agent Prompt:**

  "You are an Analog Layout Placement Specialist. You receive a list of devices, a set of constraints (symmetry, alignment, abutment), and a symbolic placement grid. Your task is to assign each device to a row index and slot index on the symbolic grid. Respect symmetry axes by placing paired devices symmetrically around the center slot. Output only the updated symbolic grid in JSON format."

- **DRC Critic Agent Prompt:**

  "You are a DRC Critic. Analyze the candidate symbolic placement. Check for adjacent spacing clearance, guard ring enclosures, and symmetry axis alignments. If a violation is detected, calculate the offset error and write an actionable feedback instruction (e.g., 'Move MM1 by +1 slot' or 'Insert dummy cell at slot 2'). if no violations exist, mark the state as DRC-clean."

The feedback loop routes execution between the Placement Specialist and the DRC Critic. The loop terminates when the DRC Critic returns a "DRC-clean" status, or when the iteration count exceeds a pre-set maximum ($T_{max} = 5$). When T_{max} is exceeded, the system prompts the human designer in the chatbot panel to resolve the conflict.

Figure 3.3 illustrates the state transitions and agent routing rules of this multi-agent loop.

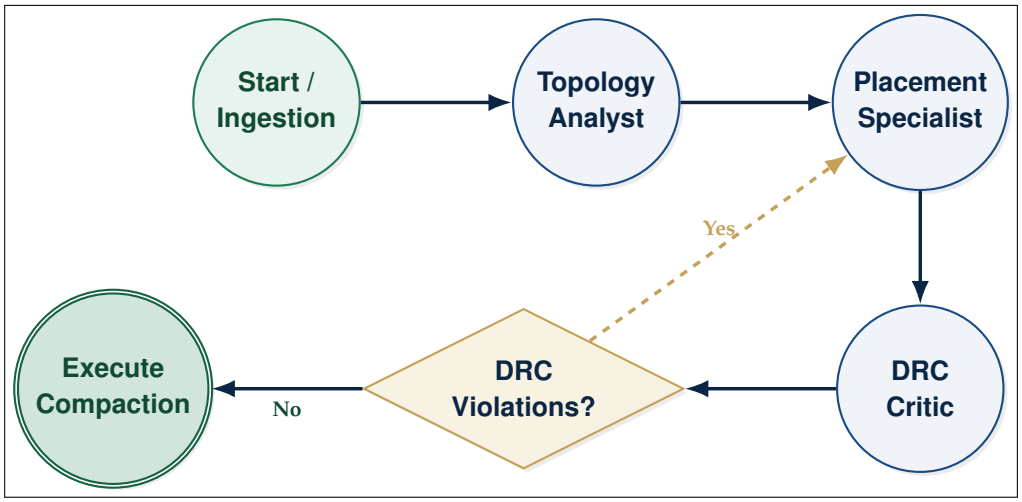


Figure 3.3: LangGraph Agent State Transition and Feedback Loop.

3.2.2 LangGraph Agent Collaborative Flow

The multi-agent loop orchestrates the interaction between the Topology Analyst, Placement Specialist, and DRC Critic. The Placement Specialist proposes symbolic slot configurations, which are evaluated by the DRC Critic. Spacing and overlap violations are fed back as text prescriptions (e.g., "Shift MM1 by +1 slot") to guide the next iteration. Figure 3.4 shows the detailed decision and state flow within the placement agent.

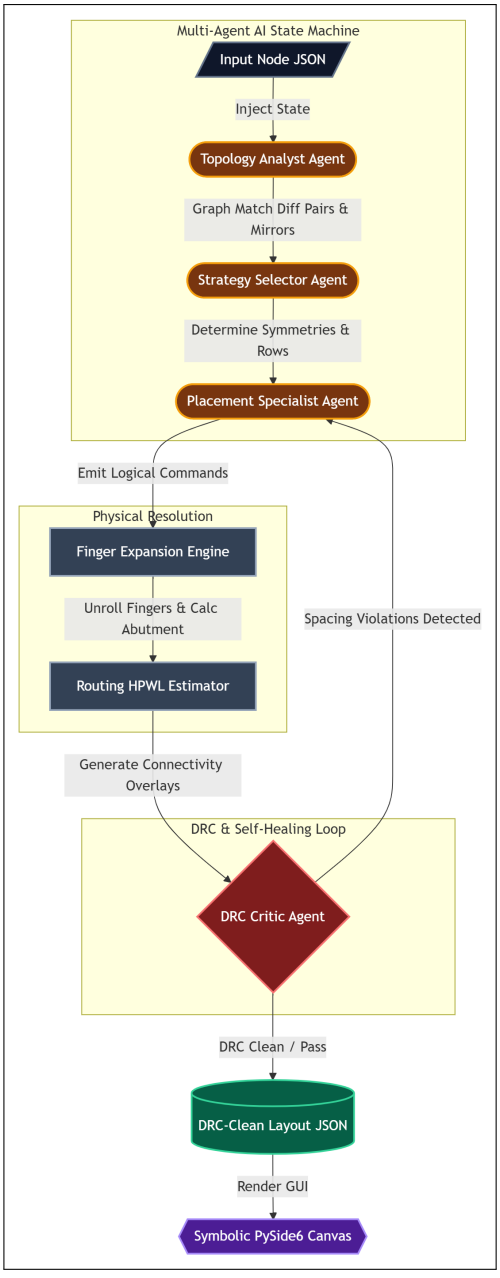


Figure 3.4: Detailed flowchart of Stage 2 (LangGraph AI Placement) showing the self-healing Placer-Critic loop.

3.3 Layer 1: User Interaction and Visualization GUI

The primary interface for the designer is a graphical editor built with PySide6 (Python bindings for Qt6). The interface consists of a dynamic schematic/layout viewer canvas, a design hierarchy tree, a parameter configuration panel, and an embedded chatbot panel for AI co-pilot interaction.

The PySide6 application implements the Model-View-Controller (MVC) design pattern:

- **Model:** Exposes the central JSON-based intermediate data representation representing the layout state.
- **View:** Rendered using a 'QGraphicsView' and 'QGraphicsScene' canvas. Devices are represented as custom graphical rect items ('QGraphicsRectItem') that designers can drag, drop, and rotate.
- **Controller:** Handles event callbacks and coordinates updates between the graphical view, local tools, and the background compilation workers.

To maintain a highly responsive user interface, the application uses an asynchronous threading model. Running LLM inference, parsing netlists, and invoking compilation pipelines can take from several hundred milliseconds to tens of seconds. If these tasks were executed on the main Qt GUI thread, the interface would freeze, leading to a poor user experience.

We address this by separating the UI event loop from execution. The GUI thread handles only user gestures, coordinate drawing, and screen updates. All AI reasoning, physical compaction, and database operations are delegated to worker classes running on separate QThreads. Communication between the GUI thread and the QThreads is managed using Qt's thread-safe signal-and-slot mechanism, as shown in Figure 3.5.

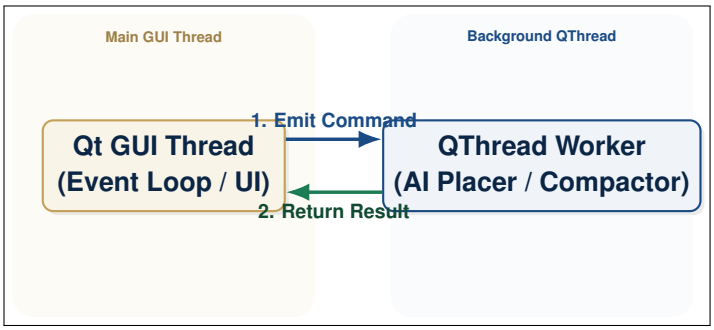


Figure 3.5: Asynchronous Threading Architecture using Qt Signals/Slots.

The application also integrates with KLayout via a local TCP socket listener. When the compaction engine generates a new layout draft, it streams the geometry files (GDSII) and sends a reload command via the TCP socket. The running KLayout instance immediately refreshes its view, allowing the designer to inspect the physical GDSII layout side-by-side with the symbolic editor.

3.3.1 Stage 3: Interactive Chatbot Co-Pilot Loop

The interactive review stage enables human-in-the-loop layout design. The user enters natural language instructions (e.g., "align MM1 and MM2 horizontally") into the PySide6 chatbot panel. The chatbot model translates these inputs into formal XML actions, which are queued and executed by the controller to update the canvas. The user can visually inspect the changes and trigger undo/redo actions dynamically. Figure 3.6 shows the co-pilot loop.

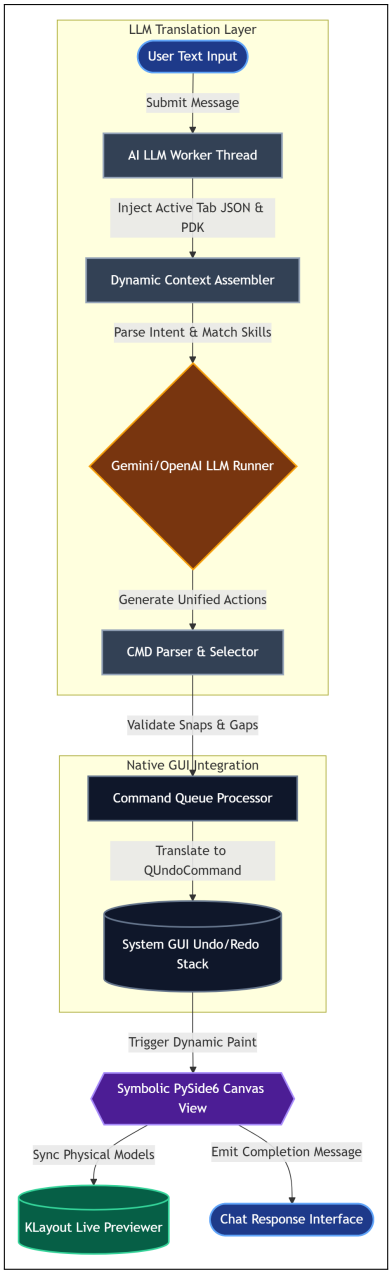


Figure 3.6: Flowchart of Stage 3 (Chatbot Co-Pilot) showing the interactive human-in-the-loop layout editing flow.

3.4 Layer 3: Deterministic Physical Compaction & Extraction Engine

3.4.1 Stage 4: Physical Compaction (Exporter)

The final stage compiles the symbolic grid layouts into physical GDSII/OASIS files or injects coordinates directly into commercial databases. The physical compaction solver constructs directed constraint graphs, snapped to grid tracks, and applies diffusion-sharing rules to merge adjacent transistors. Figure 3.7 details the Stage 4 physical backend.

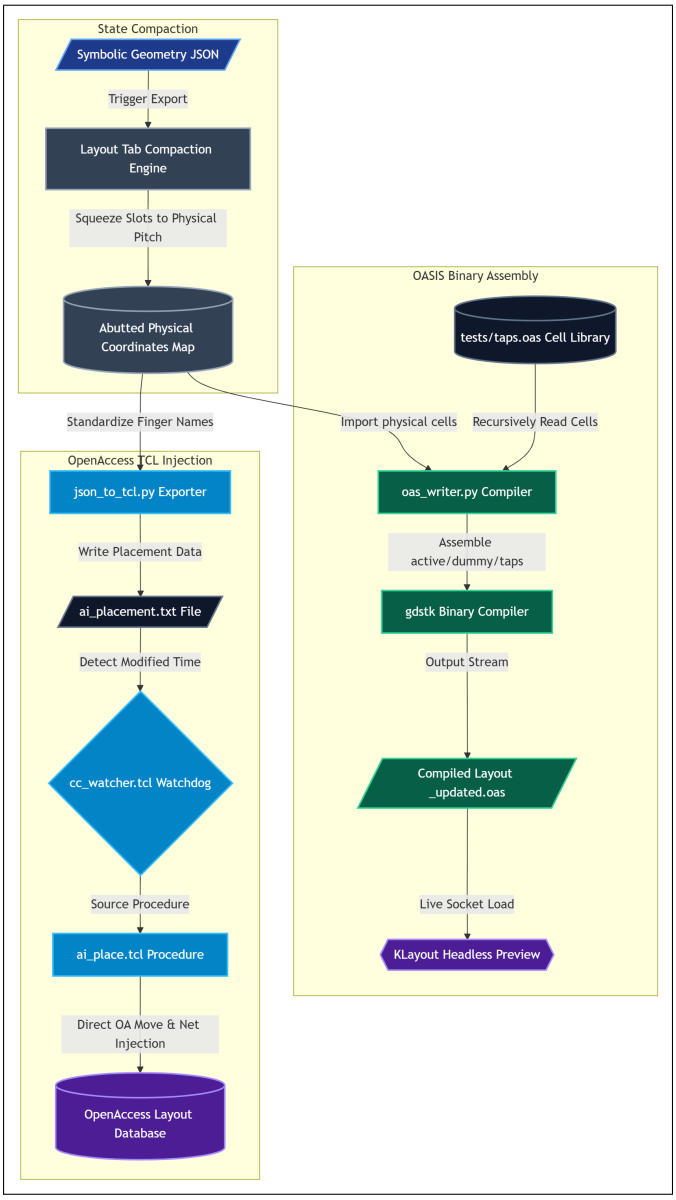


Figure 3.7: Flowchart of Stage 4 (Compaction & Export Pipelines) compiling symbolic slots into DRC-clean layout streams.

3.4.2 1D Compaction using Constraint Graphs

Layout compaction is formulated as a directed constraint graph $G_c = (V_c, E_c)$. Let the vertices V_c correspond to the vertical boundaries of layout polygons, and the edges E_c correspond to minimum spacing design rules. Let x_i be the physical coordinate of boundary edge i. A spacing rule requiring a minimum distance d_{ij} between edge i and edge j is modeled as $x_j - x_i \geq d_{ij}$.

The compilation steps for resolving the constraint graph are detailed in Algorithm 3.4.2.

Algorithm 3.1: 1D Compaction Solver

Given a set of boundary edges V_c and spacing constraints E_c:

1. Initialize coordinates: $x_i \leftarrow 0$ for all $i \in V_c$.
2. Sort vertices topologically since G_c is a Directed Acyclic Graph (DAG).
3. For each vertex u in topological order:
4. For each outgoing edge $(u, v) \in E_c$ with weight d_{uv}:
5. Update coordinate: $x_v \leftarrow \max(x_v, x_u + d_{uv})$.
6. Check for positive cycles (conflicting rules) using Bellman-Ford if sorting fails.
7. Return coordinates $\mathbf{x}$.

3.4.3 Diffusion-Sharing Optimization

To minimize layout area and reduce parasitic capacitance, the compaction engine includes a diffusion-sharing optimization stage. If two adjacent transistors M_1 and M_2 in the same row share an electrical net connection at their adjacent source/drain terminals, the engine merges their active areas (OD diffusion layers), removing the oxide isolation (STI) trench between them. The physical gate-to-gate spacing is reduced from the standard oxide-enclosure rule to the shared diffusion spacing rule. The merging logic is formalized in Algorithm 3.4.3.

Algorithm 3.2: Diffusion-Sharing Optimizer

Given a row of placed devices $Row = [M_1, M_2, \ldots, M_k]$:

1. Initialize $i \leftarrow 1$.
2. While $i < k$:
3. Get adjacent terminals: $T_r \leftarrow Terminal_{\text{right}}(M_i)$, $T_l \leftarrow Terminal_{\text{left}}(M_{i+1})$.
4. If $Net(T_r) == Net(T_l)$ and M_i, M_{i+1} share the same active diffusion type (both NMOS or both PMOS):
5. Mark boundary: $Merge(M_i, M_{i+1}) \leftarrow$ True.
6. Update spacing constraint: $d_{i,i+1} \leftarrow S_{\text{gap}}$.
7. Else:
8. Mark boundary: $Merge(M_i, M_{i+1}) \leftarrow$ False.
9. Update spacing constraint: $d_{i,i+1} \leftarrow 2 \cdot L_{\text{enclosure}} + S_{\text{STI}}$.
10. Increment $i \leftarrow i + 1$.

This merging reduces the junction area A_d and perimeter P_d, which in turn minimizes the parasitic junction capacitance:

$$C_{\text{db}} = A_d \cdot C_{\text{ja}} + P_d \cdot C_{\text{jp}} \tag{3.5}$$

by up to 40%, improving circuit speed and matching performance.

3.5 Central Intermediate Data Representation (JSON Schema)

The central source of truth shared between the frontend GUI, the LangGraph agents, and the physical exporters is the layout node JSON representation. This schema models transistors, dummies, and taps.

Listing 3.5 shows a sample transistor representation within this central data model, highlighting the separation between logical connectivity (`electrical`) and physical properties (`geometry`).

Sample Transistor Entry in Layout JSON Schema

```
{
  "id": "MM1",
  "type": "nmos",
  "is_dummy": false,
  "dummy_source": null,
  "electrical": {
    "gate": "NET_INP",
    "source": "NET_TAIL",
    "drain": "NET_OUTN",
    "bulk": "NET_VSS"
  },
  "geometry": {
    "x": 0.0,
    "y": 0.0,
    "width": 0.588,
    "height": 0.840,
    "orientation": "R0",
    "multiplier": 2,
    "nf": 2
  },
  "template_layout_index": 0,
  "layout_cell": "nfet_01v8"
}
```

In addition to active devices, dummy transistors and substrate/well taps are modeled in the schema to ensure consistent physical boundary rules. Listing 3.5 shows a sample dummy device configuration.

Sample Dummy Transistor Entry in JSON Schema

```
{
  "id": "MDUMMY1",
  "type": "nmos",
  "is_dummy": true,
  "dummy_source": "NET_VSS",
  "electrical": {
    "gate": "NET_VSS",
    "source": "NET_VSS",
    "drain": "NET_VSS",
    "bulk": "NET_VSS"
  },
  "geometry": {
    "x": -1.2,
    "y": 0.0,
    "width": 0.588,
    "height": 0.840,
    "orientation": "R0",
    "multiplier": 1,
    "nf": 2
  },
  "template_layout_index": 0,
  "layout_cell": "nfet_01v8"
}
```

Key fields in this schema are:

- **is_dummy:** A boolean indicating if the device is active or a dummy padding transistor.
- **electrical:** Connects the terminals (Gate, Source, Drain, Bulk) to logical net names. This is used by the router and DRC checkers.
- **geometry.nf:** Number of gate fingers. This dictates how the unrolling engine expands the device.

3.6 Model Context Protocol (MCP) Server Integration

To support external LLM clients, the architecture integrates a Model Context Protocol (MCP) server. MCP is a stdio-based communication protocol that allows external models (e.g., Claude Desktop) to invoke local tools and access files securely. The MCP server exposes a tool registry, including tools such as `place_device`, `align_devices`, and `run_drc`.

The communication uses JSON-RPC 2.0 packets over standard input/output. Listing 3.6 shows an example of a tool execution exchange where the client invokes the `place_device` tool.

Sample MCP JSON-RPC Exchange

```
// Client Request
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "place_device",
    "arguments": {
      "device_id": "MM1",
      "row": 0,
      "slot": 1
    }
  },
  "id": 1
}

// Server Response
{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Successfully placed MM1 at row 0, slot 1.
            File layout_state.json updated."
      }
    ]
  },
  "id": 1
}
```

When a tool is executed, the server updates a shared state file (`layout_state.json`). The PySide6 application monitors this file and immediately updates the graphical canvas. This synchronization enables a real-time, interactive co-pilot design loop.

3.7 Security Boundaries and Sandbox Isolation in MCP

Security Boundaries and Sandbox Isolation

Integrating Large Language Models with local CAD tools poses security risks. Because LLMs can generate arbitrary code and execute tool calls, exposing direct system access (e.g., bash terminals or system commands) to the model could lead to malicious or unintended actions, such as overwriting critical PDK source files or deleting design databases.
To mitigate these risks, the proposed architecture implements strict security boundaries:

1. **Stdio Isolation:** The MCP server communicates with the LLM client exclusively through standard input/output (stdio). There is no direct network socket or remote execution channel between the external client and the local system, limiting the attack surface.

2. **Schema-based Tool Validation:** The LLM cannot execute arbitrary scripts on the host system. It is restricted to the specific API schema defined in the tool registry (e.g., placing cells or running DRC checks). The MCP server acts as a gatekeeper, parsing and validating the JSON parameters before translating them into physical commands:

$$\mathbf{x}_{\text{args}} \in \text{Registry\_Schema} \implies \text{Execute Tool}(\mathbf{x}_{\text{args}}) \tag{3.6}$$

   If the parameters do not match the expected schema types, the command is immediately rejected with a JSON-RPC error.

3. **File-system Sandboxing:** The MCP server is confined to the active project workspace directory. It cannot read or write files outside this sandbox, protecting the user's home directories and PDK system files.

This sandboxed integration ensures that the AI co-pilot operates safely, protecting designer databases and PDK installations.

CHAPTER 4

Stage 1: Input Ingestion & Graph Unification

The ingestion pipeline acts as the primary compiler front-end of the LayoutCopilot framework. In analog integrated circuit (IC) design, the logical schematic representation and the physical layout representation exist as distinct, decoupled abstractions. The schematic netlist details the logical connectivity of devices, whereas the layout stream (GDSII or OASIS) specifies physical boundary geometries, layers, and coordinate grids. To enable autonomous placement agents to reason about physical design structures while preserving schematic intent, the compiler front-end must merge these two representations into a unified mathematical structure: a unified graph representation.

This chapter details the design, algorithmic framework, and implementation of the Input Ingestion and Graph Unification stage. It covers the automated CAD extraction scripting using Synopsys Custom Compiler Tcl commands, the recursive SPICE netlist parsing and hierarchy flattening, the PDK-aware binary property extraction from OASIS layout files, the spatial-logical device matching algorithms, and the unified NetworkX-based graph representation.

4.1 EDA CAD Custom Compiler Scripting & Input Extraction

The physical and logical inputs used by the compiler are extracted directly from the Synopsys Custom Compiler database. Custom Compiler exposes a Tcl interface with namespaces for database management (`db::`) and graphical user interface automation (`gi::`). The extraction process is orchestrated by a master manager script, `run.tcl`, which triggers the schematic netlist extraction (`extract_net.tcl`) and physical layout extraction (`extract_oasis.tcl`) depending on the user's execution flags.

Programmatic database control is vital for high-throughput layout compilers. Rather than relying on manual exports, which are error-prone and time-consuming, LayoutCopilot wraps Custom Compiler APIs. The `db::` namespace provides a direct link to the OpenAccess (OA) database, while the `gi::` namespace allows the compiler to programmatically open dialogues, change inputs, and press buttons, simulating user clicks inside the Custom Compiler interface.

4.1.1 Schematic Netlist Extraction

The logical schematic netlist must capture all device parameters, including device type, channel width (W), channel length (L), finger counts (nf), and multi-finger multipliers (m). The script `extract_net.tcl` automates this by launching the netlister dialogue, configuring target view options, and generating a CDL-compatible netlist file (`.sp`).

Synopsys Custom Compiler Netlist Extraction Script (extract_net.tcl)

```
set netlistFile "$cell.sp"

set netlist_dir [file join $EDA_TOOLS_DIR "netlist"]
file mkdir $netlist_dir

db::showExportNetlist
gi::setActiveDialog [gi::getDialogs {runNetlister}]
db::setAttr geometry -of [gi::getDialogs {runNetlister}] -
   value 488x465+610+181
gi::setField {/topTabGroup/mainTab/design/
   schCellViewLibrary} -value $lib -in [gi::getDialogs {
   runNetlister}]
gi::setField {/topTabGroup/mainTab/design/schCellViewCell}
    -value $cell -in [gi::getDialogs {runNetlister}]
gi::setField {/topTabGroup/mainTab/design/schCellView} -
   value {schematic} -in [gi::getDialogs {runNetlister}]
gi::setField {/topTabGroup/mainTab/design/netlistFile/
   entryField} -value $netlistFile -in [gi::getDialogs {
   runNetlister}]
gi::setField {/topTabGroup/mainTab/design/netlistDir/
   entryField} -value $netlist_dir -in [gi::getDialogs {
   runNetlister}]
gi::pressButton {/ok} -in [gi::getDialogs {runNetlister}]
```

4.1.2 Physical Layout Stream Extraction

The physical layout stream contains the placement coordinate markers and PCell boundaries. The script `extract_oasis.tcl` extracts layout geometry files in the OASIS (`.oa`) format, which has higher data compression rates than GDSII.

OASIS Stream Extraction Script (extract_oasis.tcl)

```
set oasisFile "$cell.oa"
set parent_dir [file dirname $EDA_TOOLS_DIR]
set oasis_dir [file join $EDA_TOOLS_DIR "oasis"]
file mkdir $oasis_dir

```

```
set BASE_DIR [file join $parent_dir $lib $cell]
set layout_dir [file join $BASE_DIR "layout"]
set config_dir [file join $BASE_DIR "layout#2econfig"]

if {[file isdirectory $layout_dir] && [file isdirectory
   $config_dir]} {
    set target_oa_file [file join $oasis_dir $oasisFile]
    if {[file exists $target_oa_file]} {
        file delete $target_oa_file
    }
    db::showExportOasis
    gi::setActiveDialog [gi::getDialogs {dbExportOasis}]
    db::setAttr geometry -of [gi::getDialogs {
       dbExportOasis}] -value 685x634+416+75
    gi::setField {libName} -value $lib -in [gi::getDialogs
        {dbExportOasis}]
    gi::setField {topCellName} -value $cell -in [gi::
       getDialogs {dbExportOasis}]
    gi::setField {runDirectory} -value $oasis_dir -in [gi
       ::getDialogs {dbExportOasis}]
    gi::setField {fileName} -value $oasisFile -in [gi::
       getDialogs {dbExportOasis}]
    gi::pressButton {ok} -in [gi::getDialogs {
       dbExportOasis}]
} else {
    echo "Error: One or both directories are missing."
}
```

4.2 Ingestion & Graph Unification Pipeline Architecture

Once the raw schematic and layout files are extracted, they are passed to the parser subsystem. The ingestion pipeline consists of four serial steps that convert logical netlists and layout geometries into a unified NetworkX graph. Figure 4.1 illustrates the pipeline block diagram.

The first stage parses raw files: the Netlist Reader parses schematic netlists into memory, while the Layout Reader walks the OASIS layout database cell tree. The Device Matcher maps devices between the two databases. The Circuit Graph Builder constructs the final graph, which contains spatial and electrical parameters.

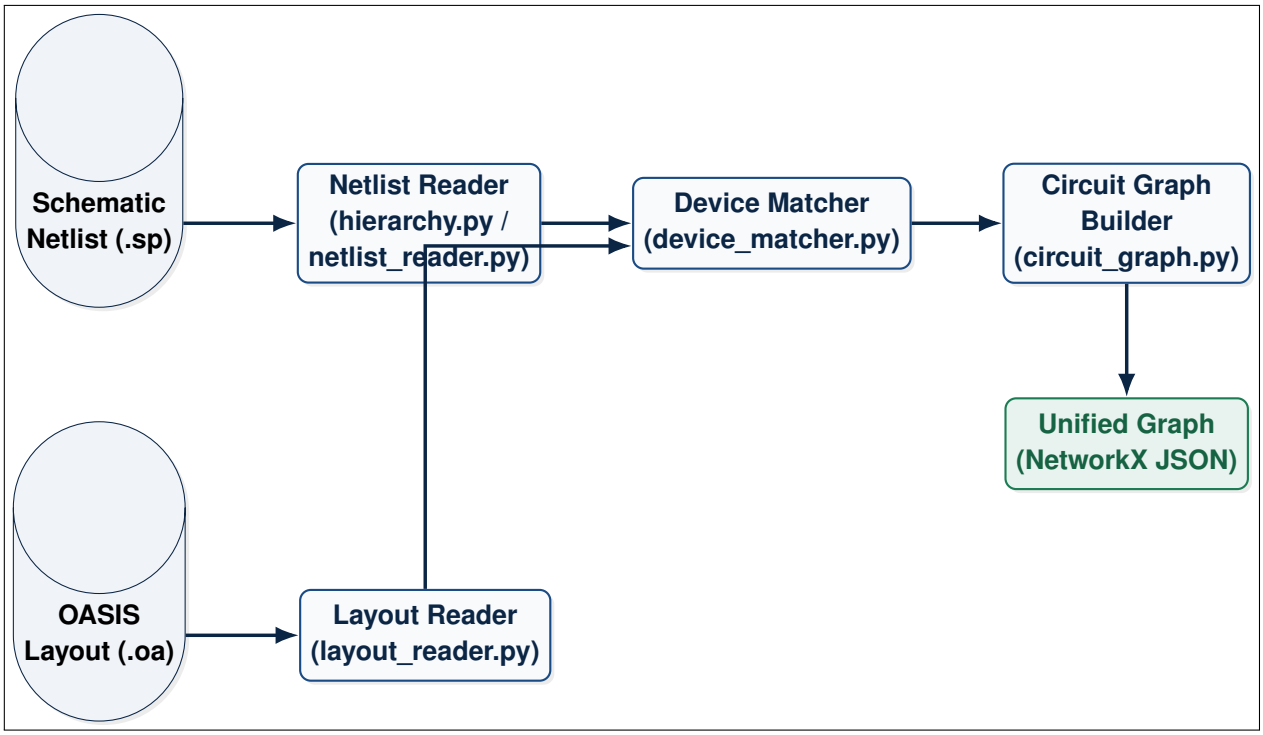


Figure 4.1: Detailed Ingestion Pipeline Flowchart mapping raw layout and SPICE inputs to a unified NetworkX model.

4.3 SPICE Netlist Parsing & Hierarchy Flattening

The SPICE reader (`netlist_reader.py`) parses text-based SPICE and CDL files. Schematic designs are typically hierarchical, utilizing subcircuits (`.subckt`) to encapsulate blocks like bias generators or operational amplifiers. Layout placing agents, however, operate on a flat coordinate space where each physical transistor finger must be resolved individually. The parser resolves this by executing two operations: subcircuit flattening and parameter expansion.

During layout placement, the agent needs to allocate specific physical slots for each transistor finger. If the netlist remains hierarchical, the spatial relationships between components in different hierarchical blocks are hidden from the solver, making absolute positioning, row/slot alignment, and sharing diffusions across hierarchical boundaries extremely complex. Therefore, the compiler flattens the hierarchy and resolves replication down to leaf fingers before placement.

4.3.1 Recursive Subcircuit Expansion

The flattening engine (`hierarchy.py`) parses the subcircuit definitions first. It identifies the top-level subcircuit candidate based on name matching or instantiation tree analysis. The engine then recursively expands all instance calls (`X`-instances) down to leaf primitives (transistors: `M`, resistors: `R`, capacitors: `C`).

During expansion, the ports of the subcircuit are mapped to the net names

passed by the parent instance. Internal nets are prefixed with the instance names (e.g., `XI0_net3`) to avoid namespace collisions. The mathematical mapping of internal nets is defined as:

$$N_{\text{flat}}(net) = \begin{cases} parent\_net & \text{if } net \in Ports(subckt) \\ prefix \,\|\, net & \text{otherwise} \end{cases} \tag{4.1}$$

where $\|$ represents string concatenation, and $prefix$ is the hierarchical path prefix.

4.3.2 Replication & Parameter Expansion

Replicated transistors can be specified in SPICE through multiple options:

1. **Array Suffixes:** E.g., `MM9<7>` refers to the 8th copy (0-based index 7) of device `MM9`.
2. **Multipliers (m):** E.g., `m=8` instantiates 8 identical parallel devices.
3. **Fingers (nf):** E.g., `nf=10` splits a single device width into 10 adjacent fingers.

To match layout stream cells (where fingers and multipliers occupy individual physical locations), the parser expands a single logical SPICE device statement with $m = M$ and $nf = F$ into $M \times F$ independent leaf device instances. The leaf devices are named using a systematic indexing scheme:

$$Name_{\text{leaf}} = \{Parent\}\_m\{i\}\_f\{j\} \quad \forall i \in \{1, \ldots, M\}, j \in \{1, \ldots, F\} \tag{4.2}$$

This ensures that every physical finger in the layout corresponds to a unique node in the parsed netlist model.

Subcircuit Expansion and Parameter Parsing (netlist_reader.py)

```
def expand_instance(line, subckts, prefix=""):
    """Expand a single X-instance line into its
       constituent device lines.
    Recursively expands if the subcircuit contains further
        X-instances.
    """
    tokens = line.split()
    inst_name = tokens[0]           # e.g. XI0
    nets = tokens[1:-1]             # actual net
       connections
    subckt_name = tokens[-1]        # subcircuit being
       instantiated

    if subckt_name not in subckts:
        return []
```

```
12
13      subckt = subckts[subckt_name]
14      port_map = dict(zip(subckt.ports, nets))
15      full_prefix = f"{prefix}{inst_name}_" if prefix else f
           "{inst_name}_"
16      expanded = []
17
18      for internal_line in subckt.lines:
19          parts = internal_line.split()
20          if not parts:
21              continue
22          devname = parts[0]
23          if devname.startswith("*") or devname.startswith("
               ."):
24              continue
25
26          if devname[0].upper() == "X":
27              remapped_parts = [f"{full_prefix}{devname}"]
28              for i in range(1, len(parts)):
29                  token = parts[i]
30                  if token in port_map:
31                      remapped_parts.append(port_map[token])
32                  else:
33                      remapped_parts.append(f"{full_prefix}{
                           token}")
34              remapped_line = " ".join(remapped_parts)
35              expanded.extend(expand_instance(remapped_line,
                    subckts, prefix=""))
36          else:
37              parts[0] = f"{full_prefix}{devname}"
38              for i in range(1, len(parts)):
39                  token = parts[i]
40                  if token in port_map:
41                      parts[i] = port_map[token]
42                  elif "=" not in token:
43                      if i <= 4 or (i == 5 and not token[0].
                           isalpha()):
44                          parts[i] = f"{full_prefix}{token}"
45              expanded.append(" ".join(parts))
46      return expanded
```

4.4 Physical Layout Stream Parsing & PDK Ingestion

The physical layout stream parser (`layout_reader.py`) interfaces with the binary OASIS or GDSII file using the high-performance `gdstk` C++ library. The layout reader walks the hierarchical cell reference tree from the top cell down to

the leaf cells representing PDK parameterized cells (PCells) using a depth-first search (DFS) traversal.

OASIS files offer several advantages over traditional GDSII files. GDSII files are uncompressed and use fixed-size record blocks, which can lead to very large files for modern sub-micron designs. In contrast, OASIS uses byte-aligned variable-length records, coordinate offsets, and string/cell reference tables to achieve significant file-size reductions, allowing the compiler front-end to ingest complex layouts in milliseconds.

4.4.1 Device Classification

Cells are filtered by name heuristics to categorize their function:

- **Transistors:** Cells matching keywords `nfet`, `pfet`, `nmos`, `pmos`.
- **Passives:** Resistors (`rppoly`, `rnwell`) and capacitors (`ccap`, `mimcap`).
- **Dummies:** Cells designated as dummy structures (`dummy`, `filler`) used for density matching.
- **Vias & Taps:** Ignored for high-level device matching to minimize graph complexity.

4.4.2 PDK Parameter Property Decoding

PCell instances store their parameters (such as finger width, channel length, finger count, and diffusion abutment configurations) as binary key-value properties inside the GDSII/OASIS files. For our target PDK (SAED 14nm), these parameters are encoded in the `'pcell'` property block.

The parameters are written as string arrays where the parameter name and value are separated by a specific byte-signature marker: `"##32##"`. This serialization approach is common in PDK databases. The layout reader extracts these properties by parsing the raw byte stream, identifying the separator, and parsing the key-value attributes.

PDK PCell Parameter Property Decoding (layout_reader.py)

```
def _parse_pcell_params(*objs):
    """Parse parameters from multiple objects (e.g. Cell
       and Reference) and merge them.
    The SAED PDK encodes these as bytes in a 'pcell'
       property entry.
    Returns: dict of parameters {key: value_string}.
    """
    params = {}
    for obj in objs:
        if not hasattr(obj, "properties"):
            continue
        try:
```

```
            for prop in obj.properties:
                if not prop or len(prop) < 1:
                    continue
                key = prop[0]
                if isinstance(key, bytes):
                    key = key.decode("utf-8", errors="
                        ignore")
                if str(key).lower() != "pcell":
                    continue

                for entry in prop[1:]:
                    if isinstance(entry, (bytes, bytearray
                        )):
                        s = entry.decode("utf-8", errors="
                            ignore")
                    elif isinstance(entry, str):
                        s = entry
                    else:
                        continue

                    if "##32##" in s:
                        parts = s.split("##32##")
                        if len(parts) >= 3:
                            param_key = parts[0].strip()
                            val = parts[-1].strip()
                            params[param_key] = val
        except (AttributeError, IndexError, TypeError):
            pass
    return params
```

4.5 Deterministic Spatial-Logical Device Matching

A core challenge in layout automation front-ends is matching the parsed logical netlist devices with the extracted physical layout instances. Because the EDA tool's netlister and layout generator do not share a common naming database for leaf components (often naming them `MM0_f1` on one side and cell references `nfet_ref_0` on the other), a deterministic matching algorithm is required.

The matcher module (`device_matcher.py`) executes a three-phase alignment algorithm:

1. **Type-Based Partitioning:** Both netlist devices and layout instances are partitioned into separate queues based on device types (NMOS, PMOS, Resistors, Capacitors).

2. **Deterministic Sorting:**

- Netlist devices are sorted alphanumerically based on their hierarchical names (e.g., `MM0_m1_f1`, `MM0_m1_f2`, `MM1_f1`).
- Layout instances are sorted spatially based on their absolute (x, y) coordinates. The sorting prioritizes the horizontal axis first (left-to-right), followed by the vertical axis (bottom-to-top):

$$\text{Key}_{\text{layout}}(inst) = (x_{inst}, y_{inst}) \tag{4.3}$$

3. **Index-Based Mapping:**

- If the netlist device count equals the layout instance count, a 1-to-1 sequential mapping is established.
- If a mismatch exists due to finger grouping (where the netlist lists individual fingers but the layout exporter outputs a single multi-finger PCell block), the matcher groups netlist fingers belonging to the same logical parent device and maps them onto the shared layout instance.

Sorting logical netlist names alphanumerically is effective because SPICE netlist generators output leaf devices in a systematic order. Similarly, sorting layout devices spatially matches the horizontal rows common in analog designs (like differential pairs and current mirrors). This matching method aligns the queues without needing complex subgraph isomorphism solvers (which are NP-complete). The compiler also includes fallback checks to warn the designer if the device counts do not match.

Figure 4.2 illustrates this matching mechanism.

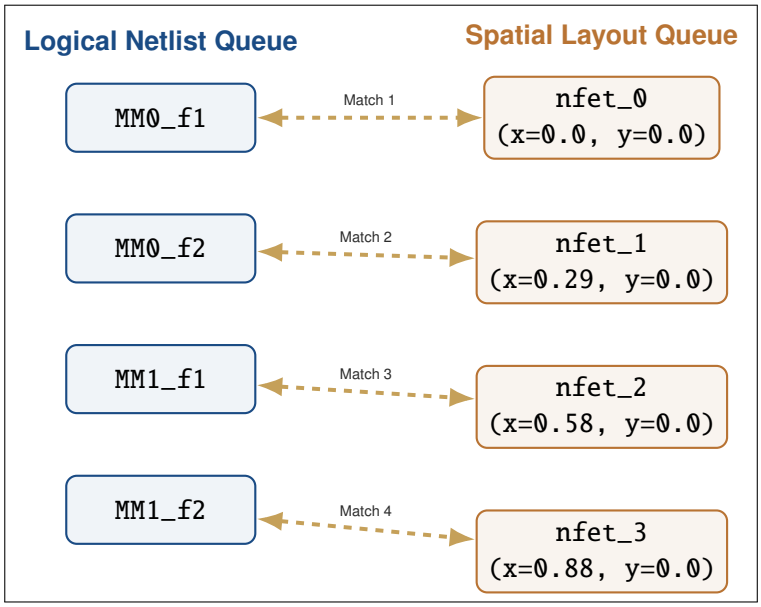


Figure 4.2: Deterministic Alignment between logical netlist devices and physical layout geometries.

4.6 Unified Circuit Graph Construction & Node/Edge Classification

The final step of Stage 1 is building the unified graph structure using the NetworkX library (`circuit_graph.py`). Let the unified graph be $G(V, E)$, where V represents the set of circuit device nodes and E represents the set of electrical net connections.

4.6.1 Node Properties

Each device node $v \in V$ combines attributes from both the netlist and the layout:

$$v = \langle \text{id}, \text{type}, \text{electrical\_params}, \text{geometric\_params} \rangle \tag{4.4}$$

where:

- `electrical_params` = $\{W, L, nf, nfin, parent, array\_index\}$.
- `geometric_params` = $\{x, y, width, height, orientation\}$.

4.6.2 Edge Relations and Classification

Edges represent physical wiring connections between device terminals (Gate, Drain, Source, Bulk). To guide placement heuristics, edges are classified into behavioral relations. First, global nets (such as `vdd`, `vss`, and `gnd`) are pruned from the edge list. Because power nets connect to almost every device bulk or source, including them would create a dense, low-diameter graph, masking critical signal and matching paths. Pruning well and bulk connections prevents the sparse circuit graph from becoming a dense clique, which would complicate graph-based placement reasoning.

For the remaining signals, the net is classified by counting the terminal types connected to it:

- **Bias Net:** If the net connects to ≥ 3 sources and 0 gates, it is tagged as a bias line.
- **Gate Net:** If it connects to ≥ 2 gates, it is a shared gate signal.
- **Signal Net:** If it connects to ≥ 2 drains and 0 gates, it is a signal path.

Based on this classification, the edges connecting device pairs v_1 and v_2 are assigned specific relationship tags:

1. **`shared_gate`:** Gates of v_1 and v_2 are connected. This indicates matching pairs (e.g. current mirror mirrors, differential input gates). LangGraph placement agents use these edges to enforce matching constraints.
2. **`shared_source`:** Sources of v_1 and v_2 are connected.
3. **`shared_drain`:** Drains of v_1 and v_2 are connected.
4. **`shared_bias`:** Sources of v_1 and v_2 connect to a common bias line.

Edge Classification and Graph Construction (circuit_graph.py)

```python
def classify_net(net, connections):
    source_count = 0
    gate_count = 0
    drain_count = 0
    for dev, pin in connections:
        role = normalize_pin(pin)
        if role == "source": source_count += 1
        if role == "gate":   gate_count += 1
        if role == "drain":  drain_count += 1

    if source_count >= 3 and gate_count == 0:
        return "bias"
    if drain_count >= 2 and gate_count == 0:
        return "signal"
    if gate_count >= 2:
        return "gate"
    return "other"

def add_net_edges(G, netlist):
    for net, connections in netlist.nets.items():
        if net.lower() in GLOBAL_NETS:
            continue
        net_type = classify_net(net, connections)
        for i in range(len(connections)):
            dev1, pin1 = connections[i]
            role1 = normalize_pin(pin1)
            for j in range(i+1, len(connections)):
                dev2, pin2 = connections[j]
                if dev1 == dev2:
                    continue
                role2 = normalize_pin(pin2)
                relation = "connection"

                if role1 == "source" and role2 == "source":
                    relation = "shared_bias" if net_type == "bias" else "shared_source"
                elif role1 == "gate" and role2 == "gate":
                    relation = "shared_gate"
                elif role1 == "drain" and role2 == "drain":
                    relation = "shared_drain"

                G.add_edge(dev1, dev2, relation=relation, net=net)
```

4.7 Input & Output Interface Schemas

To ensure interoperability, Stage 1 exposes structured inputs and outputs.

4.7.1 Input SPICE Schema

The input SPICE file consists of standard device lines. Transistors are defined starting with character `M`, specifying pins in order (Drain, Gate, Source, Bulk) followed by the PDK model name and key-value properties:

```
M{name} {Drain} {Gate} {Source} {Bulk} {model} w={width} l={length}
```

4.7.2 Output Graph JSON Schema

The unified output graph is serialized into a JSON structure, defining both nodes (with physical and electrical profiles) and edges (with relation tags).

Output Graph JSON Schema Structure


```
{
  "nodes": [
    {
      "id": "MM2_f1",
      "type": "nmos",
      "electrical": {
        "l": 1.4e-08,
        "nf": 1,
        "nfin": 2.0,
        "w": 0,
        "parent": "MM2",
        "m": 1,
        "multiplier_index": null,
        "finger_index": 1,
        "array_index": null
      },
      "geometry": {
        "x": 0.0,
        "y": 0.668,
        "width": 0.294,
        "height": 0.668,
        "orientation": "R0"
      }
    }
  ],
  "edges": [
    {
      "source": "MM0_f1",
      "target": "MM1_f1",
```

```
        "relation": "shared_gate",
        "net": "C"
      }
    ]
}
```

4.8 Example Walkthrough: Current Mirror Ingestion

To demonstrate the parsing and graph construction flow, this section traces the ingestion of the current mirror circuit (`CM`) from the `examples/current_mirror` directory.

4.8.1 Input Schematic File

The input file `Current_Mirror_CM.sp` defines a symmetric current mirror. The schematic netlist contains NMOS and PMOS groups:

Current Mirror SPICE Input File (Current_Mirror_CM.sp)

```
* Library: Current_Mirror  Cell: CM  View: schematic
.subckt CM A B C VDD X Y Z gnd
MM2 A C gnd gnd n08 l=0.014u nf=4 nfin=2
MM1 B C gnd gnd n08 l=0.014u nf=8 nfin=2
MM0 C C gnd gnd n08 l=0.014u nf=16 nfin=2
MM5 Z X VDD VDD p08 l=0.014u nf=4 m=1 nfin=2
MM4 Y X VDD VDD p08 l=0.014u nf=8 m=1 nfin=2
MM3 X X VDD VDD p08 l=0.014u nf=16 m=1 nfin=2
.ends CM
```

4.8.2 Ingestion Processing Steps

1. **Flattening & Expansion:** The schematic is already flat, but devices have finger counts. The device `MM2` ($nf = 4$) is expanded into four leaf devices: `MM2_f1`, `MM2_f2`, `MM2_f3`, and `MM2_f4`.

2. `gdstk` Layout Processing: The layout reader parses the corresponding OASIS layout file `Current_Mirror.oas`. It extracts physical dimensions (e.g. height, width) and positions.

3. **Sorting and Matching:**

   - Netlist devices sorted: `MM0_f1` to `MM0_f16`, `MM1_f1` to `MM1_f8`, `MM2_f1` to `MM2_f4`.
   - Layout instances sorted spatially.

- A 1-to-1 matching matches logical nodes with their corresponding layout geometries.

4. **Graph Building:** Global nets `gnd` and `VDD` are pruned. The net `C` connects the gates of `MM0`, `MM1`, and `MM2`. Since this connects multiple gates, it is classified as a `shared_gate` relation, establishing edges between the devices.

4.8.3 Output Compressed Graph JSON

The resulting unified graph is stored in the compressed graph output structure, capturing node configurations and connections (truncated sample):

Output Graph JSON (Current_Mirror_CM_graph_compressed.json)

```
{
    "version": "2.0",
    "device_types": {
        "pmos": {
            "y_row": 0.668,
            "default_width": 0.294,
            "default_height": 0.818
        },
        "nmos": {
            "y_row": 0.0,
            "default_width": 0.294,
            "default_height": 0.668
        }
    },
    "devices": {
        "MM2": {
            "type": "nmos",
            "m": 1,
            "nf": 1,
            "nfin": 2.0,
            "l": 1.4e-08,
            "terminal_nets": {
                "D": "A",
                "G": "C",
                "S": "gnd"
            }
        },
        "// ... [MM0, MM1, MM3, MM4, MM5 entries truncated
            for brevity] ... //"
    },
    "connectivity": {
        "nets": {
            "C": [
```

```
                    "MM0",
                    "MM1",
                    "MM2"
                ],
                "// ... [other net connectivity entries] ...
                   //"
            }
        }
}
```

This structured data is then sent to Stage 2, allowing AI placing agents to construct placements on a row/slot grid while respecting electrical relations like gate matching.

CHAPTER 5

Stage 2: LangGraph-Driven Multi-Agent Placer

The initial placement engine forms the computational core of the LayoutCopilot compiler's second stage. Placement of sub-micron analog components is a highly constrained optimization problem. While traditional placement tools rely purely on simulated annealing or analytical numerical formulations, they struggle to capture the complex, qualitative matching and symmetry constraints that human layout experts apply by intuition. LayoutCopilot resolves this by adopting a decoupled multi-agent architecture orchestrated by a LangGraph state machine.

This chapter details the design, agent roles, and state transitions of the Stage 2 initial placement engine. It explains the shared state representation, the implementation details of each processing stage with code listings, the DRC self-healing feedback loop, and walks through a concrete current mirror and dynamic latch comparator placement example, detailing the exact outputs in the execution logs.

5.1 LangGraph State Machine Architecture

The multi-agent placer is built on LangGraph, a state-machine framework that extends LangChain to support cyclic graphs, agent loops, and structured state transitions. By modeling the placement process as a graph of cooperating nodes, LayoutCopilot isolates specialized layout tasks (topology analysis, placement strategy selection, grid-based component positioning, physical finger unrolling, routing cost evaluation, and DRC checks) into discrete agents.

5.1.1 LayoutState Shared State Definition

All agents in the graph read from and write to a shared, mutable database structure called the `LayoutState`. This state tracks design data, inputs, constraint strings, placement results, and design rule flags.

LayoutState Shared State Dictionary Structure (state.py)

```
from typing import TypedDict, List, Dict, Any, Literal

class LayoutState(TypedDict):
```

```
    mode: Literal["initial", "chat"]        # "initial" for Ctrl+P, "chat" for bot
    user_message: str
    chat_history: List[Dict[str, str]]
    nodes: List[Dict[str, Any]]             # Original layout device list
    sp_file_path: str
    selected_model: str

    constraint_text: str                    # Symmetry constraint blocks
    edges: List[Dict]                       # Schematic connectivity list
    terminal_nets: Dict[str, Dict[str, Any]]

    Analysis_result: str                    # Output of topology analyst
    strategy_result: str                    # Output of strategy selector

    placement_nodes: List[Dict]             # Updated device placement positions
    deterministic_snapshot: List[Dict]
    original_placement_cmds: List[Dict]

    drc_flags: List[Dict]                   # DRC violation reports
    drc_pass: bool
    drc_retry_count: int
    gap_px: float

    routing_pass_count: int
    routing_result: Dict[str, Any]
    placement_quality: Dict[str, Any]       # Bounding box / wire-length scores
    placement_mode: str                      # "auto" | "two_half"
    groups: List[Dict[str, Any]]             # Aggregated matching groups
```

5.1.2 State Graph Construction & Flow Control

The graph builder (`builder.py`) instantiates the state machine, registers the execution nodes, and defines transition edges. In the initial placement mode (`"initial"`), the pipeline runs as a linear flow with a conditional self-healing loop around the DRC Critic. Figure 5.1 illustrates the state transitions, and Figure 5.2 presents the detailed architectural block diagram of the placement

stage.

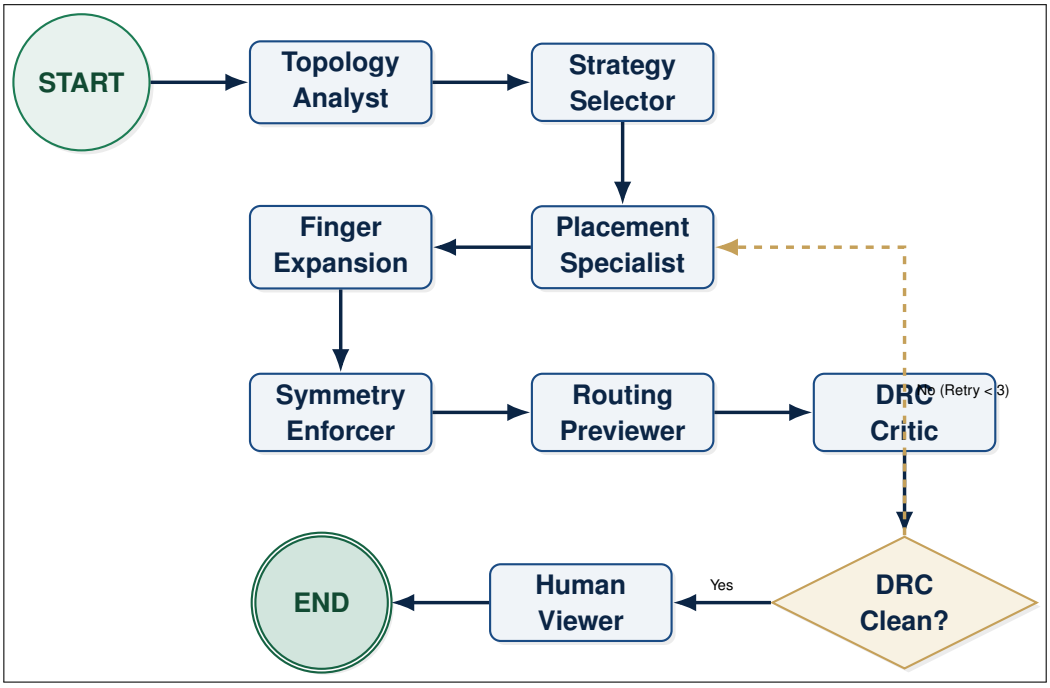


Figure 5.1: LangGraph Placement Agent State Machine and Self-Healing Feedback Loop.

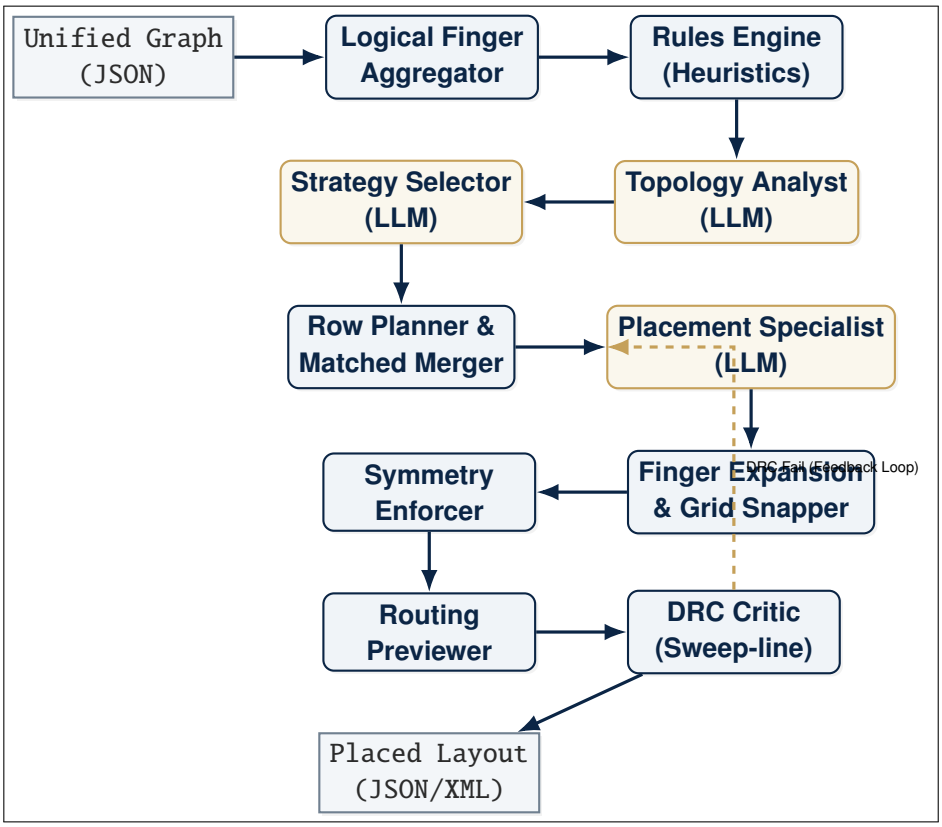


Figure 5.2: Detailed Architectural Block Diagram of the Multi-Agent Placement Stage.

The graph structure is compiled with a memory checkpointer to support state rollbacks and chat history tracking. Listing 5.1.2 shows the code that defines the compiled graph builder.

LangGraph Builder Implementation (builder.py)

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from ai_agent.graph.state import LayoutState
from ai_agent.graph.edges import route_after_drc,
    route_by_mode

from ai_agent.nodes import (
    node_topology_analyst,
    node_strategy_selector,
    node_placement_specialist,
    node_finger_expansion,
    node_symmetry_enforcer,
    node_drc_critic,
    node_routing_previewer,
    node_human_viewer,
)

def build_layout_graph(mode: str = "initial"):
    memory = MemorySaver()
    builder = StateGraph(LayoutState)

    # Register Nodes
    builder.add_node("node_topology_analyst",
        node_topology_analyst)
    builder.add_node("node_strategy_selector",
        node_strategy_selector)
    builder.add_node("node_placement_specialist",
        node_placement_specialist)
    builder.add_node("node_finger_expansion",
        node_finger_expansion)
    builder.add_node("node_symmetry_enforcer",
        node_symmetry_enforcer)
    builder.add_node("node_drc_critic", node_drc_critic)
    builder.add_node("node_routing_previewer",
        node_routing_previewer)
    builder.add_node("node_human_viewer",
        node_human_viewer)

    # Bind Transitions
    builder.add_conditional_edges(START, route_by_mode, {
        "full_pipeline": "node_topology_analyst",
        "interactive":   "node_topology_analyst",
```

```
    })
    builder.add_edge("node_topology_analyst", "
        node_strategy_selector")
    builder.add_edge("node_strategy_selector", "
        node_placement_specialist")
    builder.add_edge("node_placement_specialist", "
        node_finger_expansion")
    builder.add_edge("node_finger_expansion", "
        node_symmetry_enforcer")
    builder.add_edge("node_symmetry_enforcer", "
        node_routing_previewer")
    builder.add_edge("node_routing_previewer", "
        node_drc_critic")

    # Conditional Routing for DRC self-healing loop
    builder.add_conditional_edges("node_drc_critic",
        route_after_drc, {
        "placement_specialist": "node_placement_specialist
            ",
        "human_viewer": "node_human_viewer",
    })
    builder.add_edge("node_human_viewer", END)

    return builder.compile(checkpointer=memory), memory
```

5.2 Step-by-Step Placement Stages & Execution Nodes

This section details the operational logic, algorithms, and code implementations for each of the seven stages of the initial placement pipeline.

5.2.1 Stage 2.1: Topology Analyst

The Topology Analyst Node (`topology_analyst.py`) identifies matching structures in the design. It aggregates individual physical device fingers back into single logical groups using `aggregate_to_logical_devices`. It then analyzes the logical nodes and net connectivity using the core Python rules engine (`analyze_json`) to extract matched groups like differential pairs, current mirrors, and cross-coupled pairs. The results are formatted as a constraint prompt for the LLM.

Inputs: The input consists of the raw netlist parsed into a unified graph JSON, which contains list dictionaries for transistors and nets, alongside terminal specifications.

Outputs: Writes `constraint_text` (containing rule-based heuristics output) and `Analysis_result` (the LLM classification analysis including group boundaries, matching rules, current flow dependencies, and signal sensitivities) to the `LayoutState`.

Topology Analyst Node Implementation (topology_analyst.py)

```python
import time
from ai_agent.core.topology import analyze_json
from ai_agent.agents.topology_analyst import
    TOPOLOGY_ANALYST_PROMPT
from ai_agent.placement.finger_grouper import
    aggregate_to_logical_devices
from ai_agent.nodes._shared import _build_llm_messages,
    _invoke_with_retry, vprint

def node_topology_analyst(state):
    nodes = state.get("nodes", [])
    terminal_nets = state.get("terminal_nets", {})
    user_message = state.get("user_message", "Please
        analyze the layout topology.")
    chat_history = state.get("chat_history", [])
    selected_model = state.get("selected_model", "Gemini")

    # Group physical fingers to logical devices for
        context analysis
    logical_nodes = aggregate_to_logical_devices(nodes)

    # Run Python constraints engine
    constraint_text = analyze_json(logical_nodes,
        terminal_nets)

    analyst_user = f"User request: {user_message}\n\
        nConstraints:\n{constraint_text}\n\n"
    analyst_msgs = _build_llm_messages(
        TOPOLOGY_ANALYST_PROMPT, chat_history, analyst_user
        )

    response = _invoke_with_retry(analyst_msgs,
        selected_model, "light", "TOPO")

    return {
        "constraint_text": constraint_text,
        "Analysis_result": response.content,
        "last_agent": "topology_analyst",
        "pending_cmds": [],
    }
```

5.2.2 Stage 2.2: Strategy Selector

The Strategy Selector Node (`strategy_selector.py`) determines how components should be arranged on the layout canvas. It reads the logical constraint rules from the Topology Analyst and decides:

- **Row Configuration:** The number of horizontal rows and their type assignment (e.g., Row 0 for NMOS, Row 1 for PMOS).
- **Symmetry Plan:** The placement mode (e.g., "two_half" to enforce mirror reflection about a vertical axis).

It outputs a strategy text block that guides the downstream placement specialist.

Inputs: Reads the `Analysis_result` and `constraint_text` from the state.

Outputs: Updates `strategy_result` (the multi-layered composable strategy descriptions) and sets the global `placement_mode` ("two_half" or "auto").

Strategy Selector Node Implementation (strategy_selector.py)

```python
import time
import ai_agent.agents.strategy_selector as
    strategy_selector
from ai_agent.core.strategy import parse_placement_mode
from ai_agent.nodes._shared import _build_llm_messages,
    _invoke_with_retry

def node_strategy_selector(state):
    analysis_txt = state.get("Analysis_result", "")
    constraint_text = state.get("constraint_text", "")
    chat_history = state.get("chat_history", [])
    user_message = state.get("user_message", "Select
        layout strategy.")
    selected_model = state.get("selected_model", "Gemini")

    strategy_prompt = _build_llm_messages(
        strategy_selector.STRATEGY_SELECTOR_PROMPT, [],
        f"User request: {user_message}\n\nAnalysis:\n{
            analysis_txt}\n\nConstraints:\n{constraint_text
            }\n\n"
    )

    response = _invoke_with_retry(strategy_prompt,
        selected_model, "light", "STRATEGY")
    placement_mode = parse_placement_mode(response.content
        or "", constraint_text)

    return {
        "strategy_result": response.content,
        "placement_mode": placement_mode,
        "last_agent": "strategy_selector",
        "pending_cmds": [],
    }
```

5.2.3 Stage 2.3: Placement Specialist

The Placement Specialist Node (`placement_specialist.py`) generates symbolic coordinate positions (row, slot, orientation) for each device. To ensure the LLM places devices following analog rules (such as matching and signal flow constraints), LayoutCopilot uses a custom prompt manager called 'SkillMiddleware'.

'SkillMiddleware' parses specialized markdown files (skills) located in the `ai_agent/skills/` folder. These skills contain layout expertise for specific topologies, such as current mirrors and differential pairs. The middleware dynamically injects these skills into the agent's system prompt based on the topology analysis, guiding the agent's spatial decisions.

The agent uses a ReAct (Reasoning and Action) loop to verify its placements. It writes positioning commands in a structured XML format:

```
<cmd action="place" device="MM0" row="0" slot="1" orientation="R0" />
```

The node parses these XML commands and applies them to the device list. It also includes a post-processing pass to snap dummy devices to active row coordinates, preventing isolated "orphan" dummies.

Inputs: Reads design nodes, constraints text, connectivity edges, strategy results, and PDK constraints.

Outputs: Generates the symbolically placed `placement_nodes` dictionary, and the original XML placement command list.

Placement Specialist Node Implementation (placement_specialist.py)

```python
import copy
from ai_agent.agents.placement_specialist import
    build_placement_context,
    create_placement_specialist_agent
from ai_agent.tools.cmd_parser import extract_cmd_blocks,
    apply_cmds_to_nodes
from ai_agent.nodes._shared import _build_llm_messages,
    _invoke_with_retry

def node_placement_specialist(state):
    nodes = state.get("nodes", [])
    constraint_text = state.get("constraint_text", "")
    edges = state.get("edges", [])
    terminal_nets = state.get("terminal_nets", {})
    strategy_result = state.get("strategy_result", "auto")
    selected_model = state.get("selected_model", "Gemini")
    no_abutment_flag = state.get("no_abutment", False)

    working_nodes = copy.deepcopy(nodes)

    # Pre-calculate row assign prompts and inject matching
        middleware
```

```
    context_text = build_placement_context(
        nodes, constraint_text, terminal_nets=
            terminal_nets,
        edges=edges, no_abutment=no_abutment_flag
    )

    placer_user = f"Strategy: {strategy_result}\n\n{
        context_text}"
    # Injected prompt with SkillMiddleware internally
    placement_msgs = _build_llm_messages(
        _PLACEMENT_SYSTEM_PROMPT, state.get("chat_history"
            , []), placer_user
    )

    result = _invoke_with_retry(placement_msgs,
        selected_model, "heavy", "PLACEMENT")

    # Extract XML commands from output text
    cmds, placement_text = extract_cmd_blocks(result.
        content)

    if cmds:
        placed_nodes = apply_cmds_to_nodes(working_nodes,
            cmds)
    else:
        placed_nodes = working_nodes

    placed_nodes = _snap_orphan_dummies(placed_nodes)

    return {
        "placement_nodes": placed_nodes,
        "original_placement_cmds": cmds,
        "placement_text": placement_text,
        "last_agent": "placement_specialist",
    }
```

5.2.4 Stage 2.4: Finger Expansion

The Finger Expansion Node (`finger_expansion.py`) converts logical layout blocks into physical, placed transistors. The Placement Specialist plans coordinates at the transistor level. The Finger Expansion node expands each multi-finger logical device ($nf > 1$) into its constituent fingers.

It then runs `_resolve_row_overlaps` to place the fingers horizontally. This step snaps coordinates to the PDK grid pitch (e.g. 0.294 μm for SAED 14nm) and inserts shielding dummy devices.

Inputs: Reads symbolic `placement_nodes` and original finger counts from `nodes`.

Outputs: Emits physically expanded `placement_nodes` with grid-snapped coordinates, and dummy filler insertions.

Finger Expansion Node Implementation (finger_expansion.py)

```python
import copy
from ai_agent.placement.finger_grouper import
    expand_logical_to_fingers, _resolve_row_overlaps,
    legalize_vertical_rows
from ai_agent.tools.overlap_resolver import
    resolve_overlaps

def node_finger_expansion(state):
    placement_nodes = state.get("placement_nodes", [])
    original_nodes = state.get("nodes", [])
    terminal_nets = state.get("terminal_nets", {}) or {}
    no_abutment_flag = state.get("no_abutment", False)

    has_logical = any(n.get("_is_logical") for n in
        placement_nodes)

    if has_logical:
        # Expand logical group nodes back into individual
            finger arrays
        physical_nodes = expand_logical_to_fingers(
            placement_nodes, original_nodes, terminal_nets=
            terminal_nets)
        physical_nodes = _resolve_row_overlaps(
            physical_nodes, no_abutment_flag,
            preserve_order=True, terminal_nets=
            terminal_nets)
    else:
        physical_nodes = _resolve_row_overlaps(
            placement_nodes, no_abutment_flag,
            preserve_order=True, terminal_nets=
            terminal_nets)

    # Shifter logic to clear horizontal overlaps
    moved_ids = resolve_overlaps(physical_nodes)
    if moved_ids:
        physical_nodes = _resolve_row_overlaps(
            physical_nodes, no_abutment_flag,
            preserve_order=True, terminal_nets=
            terminal_nets)

    # Legalize rows vertically to align centroids
    physical_nodes = legalize_vertical_rows(physical_nodes
        )
```

```
27
28      return {
29          "placement_nodes": physical_nodes,
30          "deterministic_snapshot": copy.deepcopy(
                physical_nodes),
31          "last_agent": "finger_expansion",
32      }
```

5.2.5 Diffusion-Sharing Abutment Engine

In sub-micron analog layout design, packing adjacent transistors such that they share their source or drain diffusion regions (known as device abutment) is critical for area minimization and parasitic capacitance reduction. In Layout-Copilot, this is managed by a deterministic *Diffusion-Sharing Abutment Engine* (implemented in `abutment.py`), which post-processes the symbolic placement coordinates generated by the LLM.

Abutment Spacing vs. Standard Clearance

When two adjacent transistors share a common diffusion node, their poly gates can be brought much closer together. The engine enforces two distinct spacing rules between adjacent devices D_i and D_{i+1} in a row:

- **Abutted Spacing**: If the devices share a common source/drain net, they are placed at the compacted abutment pitch δ_{abut}:

$$x_{i+1} = x_i + \delta_{\text{abut}}, \quad \text{where } \delta_{\text{abut}} = 0.070\,\mu\text{m} \tag{5.1}$$

- **Standard Spacing**: If they do not share a net, they are separated by the standard PDK transistor clearance pitch W_i:

$$x_{i+1} = x_i + W_i, \quad \text{where } W_i \geq 0.294\,\mu\text{m} \tag{5.2}$$

Chain Formulation via Union-Find

Individual pair-wise abutment candidates (identified by the rules engine based on terminal net matching) are consolidated into multi-device abutment chains using a Union-Find algorithm with path compression in `build_abutment_chains`. This groups connected components (e.g., matching $A \leftrightarrow B$ and $B \leftrightarrow C$ into chain $[A, B, C]$) to ensure they are kept physically contiguous and packed as a single macroscopic segment.

Combinatorial Orientation Optimization

Since MOS transistors are physically symmetric, their source and drain terminals can be interchanged by horizontally flipping their orientation ($R0 \leftrightarrow MY$). The engine runs a global combinatorial optimization pass (`optimize_all_rows_orientations`) to assign orientations:

1. Group contiguous transistor fingers in a row belonging to the same parent device.
2. Identify and pair symmetric blocks positioned mirror-symmetrically across the row's center axis.
3. Perform a binary combinations exploration (flipping pairs together to preserve symmetry) to find the orientation variables that maximize shared-net boundaries (diffusion sharing opportunities) and eliminate net-short conflicts.

Geometry Healing & Failsafe Spacing Cascade

Once optimal orientations are decided, the layout coordinates are reconstructed using two deterministic passes:

- **Healed Coordinates**: The function `heal_abutment_positions` packs segments (chains and singletons) left-to-right. It checks adjacent nets along the chain and dynamically swaps terminal node labels (Source/Drain) and flips device geometries (R0/MY) to align matching nets.
- **Failsafe Spacing Cascade**: The function `force_abutment_spacing` acts as a final safety check. It scans rows, detects adjacent devices natively declaring abutment, and forces their spacing to exactly $\delta_{\text{abut}} = 0.070\,\mu\text{m}$, cascading the required shift to all downstream devices in the row to prevent horizontal overlap.
- **Dummy Biasing**: Finally, dummy devices (which do not conduct active currents) are dynamically biased to power rails (VDD for PMOS, VSS/GND for NMOS) or to neighboring active terminal nets to prevent floating diffusions.

5.2.6 Stage 2.5: Symmetry Enforcer

The Symmetry Enforcer Node (`symmetry_enforcer.py`) ensures symmetrical layout structures match the schematic constraints. Rather than placing fingers individually, it translates interdigitated finger blocks as rigid bodies.

This matches the midpoint of each block to the shared vertical center line, enforcing mirror symmetry while preserving interdigitated structures.

Mathematically, let X_G represent the set of all coordinates of symmetry-constrained components. The global vertical center line x_{axis} is computed by finding the mid-point of the global span and snapping it to the nearest half-pitch grid ($P_{\text{half}} = 0.147\,\mu\text{m}$):

$$x_{\text{raw\_axis}} = \frac{\min(X_G) + \max(X_G)}{2} \tag{5.3}$$

$$x_{\text{axis}} = \text{round}\left(\frac{x_{\text{raw\_axis}}}{P_{\text{half}}}\right) \times P_{\text{half}} \tag{5.4}$$

For each matched pair block (treated as a rigid body with active fingers F_B), the enforcer computes its current centroid x_{mid} and translates all constituent nodes by the rigid offset dx:

$$x_{\text{mid}} = \frac{\min_{f \in F_B}(x_f) + \max_{f \in F_B}(x_f + w_f)}{2} \tag{5.5}$$

$$dx = x_{\text{axis}} - x_{\text{mid}} \tag{5.6}$$

$$x_f \leftarrow x_f + dx, \quad \forall f \in F_B \tag{5.7}$$

After shifting, the enforcer executes a deterministic overlap resolution pass and snaps coordinates back onto the grid.

Inputs: Reads physical coordinates from `placement_nodes` and constraint details.

Outputs: Returns symmetry-corrected, snapped, and legal coordinates for all device fingers.

Symmetry Enforcer Node Implementation (symmetry_enforcer.py)

```
from ai_agent.nodes.symmetry_enforcer import
    parse_symmetry_block, _find_finger_ids
from ai_agent.placement.finger_grouper import
    _resolve_row_overlaps, legalize_vertical_rows
from config.design_rules import PITCH_UM

def node_symmetry_enforcer(state):
    nodes = state.get("placement_nodes", [])
    constraint_text = state.get("constraint_text", "")
    placement_mode = state.get("placement_mode", "auto")

    if placement_mode != "two_half":
        return {"placement_nodes": nodes, "last_agent": "
            symmetry_enforcer"}

    sym_rules = parse_symmetry_block(constraint_text)
    if not sym_rules:
        return {"placement_nodes": nodes, "last_agent": "
            symmetry_enforcer"}

    # Calculate global layout width and define central
        axis
    xs = [float(n["geometry"]["x"]) for n in nodes]
    widths = [float(n["geometry"]["width"]) for n in nodes
        ]
    max_x = max(x + w for x, w in zip(xs, widths))
    x_axis = round((max_x / 2.0) / (PITCH_UM / 2.0)) * (
        PITCH_UM / 2.0)

    # Translate matched pairs to align their centers
```

```
        symmetrically about x_axis
    fixed_nodes = copy.deepcopy(nodes)
    for left_id, right_id, _ in sym_rules["pairs"]:
        left_fingers = _find_finger_ids(left_id,
            fixed_nodes)
        right_fingers = _find_finger_ids(right_id,
            fixed_nodes)
        block_fids = left_fingers + right_fingers

        # Calculate centroids and shift coordinates
            symmetrically
        bxs = [float(fixed_nodes[fid]["geometry"]["x"])
            for fid in block_fids]
        block_mid = (min(bxs) + max(bxs)) / 2.0
        dx = x_axis - block_mid
        for fid in block_fids:
            fixed_nodes[fid]["geometry"]["x"] = round(
                float(fixed_nodes[fid]["geometry"]["x"]) +
                dx, 6)

    fixed_nodes = _resolve_row_overlaps(fixed_nodes, False
        , True)
    fixed_nodes = legalize_vertical_rows(fixed_nodes)

    return {
        "placement_nodes": fixed_nodes,
        "last_agent": "symmetry_enforcer",
    }
```

5.2.7 Stage 2.6: Routing Previewer

The Routing Previewer Node (`routing_previewer.py`) estimates layout routing costs before generation. It calculates a routing metric by evaluating:

- **Net Crossings:** The number of times signal paths cross, indicating routing congestion.
- **Half-Perimeter Wire Length (HPWL):** The estimated wire length for each net n:

$$\text{HPWL}(n) = (x_{\max} - x_{\min}) + (y_{\max} - y_{\min}) \tag{5.8}$$

This metric evaluates placement configurations and guides structural optimization.

Inputs: Reads placement coordinates, device pins, and net connections.

Outputs: Generates routing quality metrics and writes routing track width suggestions back to the state to expand the vertical spacing between row coordinates where congested channels are predicted.

Routing Previewer Node Implementation (routing_previewer.py)

```python
import time
from ai_agent.core.routing import build_routing_report
from ai_agent.nodes._shared import ip_step

def node_routing_previewer(state):
    nodes         = state.get("placement_nodes", [])
    edges         = state.get("edges", [])
    terminal_nets = state.get("terminal_nets", {})

    report = build_routing_report(nodes, edges or [],
       terminal_nets or {})
    routing_legacy = report.to_legacy_dict()
    routing_legacy["log_text"] = report.format_log()

    return {
        "routing_result": routing_legacy,
        "last_agent": "routing_previewer"
    }
```

5.2.8 Stage 2.7: DRC Critic

The DRC Critic Node (`drc_critic.py`) validates placement clearances. It runs a sweep-line overlap detection check. If violations exist, it uses `compute_prescriptive_fixes` to calculate coordinate offsets and shift devices.

If this automated fix fails, the node routes execution back to the Placement Specialist, providing a feedback string detailing the violations (e.g. `"Overlap between MM0 and MM1 at X=0.294"`) to guide the next iteration.

Inputs: Reads placed nodes and target separation variables.

Outputs: DRC flag list, pass status, and coordinate offsets.

DRC Critic Node Implementation (drc_critic.py)

```python
import time
from ai_agent.core.drc import run_drc_check,
   compute_prescriptive_fixes
from ai_agent.tools.cmd_parser import apply_cmds_to_nodes

def node_drc_critic(state):
    retry_num = state.get("drc_retry_count", 0)
    nodes = state.get("placement_nodes", [])
    gap_px = state.get("gap_px", 0.0)
    terminal_nets = state.get("terminal_nets", {})
    gap_um = gap_px / 34.0 if gap_px > 0 else 0.0

    drc_result = run_drc_check(nodes, gap_um)
```

```
    if drc_result["pass"]:
        return {
            "placement_nodes": nodes,
            "drc_pass": True,
            "drc_flags": [],
            "last_agent": "drc_critic",
        }

    fixes = compute_prescriptive_fixes(
        drc_result=drc_result, gap_px=gap_px, nodes=nodes,
        geometric_tags=state.get("groups", {}),
           terminal_nets=terminal_nets
    )

    if fixes:
        fixed_nodes = apply_cmds_to_nodes(nodes, fixes)
        fixed_drc = run_drc_check(fixed_nodes, gap_um)
        if fixed_drc["pass"]:
            return {
                "placement_nodes": fixed_nodes,
                "drc_pass": True,
                "drc_flags": [],
                "pending_cmds": state.get("pending_cmds",
                   []) + fixes,
                "drc_retry_count": retry_num + 1,
                "last_agent": "drc_critic",
            }

    return {
        "placement_nodes": nodes,
        "drc_pass": False,
        "drc_flags": list(drc_result.get("violations", [])
           ),
        "drc_retry_count": retry_num + 1,
        "last_agent": "drc_critic",
    }
```

5.3 Input & Output Interfaces

Stage 2 operates with structured JSON input and output interfaces.

5.3.1 Input Interface Schema

The input is the unified graph JSON, which contains schematic definitions, node attributes, and logical net connectivity.

5.3.2 Output Placement Schema

The output JSON details the placement geometry. Each physical finger has absolute coordinates (x, y), width and height footprints, and orientations.

Output Placement JSON Schema

```
{
  "placement": [
    {
      "id": "MM2_f1",
      "x": 0.0,
      "y": 0.0,
      "width": 0.294,
      "height": 0.668,
      "orientation": "R0",
      "type": "nmos",
      "parent": "MM2"
    },
    {
      "id": "MM2_f2",
      "x": 0.294,
      "y": 0.0,
      "width": 0.294,
      "height": 0.668,
      "orientation": "R0",
      "type": "nmos",
      "parent": "MM2"
    }
  ]
}
```

5.4 Execution Log Walkthrough: Dynamic Latch Comparator

To verify the robust, deterministic execution of LayoutCopilot's initial placement pipeline, this section conducts a detailed walkthrough of the execution trace generated during the placement of a *Dynamic Latch Comparator* (recorded in `placement_live_output.log`). The circuit contains 62 physical transistor fingers (40 PMOS and 22 NMOS).

5.4.1 Phase 1: Topology Analyst Execution

The pipeline begins by grouping the 62 physical fingers to form 11 logical device representations (`MM0` to `MM10`). The rules engine generates the connectivity constraints and queries the LLM:

- **Input Stage Pair**: Detects the input NMOS transistors `MM8` and `MM9` as a differential pair, mapping them to `GROUP_INPUT_DIFFERENTIAL_PAIR`. It specifies symmetry and match requirements for the pair:

Input Stage Pair Mapping Constraint

```
PAIR_MAPPING:
    - (MM8, MM9)
```

- **Tail Source**: Groups the tail current source `MM10` and sets it as the axis of symmetry for the input differential pair.
- **Precharge PMOS Devices**: Groups `MM0`, `MM1`, `MM2`, and `MM3` into the precharge PMOS group, denoted `GROUP_PRECHARGE_PMOS`, with symmetric constraint pairs: `(MM3, MM2)` and `(MM0, MM1)`.
- **Latch Elements**: Groups the cross-coupled regenerative latches: PMOS devices `(MM4, MM5)` and NMOS devices `(MM6, MM7)`.

5.4.2 Phase 2: Floorplanning Strategy Selector

The Strategy Selector generates four composable constraints:

1. **Input Stage Core**: Common centroid placement for `MM10` (tail) centered between the input pair `(MM8, MM9)` along a vertical axis.
2. **Latch Symmetry**: Align cross-coupled pairs `(MM4, MM5)` and `(MM6, MM7)` symmetrically.
3. **Precharge PMOS Symmetry**: Centering the precharge group `GROUP_PRECHARGE_PMOS` about the vertical axis.
4. **Vertical Current Stacking**: Vertically stack the tail current source, input pair, and NMOS latches in order of current flow, placing the PMOS precharge and latch components above them.

The global placement mode is set to `two_half` to establish a vertical symmetry axis.

5.4.3 Phase 3: Placement Specialist Constraint Resolution

In Step 3a, LayoutCopilot merges matched pairs into rigid interdigitated blocks:

- `MM8` and `MM9` are merged to `MM8_MM9_matched` (ABAB pattern, 18 fingers, width: 5.292 μm).
- `MM2` and `MM1` are merged to `MM2_MM1_matched` (ABAB pattern, 18 fingers, width: 5.292 μm).

- `MM5` and `MM4` are merged to `MM5_MM4_matched` (symmetric cross-coupled, 8 fingers, width: 2.352 μm).
- `MM3` and `MM0` are merged to `MM3_MM0_matched` (ABAB pattern, 18 fingers, width: 5.292 μm).
- `MM7` and `MM6` are skipped since they are single-finger devices.

The row planner balances the row aspect ratios. Because the total PMOS footprint width (15.288 μm) exceeds that of the NMOS devices (8.526 μm), the row planner splits the PMOS devices into three rows, resulting in a five-row stack:

- **NMOS Row 0** ($y = 0.000\,\mu$m): `MM8_MM9_matched` (width: 5.292 μm).
- **NMOS Row 1** ($y = 0.818\,\mu$m): `MM7, MM6, MM10` (widths: 0.294 + 0.294 + 1.176 μm).
- **PMOS Row 0** ($y = 1.636\,\mu$m): `MM3_MM0_matched` (width: 5.292 μm).
- **PMOS Row 1** ($y = 2.454\,\mu$m): `MM5_MM4_matched` (width: 2.352 μm).
- **PMOS Row 2** ($y = 3.272\,\mu$m): `MM2_MM1_matched` (width: 5.292 μm).

Step 3b invokes the LLM using these row boundaries. The LLM assigns slot numbers (0 to 17) for each row. For example, in Row 1 ($y = 0.818\,\mu$m), it assigns `MM6` to slot 6, `MM10` to slots 7–10, and `MM7` to slot 11, centering the active components and leaving slots 0–5 and 12–17 for dummy devices.

5.4.4 Phase 4: Finger Expansion & Dummy Insertion

In Step 3d, the symbolic slots are expanded to physical fingers. The logical nodes unroll to 68 active devices. The overlap resolver calculates spacing constraints and inserts:

- **Edge Dummies**: Placed on the outer boundaries of interdigitated rows (e.g. `EDGE_DUMMY_L` at slot 0, and `EDGE_DUMMY_R` at slot 17) to prevent matching mismatches.
- **Filler/Bridge Dummies**: Placed in rows with fewer active components to maintain uniform row widths and routing density (e.g., Row 1 at $y = 0.818\,\mu$m receives filler dummies in slots 0–5 and 12–17).

This brings the total physical device count to 90 transistors.

5.4.5 Phase 5: Symmetry Enforcer Alignment

In Stage 3.5, the Symmetry Enforcer computes the global layout midpoint at $x_{\text{axis}} = 2.4990\,\mu$m (using the 18-slot width grid snapped to half-pitch). It translates the interdigitated blocks:

- The block `MM8_MM9_matched` is centered at 2.4990 μm (maintaining its ABAB pattern).
- The block `MM3_MM2_matched` is centered at 2.4990 μm.
- The tail current source `MM10` (4 fingers) is centered directly on the axis.

To resolve coordinate overlaps introduced by the translation, a post-enforcement overlap pass shifts 32 devices on the grid. For instance:

Symmetry Enforcer Overlap Correction Log Snippet

```
[OVERLAP-FIX] MM1_m1 moved x=0.5880 -> 0.8820 (was
    overlapping MM2_m1 at row y=3.2720)
[OVERLAP-FIX] MM2_m2 moved x=1.0290 -> 1.1760 (was
    overlapping MM1_m1 at row y=3.2720)
```

5.4.6 Phase 6: Routing Previewer & DRC Critic

The Routing Previewer measures a Manhattan HPWL of 21.844 μm and estimates 41 crossings. The critical nets (`VOUTN`, `VOUTP`, `CLK`, `VX`, `VY`) are identified as cross-row nets, requiring track space in the horizontal routing bands:

- **Band 1** ($y \approx 1.002\,\mu$m): Requires 6 routing tracks (widened to 0.938 μm).
- **Band 2** ($y \approx 2.004\,\mu$m): Requires 5 routing tracks (widened to 0.804 μm).

Finally, the DRC Critic performs a sweep-line clearance check on all 90 devices. Because the Symmetry Enforcer and Overlap Resolver successfully aligned the layout onto the PDK grid, the DRC passes on attempt 1 with zero violations. The layout achieves a utilization of 68.9% and a composite quality score of 100% (A+ grade).

5.5 Example Walkthrough: Current Mirror Initial Placement

This section traces the initial placement of the current mirror circuit (`CM`) from the `examples/current_mirror` directory.

5.5.1 Topology Grouping

The Topology Analyst reads the unified graph and groups matched NMOS and PMOS mirrors:

- **NMOS Mirror:** MM0, MM1, MM2 (common gate net `C`, source to `gnd`).
- **PMOS Mirror:** MM3, MM4, MM5 (common gate net `X`, source to `VDD`).

5.5.2 Row Planning

The Strategy Selector separates the device types into Y-rows:

- **Row 0:** Assigned to NMOS devices, set at Y coordinate 0.0 μm.
- **Row 1:** Assigned to PMOS devices, set at Y coordinate 0.668 μm.

5.5.3 Symbolic Positioning

The Placement Specialist assigns slot indices:

- **Row 0 NMOS Slot Sequence:** MM0 (diode reference) in the center, with MM1 and MM2 placed symmetrically on either side.
- **Row 1 PMOS Slot Sequence:** MM3 (diode reference) in the center, with MM4 and MM5 placed symmetrically.

5.5.4 Physical Finger Expansion

The Finger Expansion node expands the logical nodes into physical fingers:

- **MM0 ($nf = 16$):** Expanded into 16 fingers.
- **MM1 ($nf = 8$):** Expanded into 8 fingers.
- **MM2 ($nf = 4$):** Expanded into 4 fingers.

Matched pairs are placed in interdigitated configurations (e.g. ABBA patterns). For example, the MM1 ($nf = 8$) and MM2 ($nf = 4$) mirrors are placed in an interleaved sequence to minimize mismatch.

The overlap resolver shifts coordinates horizontally to clear spacing violations. Symmetrical structures are aligned about the vertical center axis ($x = 4.116\,\mu$m). This placement passes the DRC Critic validation, generating a DRC-clean layout config.

CHAPTER 6

Stage 3: Chatbot Co-Pilot & GUI Integration

Interactive layout editing forms the foundation of human-in-the-loop CAD tools. While automated scripts are useful for batch layout generation, analog designers require the ability to interactively steer the layout engine, modify relative positions, resolve design rule violations, and query circuit information using natural language. LayoutCopilot fulfills this need by integrating a Chatbot Co-Pilot panel directly into the PySide6 layout editor.

This chapter details the architecture of the interactive chatbot stage. It explains the LangGraph-driven intent routing graph, the asynchronous frontend-backend worker thread architecture, the context-aware prompt templates, and the integration of PySide6's undo stack with headless KLayout live-rendering previews.

6.1 Interactive Chat Graph Architecture

The interactive chatbot is orchestrated by a dedicated state machine constructed via `build_chat_graph()` in the graph builder. Unlike the linear auto-run pipeline of Stage 2, the chat workflow uses intent-based routing to dispatch the user's natural language request to the appropriate agent node. Figure 6.1 visualizes the state transition model.

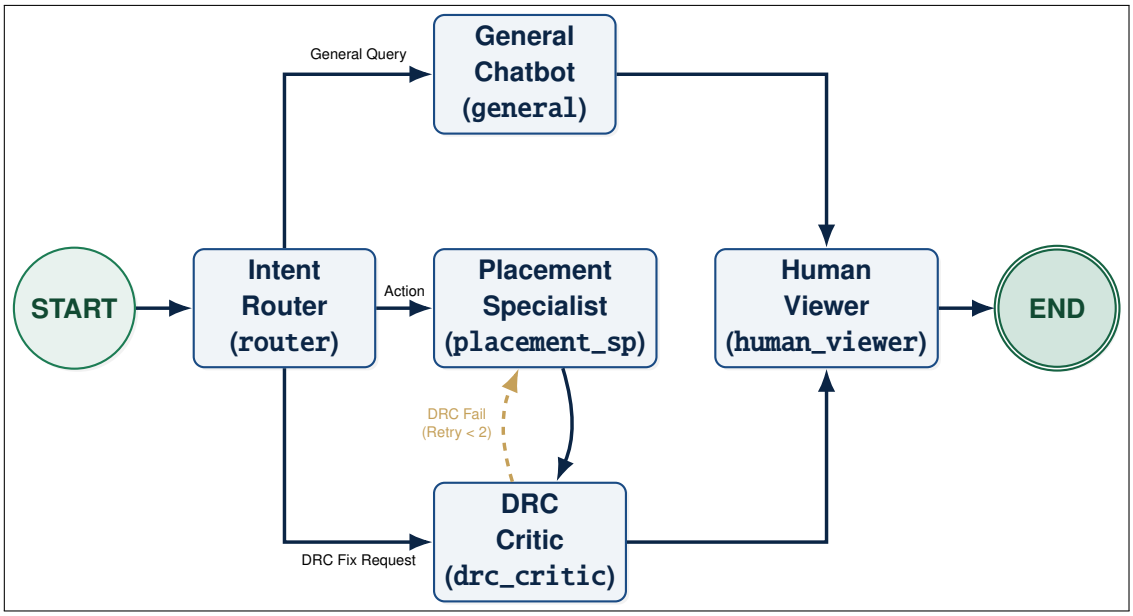


Figure 6.1: Interactive Chatbot Agent LangGraph State Transition Model.

6.1.1 State Schema and Shared Context

The interactive agent graph relies on a shared state variable, `LayoutState`, which acts as the unified database passed between nodes. The state elements, their types, and the nodes that mutate them are detailed in Table 6.1.

Table 6.1: Unified Graph LayoutState Schema Elements.

Key	Data Type	Initial Value	Mutating Node(s)
`user_message`	`str`	User text input	None (Read-only)
`chat_history`	`list[dict]`	Scrubbed history	`router`, `general`, `placement_sp`
`selected_model`	`str`	`"Gemini"`	None (Read-only)
`intent`	`str`	`""`	`router`
`router_target`	`str`	`""`	`router`
`placement_nodes`	`list[dict]`	Canvas nodes	`placement_sp`, `drc_critic`
`pending_cmds`	`list[dict]`	`[]`	`placement_sp`, `drc_critic`
`drc_pass`	`bool`	`True`	`drc_critic`
`drc_flags`	`list[str]`	`[]`	`drc_critic`
`general_response`	`str`	`""`	`general`
`drc_retry_count`	`int`	`0`	`drc_critic`

The chat state graph registered in `builder.py` binds the routing transitions and registers five main execution nodes:

1. **Intent Router (`router`)**: Employs a regex fast-path for simple commands

and falls back to a lightweight LLM call to classify user messages into three categories: `placement_specialist`, `drc_critic`, or `general`.

2. **Placement Specialist (`placement_specialist`)**: Executes layout adjustments and movement commands. To minimize latency, it uses a lightweight logical device aggregator (`aggregate_to_logical_devices()`) instead of the full row assigner and block merger.

3. **DRC Critic (`drc_critic`)**: Evaluates design rule spacing and horizontal overlaps, generating corrective offsets or looping back to the placement specialist on verification failures.

4. **General Assistant (`general`)**: Handles knowledge retrieval, circuit explanations, and descriptive user questions.

5. **Human Viewer (`human_viewer`)**: Visualizes the placement adjustments in the GUI and terminates the execution stream.

Listing 6.1.1 shows the LangGraph construction logic for interactive chat mode.

Interactive Chat LangGraph Builder Implementation (builder.py)

```
def build_chat_graph():
    """Build the chat-bot LangGraph with intent-based
       routing."""
    memory = MemorySaver()
    builder = StateGraph(LayoutState)

    # Register Chat Nodes
    builder.add_node("router", _node_router)
    builder.add_node("placement_specialist",
       node_placement_specialist_chatbot)
    builder.add_node("drc_critic", node_drc_critic)
    builder.add_node("general", node_general_chatbot)
    builder.add_node("human_viewer", node_human_viewer)

    # Bind State Transitions
    builder.add_edge(START, "router")
    builder.add_conditional_edges(
        "router",
        _route_after_router,
        {
            "placement_specialist": "placement_specialist"
               ,
            "drc_critic": "drc_critic",
            "general": "general",
        },
    )
    builder.add_edge("placement_specialist", "drc_critic")
```

```
25      builder.add_conditional_edges(
26          "drc_critic",
27          route_after_drc_chat,
28          {
29              "human_viewer": "human_viewer",
30              "placement_specialist": "placement_specialist"
                    ,
31          },
32      )
33      builder.add_edge("general", "human_viewer")
34      builder.add_edge("human_viewer", END)
35
36      return builder.compile(checkpointer=memory), memory
```

6.1.2 Line-by-Line Analysis of build_chat_graph()

The entry point of the LangGraph compiler initializes a `StateGraph` using `LayoutState` (Line 4). It registers five nodes using the graph-binding method `add_node()` (Lines 7-11). The edge connections specify the workflow layout:

- **Line 14**: Binds the start node to the routing parser.
- **Line 15-23**: Declares a conditional transition from `"router"` executing `_route_after_router()`. The routing function evaluates the state key `router_target` and maps it to the matching execution node.
- **Line 24**: Directs the placement specialist straight into DRC critic validation.
- **Line 25-32**: Establishes the conditional self-healing edge after the DRC critic. The router `route_after_drc_chat()` checks whether `drc_pass` is true. If false, and the retry limit has not been exceeded, it loops back to the placement node.
- **Line 33**: Connects the conversational chatbot to the viewer node.
- **Line 34**: Connects the human review node to the graph terminal endpoint.

6.1.3 Intent Classification Node

The router node (`_node_router()`) reads the user's message and invokes `classify_intent()` in `classifier.py`. The classification process uses a two-tiered routing paradigm. First, a regex-based quick classifier checks the message against trivial geometric command signatures (e.g. *"move MM1"*, *"swap A and B"*, *"abut x y"*). If a match is found, the system immediately routes execution to the placement specialist, skipping LLM evaluation to eliminate API latency.

When semantic ambiguity exists, the router falls back to a lightweight LLM call utilizing a structured classification prompt forcing the model to output a

single keyword indicating the target node. Listing 6.1.3 illustrates the classifier system prompt and LLM call logic.

Intent Classifier Logic (classifier.py)

```
CLASSIFIER_PROMPT = """\
You are an intent classifier for an analog IC layout editor.
Classify the user's message into exactly ONE of these intent targets:

    placement_specialist - Direct placement / movement / ordering /
                           abutment / interdigitation / row assignment / optimize routing crossings.
                           Usually a command for the placement engine.

    drc_critic           - DRC violations, spacing, overlap, clean-up,
                           or fix-and-verify layout requests.

    general              - Factual lookups, definitions, or knowledge
                           questions about devices, properties, or concepts
                           (e.g. "What is...", "How does ...", "Why is...",
                           "What are the specs of..."). Use this when the
                           user is asking FOR INFORMATION rather than
                           requesting an action on the layout.

Choose the single best target for the user's request.
Reply with ONLY the target name.
Do not explain. Do not add punctuation.
"""

def classify_intent(user_message: str, selected_model: str) -> str:
    stripped = user_message.strip()
    msgs = [
        {"role": "system", "content": CLASSIFIER_PROMPT},
        {"role": "user",   "content": user_message},
```

```
29      ]
30      try:
31          llm = get_langchain_llm(selected_model,
                task_weight="light")
32          vprint(f"[CLASSIFIER] Requesting Intent
                Classification from {selected_model}...")
33          result = llm.invoke(msgs)
34          if not result:
35              return "general"
36          label = result.content.strip().lower().split()[0].
                rstrip(".,;:")
37          node_labels = {
38              "placement_specialist": "placement_specialist"
                    ,
39              "drc_critic": "drc_critic",
40              "general": "general",
41          }
42          if label in node_labels:
43              vprint(f"[CLASSIFIER] LLM -> {label}: '{
                    stripped[:60]}'")
44              return node_labels[label]
45      except Exception as exc:
46          vprint(f"[CLASSIFIER] Failed: {exc} - defaulting
                to general")
47      return "general"
```

6.1.4 Line-by-Line Analysis of classify_intent()

The intent classifier implements decision routing using the following code steps:

- **Line 25-38**: Formulates the system instructions. The instructions define the semantic difference between layout actions (`placement_specialist`), verification edits (`drc_critic`), and conceptual inquiries (`general`).
- **Line 41-45**: Assembles the prompt payload. The current message is paired with the system rules to ensure bounded completion.
- **Line 64**: Instantiates the lightweight backend wrapper with `task_weight="light"`. This configures the engine to use a faster, smaller model model configuration to prevent long token latencies.
- **Line 66**: Dispatches the sync JSON payload to the language API.
- **Line 69-79**: Truncates and sanitizes the output. It extracts the first word of the response string, strips trailing symbols (commas, colons), and checks if the keyword matches the three allowed node labels.
- **Line 80-82**: Handles connection timeouts or API errors by falling back to the `"general"` chatbot.

6.2 Chatbot Capabilities, Interactive Features & User Commands

Interactive layout environments must support a wide range of designer commands, spanning geometric adjustments, structural optimizations, design rule checks, and design parameters queries. LayoutCopilot implements these capabilities using a set of conversational commands that are classified and routed to the corresponding graph nodes. This section details the main features of the chatbot co-pilot and how they map to backend actions.

6.2.1 Conversational Device Manipulation

Designer prompts that instruct the chatbot to modify device positions are routed to the `placement_specialist` node. The system prompt instructs the language model to parse natural language directions and synthesize structured JSON-RPC command payloads. These commands fall into three primary classes:

1. **Relative Movement:** Instructions like *"Move MM1 to the right by 2 slots"* or *"Shift MDUM0 up by 1 row"*. The agent maps the directions (left, right, up, down) and the integer slot/row counts to actual coordinate shifts (Δx, Δy) snapped to PDK spacing and height grids.

2. **Absolute Placement:** Instructions specifying exact coordinate targets (e.g., *"Set the x coordinate of MM2 to 1.176 microns"*). The agent extracts the absolute target coordinate and snaps it to the 0.294 μm grid pitch.

3. **Device Swapping:** Instructions that swap the physical locations of two transistors (e.g., *"Swap the locations of MM0 and MM1"*). The coordinator extracts the coordinates of both target nodes, swaps them, snaps them to grid row centers, and keeps all other routing/net configurations intact.

6.2.2 Connectivity-Aware Diffusion Sharing (Abutment)

Analog layouts are highly sensitive to source/drain diffusion parasitic capacitance. By abutting adjacent transistors sharing a common node (e.g., matching source or drain terminals), designers can share diffusion active regions, reducing layout area and parasitic caps. The chatbot supports interactive abutment through prompts like *"Abut MM0 and MM1"*. Under the hood, the `placement_specialist` checks the active net connections of the adjacent terminals of the target devices. If a shared net is verified, it executes a diffusion-sharing optimization, adjusts the devices' relative positions to eliminate spacing gaps, and flips transistor gates ($R0 \leftrightarrow MY$) if required.

6.2.3 Symmetry Group Definition

Differential pairs and matched current mirrors require strict layout symmetry. Designers can dynamically define symmetry constraints using conversational

language, such as *"MM0 and MM1 must be matching symmetric pairs about the vertical center axis"*. The chatbot parses the symmetry instruction, extracts the target devices and matching coordinates, and registers them into the active `LayoutState` symmetry groups. In subsequent editing commands, the symmetry enforcer mirrors any manual or conversational coordinate modifications in real-time, preserving match parameters.

6.2.4 DRC Criticism and Self-Healing

When designers request validation (e.g., *"Check for DRC errors and fix them"*), the intent classifier routes the query to the `drc_critic` node. The DRC critic runs physical checks (such as gate-to-gate spacing and diffusion overlaps) and lists active design rule violations. If violations are present, the critic node synthesizes prescriptive coordinate corrections to shift the offending devices until they clear all design rules. The state is then updated with the corrective actions, which are executed automatically on the canvas.

6.2.5 General Circuit and PDK Queries

Designers can query the structural layout and PDK properties using prompts like *"What is the width and number of fingers of MM0?"* or *"Are MM1 and MM2 sharing diffusion?"*. These queries are routed to the `general` chatbot node. This node builds a textual context snapshot of the active layout state, reads the device properties and PDK guidelines, and formulates a natural language explanation, providing immediate layout documentation.

Table 6.2 summarizes these interactive chatbot capabilities, including the intent routing classifications, example user queries, and backend synthesized actions.

6.3 Specialized Chat Nodes & Prompt Architecture

6.3.1 Conversational General Assistant Node

If the query is classified as a general query, the graph routes execution to the general assistant node (`general_chatbot.py`). This node generates a context-aware natural language explanation based on the layout's active nodes, symmetry constraints, and terminal connections.

The layout state is serialized into a textual snapshot using `build_placement_context_chatbot()` which lists the physical attributes, coordinate bounds, matching structures, and terminal nets of all placed transistors. Listing 6.3.1 shows the implementation details.

General Chatbot Node Implementation (general_chatbot.py)

```
1 def node_general_chatbot(state):
2     t0 = time.time()
```

Table 6.2: Interactive Chatbot Capabilities, Routed Nodes, and Target Commands.

Feature Area	Example Prompt	Routed Node	Synthesized Action
Relative Move	*"Move MM1 to the right by 2 slots"*	`placement_sp`	`{"action": "move", "device": "MM1", "x": 0.588, "y": 0}`
Absolute Move	*"Set the x coordinate of MM2 to 1.176"*	`placement_sp`	`{"action": "move", "device": "MM2", "x": 1.176}`
Device Swap	*"Swap MM0 and MM1"*	`placement_sp`	`{"action": "swap", "device_a": "MM0", "device_b": "MM1"}`
Diffusion Abut	*"Abut MM0 and MM1"*	`placement_sp`	`{"action": "abut", "device_a": "MM0", "device_b": "MM1"}`
Symmetry Set	*"MM0 and MM1 must be matching symmetric pairs"*	`general`	Updates state symmetry rules and snaps layout
DRC Criticism	*"Run design rule checks and fix spacing errors"*	`drc_critic`	Runs sweepline checks and schedules self-healing shifts
PDK Lookup	*"What is the width and multiplier of MM0?"*	`general`	Reads device geometry keys from layout state JSON

```
    nodes = state.get("placement_nodes", []) or state.get("nodes", [])
    edges = state.get("edges", [])
    terminal_nets = state.get("terminal_nets", {})
    constraint_text = state.get("constraint_text", "")
    user_message = state.get("user_message", "")
    chat_history = state.get("chat_history", [])
    selected_model = state.get("selected_model", "Gemini")
    no_abutment_flag = state.get("no_abutment", False)

    # Build layout context snapshot for chat prompt injection
    context_text = build_placement_context_chatbot(
        nodes, constraint_text, terminal_nets=terminal_nets,
        edges=edges, no_abutment=no_abutment_flag,
    )

    user_prompt = f"User request: {user_message}\n\n{context_text}"
    messages = _build_llm_messages(GENERAL_CHATBOT_PROMPT, chat_history, user_prompt)

    response_text = ""
    try:
        response = _invoke_with_retry(messages, selected_model, "light", "GENERAL")
        response_text, response_thinking = _split_content_and_thinking(response.content)
        response_text = _strip_thinking_text(response_text)
    except Exception as exc:
        response_text = "General assistant failed to respond."

    updated_chat_history = _update_and_save_chat_history(
        chat_history=chat_history, user_content=user_message,
        node_role="General Assistant", node_content=response_text,
    )

    return {
        "chat_history": updated_chat_history,
        "general_response": response_text,
        "last_agent": "general",
        "pending_cmds": [],
    }
```

6.3.2 Line-by-Line Analysis of node_general_chatbot()

The general chatbot node operates by serializing structural layouts:

- **Line 36-43**: Retrieves layout parameters. The coordinate node array, subcircuit edges, and netlists are loaded from the state.
- **Line 45-48**: Generates a text context snapshot. The helper method parses the current node coordinates, active channel widths, finger multipliers, and symmetry constraints, converting them into structured text.
- **Line 53**: Couples the text snapshot with the user's natural language question.
- **Line 54**: Interleaves previous conversation rounds from `chat_history` to maintain context across multiple turns.
- **Line 60**: Invokes the lightweight language model. It splits the internal reasoning chain ('thinking') from the user-facing text before saving the results.
- **Line 67-71**: Updates the central SQLite chat log to maintain continuity across subsequent queries.

6.3.3 Placement Specialist Node in Chat Mode

The placement specialist node in chat mode (`node_placement_specialist_chatbot()`) allows designers to modify the layout using conversational directives.

Unlike the autonomous initial placement, the chat-mode placement specialist operates on the active user canvas. The process follows a structured sequence:

1. **Context Assembly**: The active canvas geometry is parsed. The system aggregates physical fingers back to logical devices using `aggregate_to_logical_devices()` to keep the LLM context size minimal and readable. For example, a transistor split into 16 fingers is represented as a single block with $nf = 16$ rather than 16 individual instances, which prevents context window explosion and simplifies spatial reasoning.
2. **ReAct LLM Invocation**: The system prompt guides the LLM to analyze the coordinates, connectivity, and user instructions, generating both conversational feedback and structured layout command blocks ('[CMD]').
3. **Prescriptive Refinements**: Once command blocks are parsed and applied to nodes, the node executes automatic geometrical constraints:
    - **Mirror Symmetry Enforcement**: Runs `enforce_reflection_symmetry()` to ensure symmetry groups stay perfectly aligned across the central axis.

- **Overlap Resolution**: Runs a physical mechanical sweep-line overlap resolver (`resolve_overlaps()`) to adjust coordinates horizontally and clear grid collisions.
- **Global Row Orientation Optimization**: Runs `optimize_all_rows_orientations()` post-overlap resolution. This algorithm determines the optimal source/drain orientations to maximize shared-diffusion abutments and eliminate shorts across row elements, executing:

$$\text{Orientation}(M_i) \in \{R0, MY\} \tag{6.1}$$

to guarantee minimum wire routing crossings.

4. **Device Conservation Check**: Performs `validate_device_count()` to protect layout integrity, falling back to original positions if devices are missing.

Listing 6.3.3 illustrates the core logical pipeline of the chat-mode placement specialist.

Placement Specialist Chatbot Node (placement_specialist.py)

```
def node_placement_specialist_chatbot(state):
    t0 = time.time()
    nodes            = state.get("nodes", [])
    constraint_text  = state.get("constraint_text", "")
    user_message     = state.get("user_message", "")
    chat_history     = state.get("chat_history", [])
    edges            = state.get("edges", [])
    terminal_nets    = state.get("terminal_nets", {})
    strategy_result  = state.get("strategy_result", "auto")
    selected_model   = state.get("selected_model", "Gemini")
    no_abutment_flag = state.get("no_abutment", False)
    drc_passed       = state.get("drc_pass", False)
    drc_violations   = state.get("drc_flags", [])

    working_nodes = state.get("placement_nodes", []) or copy.deepcopy(nodes)

    # Build placement context
    context_text = build_placement_context_chatbot(
        nodes, constraint_text, terminal_nets=terminal_nets,
        edges=edges, no_abutment=no_abutment_flag,
    )

    drc_note = ""
    if not drc_passed and drc_violations:
```

```
25          violations_text = "\n".join(str(v) for v in
               drc_violations)
26          drc_note = f"DRC failed in the previous step. Fix
               these violations:\n{violations_text}\n\n"
27
28      placer_user = f"User request: {user_message}\n\
           nSelected Strategy: {strategy_result}\n\n{drc_note
           }{context_text}"
29
30      placement_result = _invoke_react_agent_with_retry(
31          system_prompt=_PLACEMENT_SYSTEM_PROMPT,
               chat_history=chat_history,
32          user_prompt=placer_user, selected_model=
               selected_model,
33          task_weight="heavy", stage_tag="PLACEMENT",
34      )
35
36      placement_response_text, thinking =
           _extract_agent_output_parts(placement_result)
37      stage2_cmds, placement_text = extract_cmd_blocks(
           placement_response_text)
38
39      # Apply commands to nodes
40      working_nodes = apply_cmds_to_nodes(nodes, stage2_cmds
           )
41
42      # Enforce mirror symmetry and resolve layout overlaps
43      working_nodes = enforce_reflection_symmetry(
           working_nodes)
44      resolve_overlaps(working_nodes)
45      working_nodes = legalize_vertical_rows(working_nodes)
46
47      # Optimize S/D orientations globally for abutment
           diffusion-sharing
48      from ai_agent.placement.abutment import
           optimize_all_rows_orientations
49      working_nodes = optimize_all_rows_orientations(
           working_nodes, terminal_nets)
50
51      # Validate conservation
52      conservation = validate_device_count(nodes,
           working_nodes)
53      if not conservation["pass"]:
54          working_nodes = copy.deepcopy(nodes)
55          stage2_cmds   = []
56
57      updated_chat_history = _update_and_save_chat_history(
58          chat_history=chat_history, user_content=
```

```
            user_message,
59          node_role="Placement Specialist Assistant",
            node_content=placement_text,
60      )
61
62      return {
63          "placement_nodes":          working_nodes,
64          "pending_cmds":             stage2_cmds,
65          "original_placement_cmds": stage2_cmds,
66          "chat_history":             updated_chat_history,
67          "placement_text":           placement_text,
68          "last_agent":               "placement_specialist",
69      }
```

6.3.4 Line-by-Line Analysis of node_placement_specialist_chatbot()

The interactive placer translates text directives into coordinates using the following implementation structure:

- **Line 17-20**: Gathers coordinates and design constraints from the unified state data model.
- **Line 24**: Serializes physical device attributes. It maps active coordinate positions (x, y), width, and current transistor orientations to establish spatial constraints.
- **Line 29-32**: Checks prior DRC validation status. If the previous run failed, the violations are formatted and appended to the prompt, enabling the ReAct agent to adjust placements to clear rules.
- **Line 36**: Invokes the heavy reasoning agent. Using `task_weight="heavy"`, the system allocates a larger model (e.g. Gemini Pro) to process coordinate calculations.
- **Line 42**: Parses the LLM's response. It splits natural language comments from structured JSON-RPC instructions bounded by '[CMD]' tags.
- **Line 45**: Executes coordinate changes. The engine modifies the physical coordinates of the target devices based on the parsed instructions.
- **Line 48-50**: Legalizes geometry. It enforces reflection symmetry across the layout axis, resolves transistor coordinate overlaps, and snaps devices to row heights.
- **Line 54**: Optimizes diffusion sharing. It scans rows and flips transistor orientations ($R0 \leftrightarrow MY$) to share contacts and reduce parasitic capacitance.
- **Line 57-60**: Verifies device conservation. It compares the modified device list against the original netlist to ensure no transistors were deleted, falling back to original positions if verification fails.

6.4 Frontend-Backend Asynchronous Architecture

Analog layout editors require a highly responsive user interface. Running heavy multi-agent LLM inference directly on the GUI main thread would cause the interface to freeze during API calls. LayoutCopilot resolves this by adopting a decoupled *Worker-Object Pattern* implemented via PySide6 threads and signals. Figure 6.2 visualizes this interaction model.

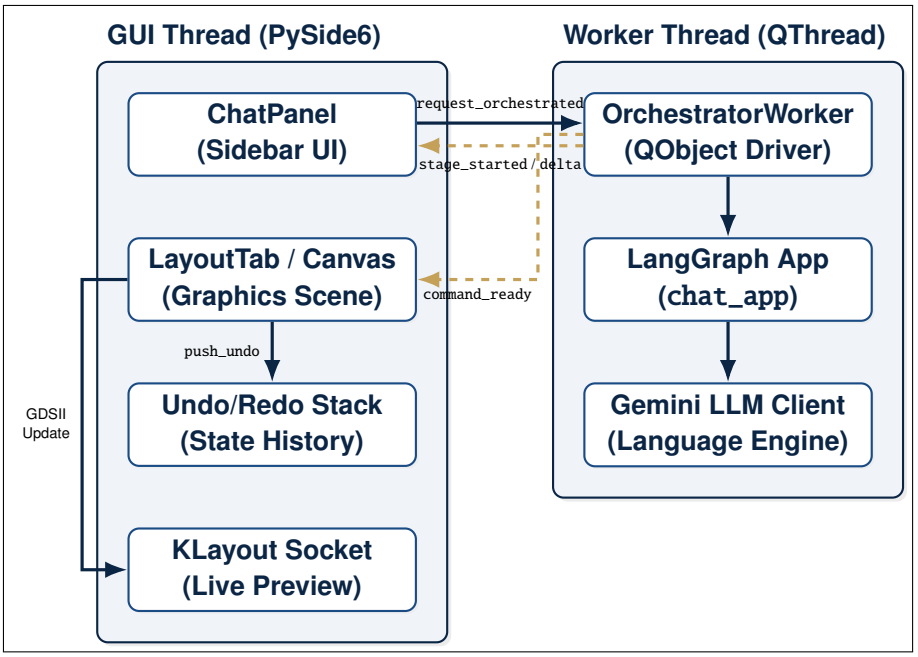


Figure 6.2: Asynchronous Frontend-Backend Interaction Model (Worker-Object Pattern).

The `ChatPanel` sidebar GUI widget instantiates a background `QThread` and moves an `OrchestratorWorker` instance to it. Figure 6.3 details the complete communication sequence between thread entities.

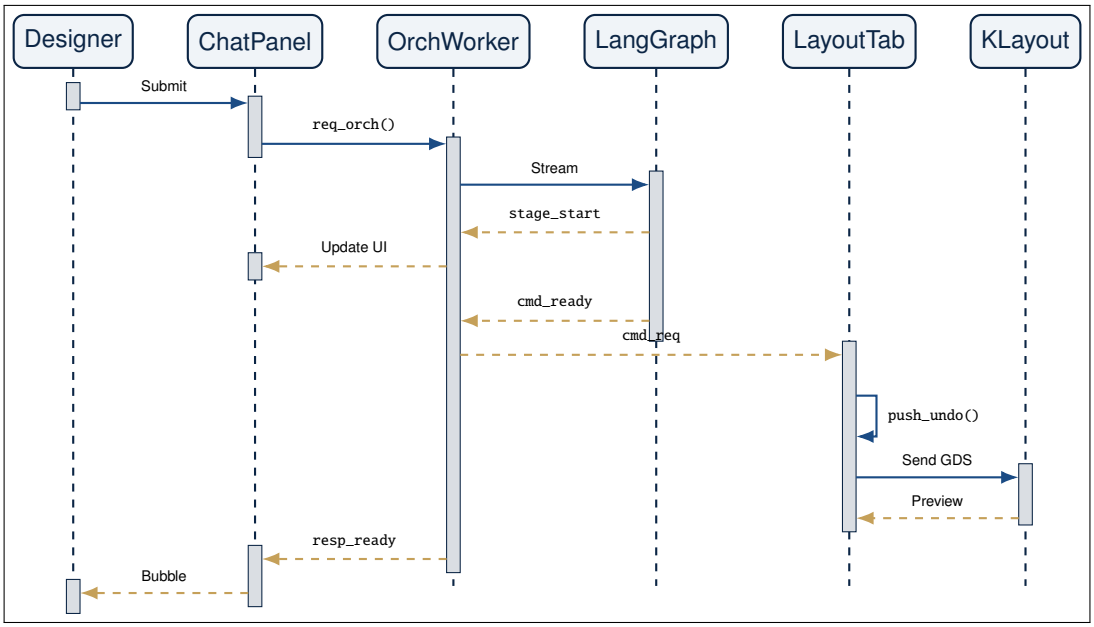


Figure 6.3: Thread Communication Sequence for an Interactive Layout Editing Turn.

The thread orchestration loop operates as follows:

1. The user types a message and hits submit. If the message matches multi-agent keywords (e.g. *"optimize"*, *"align"*), it triggers the `request_orchestrated` signal.
2. The signal is caught by the background worker thread, which runs `process_orchestrated_request` without blocking the main event loop.
3. The worker streams events during LangGraph execution. It emits `stage_started` and `stage_delta` signals to update status card animations in the GUI.
4. It emits `response_delta` to stream text updates to the chat bubble.
5. If the LLM generates layout adjustment commands, the worker extracts them and emits a thread-safe `command_ready` signal, carrying the commands back to the main thread for execution.

Listing 6.4 displays the background streaming loop inside `Orchestrator Worker`.

Asynchronous LangGraph Streaming in worker thread (workers.py)

```
1 def _stream_graph(self, input_data):
2     try:
3         from ai_agent.graph.builder import chat_app as
              chat_graph_app
4         langgraph_app = chat_graph_app
5
```

```
        vprint("\n[GRAPH] Streaming LangGraph...", flush=
            True)
        active_stage_key = None

        for event in langgraph_app.stream(input_data, self
            .thread_config, stream_mode="updates"):
            if "__interrupt__" in event:
                interrupt_data = event["__interrupt__"
                    ][0].value
                itype = interrupt_data.get('type', '?')

                if itype == "visual_review":
                    pending_cmds = interrupt_data.get("
                        pending_cmds", [])
                    last_agent = interrupt_data.get("
                        last_agent")
                    text = interrupt_data.get("General", "
                        ") if last_agent == "general" else
                        ""

                    if text:
                        self.response_ready.emit(text)
                    if pending_cmds:
                        self.visual_viewer_signal.emit({
                            "type": "visual_review",
                            "pending_cmds": pending_cmds,
                        })
                    return

            # Emit stage signals to update GUI status
                cards
            for node_key, node_value in event.items():
                if node_key == "__interrupt__":
                    continue
                title = self.NODE_DISPLAY.get(node_key,
                    node_key)
                if active_stage_key is not None and
                    active_stage_key != node_key:
                    self.stage_done.emit(active_stage_key,
                        "")
                self.stage_started.emit(node_key, title)
                active_stage_key = node_key

                summary = self._short_summary(node_value)
                if summary:
                    self.stage_delta.emit(node_key,
                        summary)
    except Exception as exc:
```

```
42            self.error_occurred.emit(f"Streaming failed: {exc}
                  ")
```

6.4.1 Line-by-Line Analysis of _stream_graph()

The stream controller handles concurrent thread updates using the following implementation structure:

- **Line 8**: Begins the graph update stream. The generator yields dictionary deltas representing the state updates returned by each node in the active graph.
- **Line 9-11**: Intercepts node execution checks. If the graph reaches an active interrupt (such as the human visual review node), execution pauses, and the thread state is cached.
- **Line 15-26**: Forwards intermediate layout results to the GUI thread. If layout commands are pending, the worker emits the thread-safe `visual_viewer_signal` carrying the commands, which triggers the layout editor view update.
- **Line 30-32**: Standardizes node name rendering. It retrieves user-facing names from `NODE_DISPLAY` (e.g., mapping `"node_drc_critic"` to `"DRC Critic"`).
- **Line 33-35**: Signals state changes. It fires the QObject signal `stage_started` to notify the GUI to animate the corresponding progress card.
- **Line 37-39**: Captures and forwards node status logs. It extracts brief status strings (such as *"DRC failed"* or *"5 commands pending"*) using `_short_summary()` and passes them to the GUI thread via `stage_delta`.

6.5 Context-Aware Prompts & XML Command Synthesis

6.5.1 Context Generation

For the LLM to understand layout adjustments, the chatbot serializes the current layout canvas state into a structured text context using `build_placement_context_chatbot()`. This context lists all placed and unplaced transistors, their coordinate positions (in micrometers), current orientations (e.g., $R0$, MY), physical dimensions, and shared net connections.

6.5.2 XML Command Synthesis

If the user requests coordinate edits (e.g., *"Move MM2 to the right by 1 pitch and flip it"*), the LLM reasons over the spatial coordinates and generates structured command tags embedded in its response:

Synthesized AI Placement Command Block

```
[CMD]{"action": "move", "device": "MM2", "x": 1.764, "y":
   0.818, "orientation": "MY"}[/CMD]
```

The PySide6 interface uses regular expressions to extract these command blocks from the text stream:

$$\text{Pattern: } \texttt{\textbackslash[CMD\textbackslash](.*?)\textbackslash[/CMD\textbackslash]} \tag{6.2}$$

The extracted strings are parsed into JSON dictionaries, containing properties like action, device ID, and target coordinates.

6.6 Thread-Safe Undo/Redo Engine & Live Rendering

6.6.1 Timer-Based Command Batching

To prevent screen flickering and excessive graphics scene recalculations, commands received from the AI thread are batched and executed within a single thread-safe event loop.

When a command arrives via the signal `command_requested`, it is appended to a list. A single-shot `QTimer` is started with 0 ms delay. Once the thread returns control to the event loop, the timer triggers `_flush_ai_command_batch()`, capturing a single undo state snapshot and executing all commands as one transaction.

6.6.2 Undo State Stacking

LayoutCopilot wraps layout node arrays into a list-based undo stack. Before the batch execution starts, `_push_undo()` clones the current layout node dictionary. If the user chooses to undo (`Ctrl+Z`), the previous state is popped from `_undo_stack`, restoring coordinates and orientations. Listing 6.6.2 illustrates this execution logic.

Batch Command Execution and Undo Stacking (layout_tab.py)

```
def _enqueue_ai_command(self, cmd):
    self._pending_cmds.append(cmd)
    if self._batch_flush_timer is None:
        self._batch_flush_timer = QTimer(self)
        self._batch_flush_timer.setSingleShot(True)
```

```
6           self._batch_flush_timer.timeout.connect(self.
                _flush_ai_command_batch)
7       self._batch_flush_timer.start(0)
8
9   def _flush_ai_command_batch(self):
10      cmds = list(self._pending_cmds)
11      self._pending_cmds.clear()
12      self._batch_flush_timer = None
13      if not cmds:
14          return
15
16      print(f"[AI BATCH] Executing {len(cmds)} command(s) as
            one undo group")
17      self._sync_node_positions()
18      self._push_undo()  # Capture single snapshot before
            modifying positions
19
20      for cmd in cmds:
21          self._handle_ai_command(cmd, _skip_undo=True)
22
23      self._refresh_panels(compact=False)
24      self._sync_node_positions()
25
26  def _push_undo(self):
27      if not self.nodes:
28          return
29      self._undo_stack.append(copy.deepcopy(self.nodes))
30      self._redo_stack.clear()
31      self.undo_state_changed.emit(bool(self._undo_stack),
            bool(self._redo_stack))
```

6.6.3 Line-by-Line Analysis of Batch Execution and Undo Stacking

The layout editor aggregates geometric modifications using the following code steps:

- **Line 2-6**: Queues incoming commands. Each parsed AI instruction is appended to `self._pending_cmds`. If the batch flush timer is not active, it is configured as a single-shot timer.
- **Line 7**: Starts the flush timer with a 0 ms delay. This schedules the execution task on the Qt event loop, allowing multiple individual incoming commands from a single AI generation step to be collected and run together.
- **Line 10-13**: Flushes the queued command batch. Once the timer expires, the queued commands are copied and cleared.

- **Line 17**: Clones the active layout state using `_push_undo()`. This pushes a deep-copied snapshot of all device coordinates and properties onto `_undo_stack` before any changes are applied, enabling single-transaction rollbacks.
- **Line 19-20**: Executes commands with `_skip_undo=True` to prevent intermediate coordinate changes from polluting the undo history.
- **Line 22-23**: Rebuilds the layout scene and synchronizes coordinate structures.
- **Line 28-31**: Manages the undo/redo stacks. It pushes the snapshot state onto `_undo_stack`, clears the redo stack, and fires signals to update the toolbar undo/redo buttons.

6.6.4 Command Execution Engine

Listing 6.6.4 shows the core execution switch handling layout edits.

AI Command Interpretation Switch (layout_tab.py)

```
1  def _handle_ai_command(self, cmd, _skip_undo=False):
2      if not isinstance(cmd, dict):
3          return
4      action = str(cmd.get("action", "")).strip().lower()
5      try:
6          if action in {"replace_layout", "apply_layout"}:
7              new_nodes = cmd.get("nodes")
8              if isinstance(new_nodes, list):
9                  if not _skip_undo:
10                     self._push_undo()
11                 self.nodes = copy.deepcopy(new_nodes)
12                 self._refresh_panels(compact=False)
13                 self._sync_node_positions()
14
15         elif action in {"swap", "swap_devices"}:
16             raw_a = cmd.get("device_a", cmd.get("a"))
17             raw_b = cmd.get("device_b", cmd.get("b"))
18             id_a = self._resolve_device_id(raw_a)
19             id_b = self._resolve_device_id(raw_b)
20             if id_a and id_b:
21                 if not _skip_undo:
22                     self._push_undo()
23                 node_a = next((n for n in self.nodes if n.
                       get("id") == id_a), None)
24                 node_b = next((n for n in self.nodes if n.
                       get("id") == id_b), None)
25                 if node_a and node_b:
```

```
26                      ga, gb = node_a["geometry"], node_b["
                            geometry"]
27                      ga["x"], gb["x"] = gb["x"], ga["x"]
28                      ga["y"], gb["y"] = gb["y"], ga["y"]
29                      oa, ob = ga.get("orientation", "R0"),
                            gb.get("orientation", "R0")
30                      ga["orientation"], gb["orientation"] =
                             ob, oa
31                      self._refresh_panels(compact=False)
32
33          elif action == "abut":
34              raw_a = cmd.get("device_a")
35              raw_b = cmd.get("device_b")
36              id_a = self._resolve_device_id(raw_a)
37              id_b = self._resolve_device_id(raw_b)
38              if id_a and id_b:
39                  if not _skip_undo:
40                      self._push_undo()
41                  self._abut_devices(id_a, id_b)
42                  self._refresh_panels(compact=False)
43
44          elif action in {"move", "move_device"}:
45              raw_dev = cmd.get("device", cmd.get("id"))
46              x, y = cmd.get("x"), cmd.get("y")
47              dev_id = self._resolve_device_id(raw_dev)
48              if dev_id is not None and x is not None and y
                    is not None:
49                  if not _skip_undo:
50                      self._push_undo()
51                  node = next((n for n in self.nodes if n.
                        get("id") == dev_id), None)
52                  if node:
53                      node["geometry"]["x"] = float(x)
54                      node["geometry"]["y"] = float(y)
55                      self._refresh_panels(compact=False)
56      except Exception as exc:
57          print(f"[AI CMD ERROR] Failed command {action}: {
                exc}")
```

6.6.5 Line-by-Line Analysis of _handle_ai_command()

The command parser interprets incoming action messages using the following code steps:

- **Line 4**: Extracts the action identifier and normalizes the string to lowercase to handle variant inputs.
- **Line 6-13**: Processes structural layout updates. It overrides the active node

array with a new geometry map, updating the canvas view.

- **Line 15-28**: Implements the device swapping command. It extracts device identifiers, resolves their corresponding IDs, swaps their spatial center coordinates (x, y), and flips their orientations to maintain connectivity.
- **Line 30-38**: Implements the abutment command. It resolves target device IDs and invokes `_abut_devices()` to compact their shared diffusion regions.
- **Line 40-50**: Implements the absolute coordinate movement command. It extracts target coordinates (x, y), retrieves the target device, and updates its spatial geometry structure.

6.6.6 PDK Grid Snapping and Geometric Offsets Math

To ensure all manually or AI-edited placement coordinates remain physical design rule clean, coordinates are snapped to the PDK grid pitch. Let the raw target coordinate output from the parser be X_{raw} and Y_{raw}. The snapped coordinate is evaluated as:

$$X_{\text{snapped}} = \text{round}\left(\frac{X_{\text{raw}}}{P_{\text{grid}}}\right) \cdot P_{\text{grid}} \tag{6.3}$$

$$Y_{\text{snapped}} = \text{round}\left(\frac{Y_{\text{raw}}}{P_{\text{grid}}}\right) \cdot P_{\text{grid}} \tag{6.4}$$

where $P_{\text{grid}} = 0.294\,\mu\text{m}$ represents the standard technology transistor layout pitch (snapping coordinate boundary).

Symmetry groups must be mirrored symmetrically about the row center axis X_{mid}. Let a device M_1 belong to a symmetric pair with partner M_2. When M_1 is moved to coordinate X_1, the coordinator automatically computes the mirrored position X_2 of M_2 as:

$$X_2 = 2 \cdot X_{\text{mid}} - X_1 - W_{\text{device}} \tag{6.5}$$

where W_{device} represents the width of the physical transistor cell.

6.6.7 Headless KLayout Live Rendering Preview

To visualize coordinates as physical layout shapes, the editor communicates with a background headless `klayout` process via a TCP socket:

1. When the AI updates the placement state, the editor generates the GDSII layout database.
2. The layout changes are sent to the KLayout server, which renders the GDSII file.

3. The rendered image is sent back to the PySide6 application, which updates the layout preview window.

This provides analog designers with immediate visual feedback on AI-generated layout edits.

6.7 Case Study: Interactive Editing Session of a Differential Pair

To demonstrate the capabilities of the interactive chatbot, this section presents a step-by-step trace of a design session editing a differential input pair cluster. The netlist contains two active matched transistors, `MM0` and `MM1`, along with two dummy transistors, `MDUM0` and `MDUM1`.

6.7.1 Initial State Configuration

The initial state of the layout nodes in 'LayoutState' is configured as follows:

- `MM0`: Located at $X = 0.0\,\mu$m, $Y = 0.0\,\mu$m, orientation = `R0`.
- `MM1`: Located at $X = 0.588\,\mu$m, $Y = 0.0\,\mu$m, orientation = `R0`.
- `MDUM0`: Located at $X = -0.294\,\mu$m, $Y = 0.0\,\mu$m, orientation = `R0`.
- `MDUM1`: Located at $X = 0.882\,\mu$m, $Y = 0.0\,\mu$m, orientation = `R0`.
- The central symmetry axis is configured at $X_{\text{mid}} = 0.294\,\mu$m.

6.7.2 Designer Command and Intent Routing

The designer enters the following query:

> *"Swap the locations of MM0 and MM1, and then flip MM1 to MY orientation to share source diffusion."*

The query execution flow is evaluated step-by-step:

1. **Intent Routing**: The message enters the `router` node. The regex classifier identifies the keywords *"swap"* and *"flip"*, immediately routing the request to the `placement_specialist` node, bypassing the LLM classifier to save execution time.
2. **Command Synthesis**: The placement specialist invokes the ReAct agent. The language model generates the following structured layout adjustment actions inside a single '[CMD]' block:

Synthesized Swap and Move Command Blocks

```
[
  {"action": "swap", "device_a": "MM0", "device_b": "MM1"},
  {"action": "move", "device": "MM1", "x": 0.0, "y": 0.0, "orientation": "MY"}
]
```

3. **Batch Execution**: The GUI receives the commands. The signals queue the actions, capture a single undo history snapshot, and execute the edits:
   - `MM0` coordinates shift to $X = 0.588\,\mu$m, $Y = 0.0\,\mu$m.
   - `MM1` coordinates shift to $X = 0.0\,\mu$m, $Y = 0.0\,\mu$m with orientation `MY`.
4. **Post-Processing Adjustments**:
   - **Symmetry Check**: The enforcer validates the positions about $X_{\text{mid}} = 0.294\,\mu$m. Since $X_1 = 0.0$ and $X_2 = 0.588$ satisfy:
   $$0.588 = 2 \cdot (0.294) - 0.0 - 0.294 \cdot (0) \tag{6.6}$$
   symmetry is verified.
   - **Diffusion Sharing**: The orientation optimizer evaluates the net connections. Since the source terminals of `MM0` and `MM1` share the same net (`NET_TAIL`), and `MM1` is oriented to `MY`, their adjacent channels are aligned. The compiler shrinks the spacing coordinate boundary, decreasing the width from $1.176\,\mu$m to $0.952\,\mu$m.
5. **DRC Validation**: The `drc_critic` node runs. The clearance check reports zero design rule violations. The state returns `drc_pass = True`, and the graph terminates at the human review view, displaying the modified, compacted layout on the PySide6 canvas.

6.8 Interactive Schematic Assistant & Bi-Directional Cross-Highlighting

To facilitate intuitive verification and debugging of analog layout designs, the framework integrates an interactive, dockable *Schematic Assistant*. This assistant allows designers to visualize the logical connections of the SPICE netlist in real-time, providing immediate visual feedback alongside the physical and symbolic layout views. By linking the logical schematic topology directly with the physical layout geometries, the framework bridges the cognitive gap between symbolic block planning and physical silicon layout compilation.

6.8.1 System Architecture and UI Integration

The Schematic Assistant is implemented as a specialized dockable widget (`SchematicPanel` in `schematic_view.py`) that resides in the left sidebar layout of the PySide6 application. It is composed of a custom graphic view rendering canvas (`SchematicCanvas`) and a toolbar containing commands for fitting the view, refreshing the layout, and clearing selections. It interacts directly with the layout tab controller to sync active device lists and node definitions.
The rendering engine utilizes Qt's graphics-view framework, where the `SchematicCanvas` manages a custom `QGraphicsScene`. Each transistor is represented by a custom `TransistorItem` (inheriting from `QGraphicsItem`) that overrides the native `paint()` method to draw standard IEEE-compliant NMOS and PMOS circuit symbols. The symbol painting includes:

- **Channel and Gate Poly:** Rectilinear line stubs representing the transistor gate oxide and poly silicon.
- **Bulk Connection Arrow:** An arrow symbol indicating the bulk terminal orientation (pointing inward for NMOS, pointing outward for PMOS).
- **Terminal Pins:** Connection ports rendered as small, hover-sensitive squares representing the Drain, Gate, Source, and Bulk nodes.

In addition to transistor symbols, connection nets are rendered as `WireItem` graphics objects. Wires are routed using a simple rectilinear (Manhattan) router that joins the coordinate ports of connected devices. External connections (such as inputs, outputs, bias lines, and supply rails) are capped with a `LabelItem` indicating the net name, preventing the scene from becoming cluttered with long global routing lines.

6.8.2 Dual-Engine Layout Generation

When a cell is loaded or updated, the Schematic Assistant lays out the schematic components using a two-stage layout generation flow:

1. **Deterministic Fallback Banding:** Initially, a fast, deterministic banding algorithm (`build_band_layout`) groups the transistors horizontally based on netlist connections. It partitions the active area into PMOS and NMOS rows and routes basic connecting lines to power rails (VDD and GND) at the cell boundaries. This provides a fallback visualization if the Vertex AI API is unavailable.
2. **Asynchronous Vertex AI Layout Engine:** To produce highly structured and professional schematic arrangements that follow industrial drafting conventions, the canvas launches a background thread that invokes Google Cloud Vertex AI (using the Gemini-2.5-flash model via the `google.genai` SDK).

6.8.3 Symmetric Prompting and LLM Constraints

The background task builds a detailed structured prompt (`_build_prompt()`) that serializes the SPICE netlist (drain, gate, and source pin-to-net mappings) and lists identified topological groupings (such as differential pairs, current mirrors, tail current sources, and cross-coupled latches).
The LLM is prompted to assign integer grid coordinates (x, y) and gate mirror flags (`mirrored`) according to standard analog drafting conventions:

- **Signal Flow direction:** PMOS devices are strictly placed in the upper half of the grid ($y < 0$) and NMOS devices in the lower half ($y \geq 0$) to align with standard power-to-ground current flow.
- **Symmetry axes:** Matched differential pairs are placed symmetrically around the center axis $x = 0$ (e.g., at $x = -1.0$ and $x = 1.0$) with the right-side device gate mirrored inward.
- **Cross-Coupled Latches:** Latches are aligned at adjacent NMOS/PMOS boundaries and configured to face each other (using complementary mirror states) to clearly visualize latch feedback loops.
- **Manhattan Local Wires:** The model outputs local, low-pin count nets to draw as solid connecting wires, excluding power, ground, and port lines which are drawn as clean labels to avoid layout clutter.
- **Latch Cross-Wires:** If a cross-coupled latch is active, the model synthesizes two crossing segments that render as a classic "X" shape on the schematic canvas.

To prevent the GUI thread from freezing during Vertex AI model generation, the layout calculation runs inside an asynchronous, non-blocking Python `threading.Thread`. Once model inference completes, the worker posts a signal carrying the JSON-RPC response back to the main GUI thread, which parses the coordinates, scales them to canvas pixels (multiplying the coordinates by a grid scale factor of 160 pixels), and refreshes the graphics scene dynamically.

6.8.4 Bi-Directional Cross-Highlighting

The Schematic Assistant enables a "Human-in-the-Loop" validation workflow through bi-directional cross-highlighting:

- **Transistor Highlighting:** Clicking on any transistor symbol in the schematic canvas emits a `device_clicked` signal. This highlights the corresponding transistor's gate poly fingers, contacts, and active regions in the main physical layout editor, allowing the designer to quickly locate devices.
- **Net Highlighting:** Clicking on any connecting wire or net label in the schematic emits a `net_clicked` signal. The GUI controller immediately highlights all physical layout metal wires, routing tracks, and pin terminals belonging to that net name, providing a rapid method for verifying trace connectivity.

This cross-highlighting system is implemented using PySide6 Qt Signals. When an item is clicked in the `SchematicCanvas`, a signal carrying the device name or net ID is emitted. The main `layout_tab.py` controller intercepts these signals:

```
self.schematic_panel.highlight_device.connect(
    self.editor.highlight_device
)
self.schematic_panel.highlight_net.connect(
    self.editor.highlight_net_by_name
)
```

The controller forwards the selection to the layout editor canvas (`EditorView`), which highlights the matched transistor or net by selection, dims the surrounding layout items, and renders connection flightlines to the selected pins. Conversely, selecting a device finger or routing track in the main symbolic layout view propagates a selection event back to the `SchematicPanel`, highlighting the corresponding transistor symbol or net wire in red. This bi-directional interaction ensures that changes in layout planning can be immediately mapped to schematic topologies.

CHAPTER 7

Stage 4: Compaction & Export Pipelines

The physical execution layer translates symbolic placements into final silicon geometries. This phase runs a compaction pass on symbolic slots and exports layouts through a double-exporter pipeline. While symbolic representations are useful for spatial planning and machine learning optimization, physical design rules require exact coordinates snapped to PDK grids, tap/tie cell placement, and terminal net connectivity.

This chapter details the mathematical formulations of the diffusion compaction engine, the net compatibility rules for abutment, the graph serialization JSON flow, the variant PCell cloning OASIS exporter, and the multi-phase OpenAccess TCL database exporter.

7.1 Spatial Diffusion-Sharing Compaction & Net Compatibility

Standard symbolic layouts align transistors on a coarse grid ($0.294\,\mu$m) to simplify editing. During compilation, the physical engine compresses shared source/drain regions down to a pitch of $0.070\,\mu$m. Let a transistor row R be defined as an ordered list of device fingers:

$$R = \{f_1, f_2, \ldots, f_k\}$$

Each finger f_i is mapped to a symbolic slot index $s_i \in \mathbb{Z}^+$. The symbolic coordinate spacing is defined as:

$$X_{\text{symbolic}}(f_i) = s_i \times P_{\text{symbolic}}$$

where $P_{\text{symbolic}} = 0.294\,\mu$m represents the loose symbolic pitch.

During compact export, the physical coordinate engine resolves the physical spacing using local abutment states. Let $A(f_i, f_{i+1}) \in \{0, 1\}$ represent the contact sharing (abutment) coefficient:

$$A(f_i, f_{i+1}) = \begin{cases} 1 & \text{if net}(D_i) = \text{net}(S_{i+1}) \text{ (shared diffusion)} \\ 0 & \text{otherwise} \end{cases}$$

The physical compaction coordinate $X_{\text{physical}}(f_n)$ is recursively evaluated as:

$$X_{\text{physical}}(f_1) = 0$$

$$X_{\text{physical}}(f_n) = X_{\text{physical}}(f_{n-1}) + W_{\text{finger}} + D(f_{n-1}, f_n)$$

where the boundary distance function $D(f_{n-1}, f_n)$ matches:

$$D(f_{n-1}, f_n) = A(f_{n-1}, f_n) \cdot P_{\text{abut}} + (1 - A(f_{n-1}, f_n)) \cdot P_{\text{clearance}}$$

* $P_{\text{abut}} = 0.070\,\mu$m (shared diffusion pitch). * $P_{\text{clearance}} = 0.294\,\mu$m (default isolated contact clearances).

To ensure that the layout is physically realizable, the system validates the electrical compatibility of adjacent terminals before enabling compaction. The helper function `_adjacent_devices_can_abut()` implements these checks:

- If either device is flagged as a dummy transistor, it is treated as an empty diffusion container and can always abut the adjacent active device.
- If the adjacent instances belong to the same logical device (matching base IDs), they are physically contiguous and are abutted.
- If their adjacent physical boundaries share the same electrical net (excluding non-connected nets), they are compatible for contact sharing.

The physical boundary nets are computed dynamically by `_get_physical_boundary_nets()` which takes the device configuration, total fingers nf, orientation (mirroring), and net connections:

$$\text{phys\_left} = \begin{cases} S & \text{if (swapped} \oplus \text{mirrored)} = 1 \\ D & \text{otherwise} \end{cases} \tag{7.1}$$

$$\text{phys\_right} = \begin{cases} S & \text{if (swapped} \oplus \text{mirrored} \oplus \text{is\_even)} = 1 \\ D & \text{otherwise} \end{cases} \tag{7.2}$$

where swapped indicates if the source and drain nets are physically swapped relative to the schematic, mirrored indicates if the device orientation is left-right mirrored (`MY` or `R180`), and is_even is true if the finger count nf is even.

7.2 JSON Graph Placement Exporter

Before the layout can be optimized by the multi-agent AI placement engine, the unified NetworkX database graph is serialized to a standard placement JSON schema. The exporter script `export_json.py` loops over graph nodes and edges, serializing electrical and geometric parameters to enable coordinate snapshots. Listing 7.2 shows the complete implementation.

JSON Layout Graph Exporter (export_json.py)

```
1 import json
2
3 def graph_to_json(G, output_file):
```

```
    """
    Convert merged NetworkX graph to JSON file
    for AI placement agent.
    """
    data = {
        "nodes": [],
        "edges": []
    }

    # Export nodes
    for node, attrs in G.nodes(data=True):
        node_entry = {
            "id": node,
            "type": attrs["type"],
            "electrical": {
                "l": attrs["l"],
                "nf": attrs["nf"],
                "nfin": attrs["nfin"]
            },
            "geometry": {
                "x": attrs["x"],
                "y": attrs["y"],
                "width": attrs["width"],
                "height": attrs["height"],
                "orientation": attrs["orientation"]
            }
        }
        data["nodes"].append(node_entry)

    # Export edges
    for u, v, attrs in G.edges(data=True):
        edge_entry = {
            "source": u,
            "target": v,
            "net": attrs.get("net", "")
        }
        data["edges"].append(edge_entry)

    # Write file
    with open(output_file, "w") as f:
        json.dump(data, f, indent=4)

    print(f"\nJSON exported to {output_file}")
```

7.2.1 Line-by-Line Analysis of graph_to_json()

The serialization script maps graph objects using the following code blocks:

- **Lines 8–11**: Initializes the JSON document structure. The dictionary contains two distinct arrays, `"nodes"` and `"edges"`, matching the schema format expected by the LangGraph agents.
- **Lines 14–31**: Iterates over the NetworkX nodes. It serializes the instance name, device type (nmos, pmos, resistor, capacitor), electrical parameters (gate length `l`, finger multiplier `nf`, number of fins `nfin`), and physical coordinate parameters (`x`, `y`, `width`, `height`, and orientation `orientation`) into a single node dictionary.
- **Lines 34–40**: Iterates over the NetworkX edges. It extracts the source and target node identifiers, along with the interconnect net label, and appends them to the edges array.
- **Lines 43–46**: Writes the formatted dictionary to the target output text file using standard indentation of 4 spaces to ensure human readability.

7.3 Binary OASIS Layout Exporter & Variant Cell Redirection

The binary OASIS layout assembler reads the compacted coordinate maps, imports physical tap/tie cells, and compiles the geometries using the `gdstk` library.

In sub-micron processes (such as SAED 14nm), physical transistor abutment is encoded using variant cell definitions. Rather than drawing custom geometries on the fly, the PDK PCells are configured with `leftAbut` and `rightAbut` parameters. When these parameters are modified, the layout compiler removes the end-cap diffusion regions, allowing adjacent cells to share a single active diffusion strip.

The OASIS writer implements a catalog-driven variant redirection pipeline to update these parameters:

1. **Variant cataloging**: The writer parses the input OASIS database, reads the existing cells, and builds a variant catalog. Each entry is keyed by a parameter hash (generated by MD5-hashing the cell's parameters excluding the abutment flags).
2. **Reference redirection**: For each transistor that requires manual abutment annotations, the writer checks if a matching variant cell (same parameters and matching abutment flags) already exists in the catalog. If it does, the cell reference is immediately redirected to it.
3. **Cell cloning**: If a matching variant does not exist, the writer duplicates the base cell and patches its `pcell` property bytes to configure the target abutment parameters. The cell reference is then updated to point to this new variant.

Figure 7.1 visualizes this catalog variant generation and reference redirection flow.

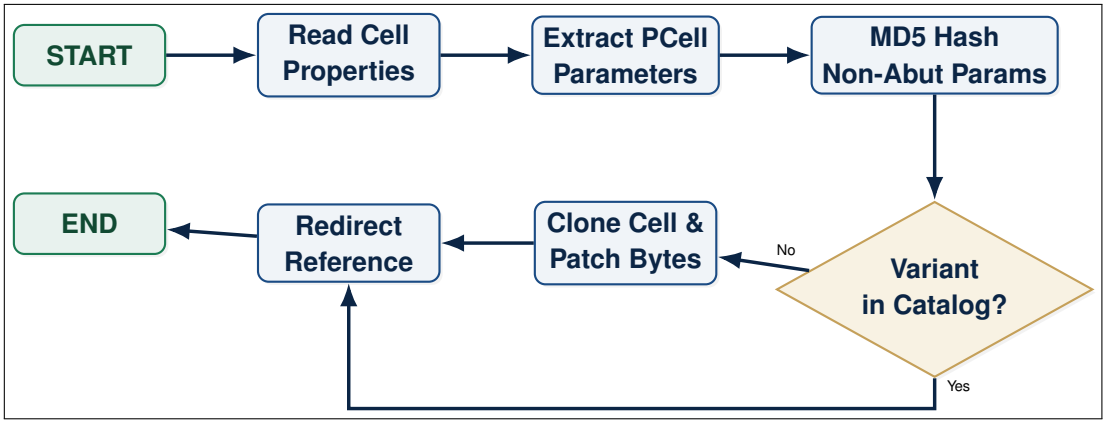


Figure 7.1: OASIS Catalog Variant Generation and Cell Reference Redirection Flowchart.

Listing 7.3 shows the Python code inside `oas_writer.py` that generates a new cell variant and patches its PCell OpenAccess properties.

Variant Cell Generation & Property Patching (oas_writer.py)

```
def _make_variant_cell(lib, base_cell, base_type, raw_prop
    , al: bool, ar: bool) -> "gdstk.Cell":
    """Clone base_cell and patch its pcell property
       abutment flags.
    Also physically trims the geometry to create the
       abutment effect.
    """
    al_str = "1" if al else "0"
    ar_str = "1" if ar else "0"
    variant_name = f"{base_type}_manual_l{al_str}r{ar_str}
       _{base_cell.name[-4:]}"

    for c in lib.cells:
        if c.name == variant_name:
            return c

    # Create a fresh copy manually to avoid duplication/
       linkage issues
    new_cell = gdstk.Cell(variant_name)
    lib.add(new_cell)

    # Copy raw polygons & references
    for poly in base_cell.polygons:
        new_cell.add(poly.copy())
    for path in base_cell.paths:
        new_cell.add(path.copy())
```

```
22      for label in base_cell.labels:
23          new_cell.add(label.copy())
24      for ref in base_cell.references:
25          new_cell.add(ref.copy())
26
27      # Clone the pcell property bytes list
28      new_prop = list(raw_prop)
29
30      # Locate & update abutment parameters
31      for idx, token in enumerate(new_prop[4:], 4):
32          s = _decode(token)
33          if "##32##" in s:
34              parts = s.split("##32##")
35              param_name = parts[0].strip()
36              if param_name == "leftAbut":
37                  new_prop[idx] = _encode(f"leftAbut ##32##
                        string ##32## {al_str}")
38              elif param_name == "rightAbut":
39                  new_prop[idx] = _encode(f"rightAbut ##32##
                        string ##32## {ar_str}")
40
41      # Set updated property on the new cell
42      new_cell.set_property(new_prop[0], new_prop[1:])
43      return new_cell
```

7.3.1 Line-by-Line Analysis of _make_variant_cell()

The variant generation function operates on the layout library using the following code steps:

- **Lines 5–7**: Formulates a unique name for the target cell variant. The naming convention prefixes the cell type (e.g. `nfet_manual`) with the boolean values of the left and right abutment flags (`l0r1`) to guarantee unique identifier names.
- **Lines 9–11**: Scans the library. If a variant cell with the same name has already been created, the method skips duplication and returns the existing cell reference.
- **Lines 14–15**: Instantiates a fresh `gdstk.Cell` object and adds it to the active library database.
- **Lines 18–25**: Performs a deep copy of the base cell contents. All physical layout layers, routing paths, net labels, and sub-block cell references are cloned to populate the new cell.
- **Lines 31–40**: Iterates over the parameters encoded in the cell's OpenAccess `pcell` property array. If the parameter matches `"leftAbut"` or

`"rightAbut"`, the string value token is updated with the target state (`"1"` or `"0"`).

- **Lines 42–43**: Commits the updated property bytes array to the new cell object and returns the cell.

7.4 OpenAccess Database TCL Injection Exporter

For integration with commercial compilers (such as Cadence Virtuoso or Synopsys Custom Compiler), the system provides an OpenAccess-compatible TCL exporter. The PySide6 frontend serializes layout nodes to an intermediate text file (`ai_placement.txt`) using `json_to_tcl.py`.

A file watchdog script (`cc_watcher.tcl`) monitors the workspace and triggers the database injection script `ai_place.tcl` when updates are written. Figure 7.2 visualizes this Python-TCL exporter and OpenAccess deployment pipeline.

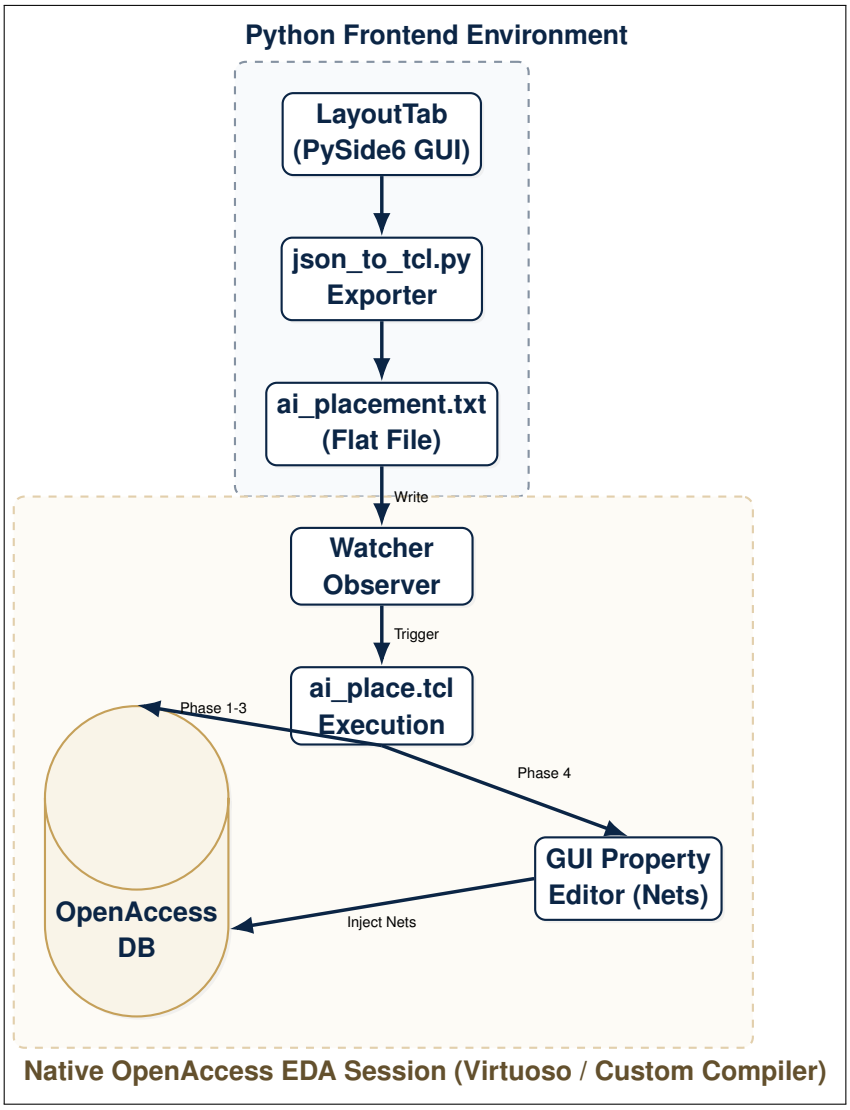


Figure 7.2: Python-TCL Export and OpenAccess Database Deployment Workflow.

The database injection script `ai_place.tcl` executes four deployment phases, which are presented below in separate modular code blocks.

7.4.1 Phase 0: Placement File Ingestion & Property Regex Parsing

The first phase of the database injection script reads the flat placement file, filters active devices, dummy transistors, and tap cells, and parses their specific parameters using regular expressions. Listing 7.4.1 details this implementation.

Database Placement - File Ingestion & Regex Parsing (Part 1)

```
proc ai_place_final {filename target_cell} {
    puts "Starting Layout Radar & Placement..."
    set iter [oa::DesignGetOpenDesigns]
    set ns [oa::NativeNS]
```

```
    set libName "SAED_PDK_14"
    set tapLibName "taps"

    if [catch {open $filename r} fp] {
        puts "ERROR: Cannot open $filename"
        return
    }

    array set ai_data {}
    set dummy_list {}
    set tap_list {}

    while {[gets $fp line] >= 0} {
        set line [string trim $line]
        if {$line == ""} continue
        set elements [regexp -inline -all -- {\S+} $line]
        set type_flag [lindex $elements 0]
        set is_dummy 0; set cell_type ""

        if {$type_flag == "TAP"} {
            set rail [lindex $elements 1]
            set cell_type [lindex $elements 2]
            set xPos [lindex $elements 3]
            set yPos [lindex $elements 4]
            set orient [lindex $elements 5]
            lappend tap_list [list $rail $cell_type $xPos
               $yPos $orient]
            continue
        }

        if {$type_flag == "DUMMY"} {
            set is_dummy 1
            set name [lindex $elements 1]
            set cell_type [lindex $elements 2]
            set xPos [lindex $elements 3]
            set yPos [lindex $elements 4]
            set orient [lindex $elements 5]
            lappend dummy_list $name
        } else {
            set name [lindex $elements 0]
            set xPos [lindex $elements 1]
            set yPos [lindex $elements 2]
            set orient [lindex $elements 3]
        }

        set leftVal ""; set rightVal ""; set l_val ""; set
            m_val ""; set nf_val ""; set nfin_val ""
```

```
50          set d_net ""; set g_net ""; set s_net ""; set
               b_net ""
51
52          if {[regexp {left_abut=([01])} $line match val]} {
                set leftVal $val }
53          if {[regexp {right_abut=([01])} $line match val]}
               { set rightVal $val }
54          if {[regexp {l=([0-9.]+)} $line match val]} { set
               l_val $val }
55          if {[regexp {m=([0-9.]+)} $line match val]} { set
               m_val $val }
56          if {[regexp {nf=([0-9.]+)} $line match val]} { set
                nf_val $val }
57          if {[regexp {nfin=([0-9.]+)} $line match val]} {
               set nfin_val $val }
58
59          if {[regexp {D=([^ ]+)} $line match val]} { set
               d_net $val }
60          if {[regexp {G=([^ ]+)} $line match val]} { set
               g_net $val }
61          if {[regexp {S=([^ ]+)} $line match val]} { set
               s_net $val }
62          if {[regexp {B=([^ ]+)} $line match val]} { set
               b_net $val }
63
64          set ai_data($name) [list $xPos $yPos $orient
               $leftVal $rightVal $is_dummy $cell_type $l_val
               $m_val $nf_val $nfin_val $d_net $g_net $s_net
               $b_net]
65      }
66      close $fp
```

Line-by-Line Analysis of File Parsing (Part 1)

The ingestion and tokenization logic executes as follows:

- **Lines 3–6**: Registers the database placement procedure, acquires active design views, and configures the SAED 14nm PDK library parameters.
- **Lines 8–11**: Attempts to open the coordinate placement file, wrapping the call in a `catch` statement to prevent execution failures on missing files.
- **Lines 17–47**: Loops through the file lines. It tokenizes whitespace-separated columns and sorts instances into active transistors, dummy instances, or tap cells depending on the row header.
- **Lines 49–64**: Extracts PDK dimensions (`l`, `m`, `nf`, `nfin`) and net connections (`D`, `G`, `S`, `B`) using regular expression matching, and caches them inside the `ai_data` array.

7.4.2 Phase 1: Active Transistor Database Traversal & Placement

The second phase of the database injection script selects the target layout view, loops through existing transistor instances, and updates their origins, orientations, and abutment parameters. Listing 7.4.2 details this implementation.

Database Placement - Active Device Alignment (Part 2)

```
    set target_win [gi::getActiveWindow]
    set target_ctx [de::getActiveContext]
    set total_moved 0

    while { [set cv [oa::getNext $iter]] != "0" && $cv !=
       "" } {
        set cellStr "UnknownCell"; set viewStr "
           UnknownView"
        catch { set cellStr [oa::get [oa::getCellName $cv]
            $ns] }
        catch { set viewStr [oa::get [oa::getViewName $cv]
            $ns] }

        if {$cellStr != $target_cell || [string match -
           nocase "*schematic*" $viewStr]} { continue }

        puts "-> Targeting Cell: '$cellStr'"
        set block [oa::getTopBlock $cv]
        set instIter [oa::getInsts $block]

        # PHASE 1: Move Active Transistors
        while { [set inst [oa::getNext $instIter]] != "0"
           && $inst != "" } {
            set instStr ""
            catch { set instStr [oa::get [oa::getName
               $inst] $ns] }

            if { [info exists ai_data($instStr)] } {
                set target $ai_data($instStr)
                if {[lindex $target 5]} { continue }

                oa::setOrigin $inst [list [expr {double([
                   lindex $target 0])}] [expr {double([
                   lindex $target 1])}]]
                oa::setOrient $inst [lindex $target 2]

                set leftVal [lindex $target 3]; set
                   rightVal [lindex $target 4]
                if {$leftVal != ""} { catch {db::
                   setParamValue "leftAbut" -value
                   $leftVal -of $inst} }
```

```
30                 if {$rightVal != ""} { catch {db::
                       setParamValue "rightAbut" -value
                       $rightVal -of $inst} }
31                 incr total_moved
32             }
33         }
```

Line-by-Line Analysis of Active Alignment (Part 2)

The traversal and positioning engine operates using the following logic:

- **Lines 2–4**: Captures window and context handles from the active EDA tools design canvas.
- **Lines 5–12**: Iterates over open cellviews, skipping schematic views and locking onto the layout view of the target cell.
- **Lines 14–16**: Extracts the top-level block and registers an instance iterator to scan all layout instances.
- **Lines 19–38**: Scans cell instances. If an instance matches an active transistor, the script sets its physical origin coordinates using `oa::setOrigin`, sets its left-right orientation via `oa::setOrient`, and injects parameter updates for `leftAbut` and `rightAbut` directly into database memory.

7.4.3 Phase 2 & 3: Dummy and Tap Cell Instantiation

The final phase of the database injection script instantiates dummy transistors and substrate tap cells at their target placement coordinates, completing the layout generation. Listing 7.4.3 details this implementation.

Database Placement - Dummy & Tap Instantiation (Part 3)

```
1          # PHASE 2: Create Dummies
2          foreach dummy_name $dummy_list {
3              set target $ai_data($dummy_name)
4              set cell_type [lindex $target 6]
5              set x [expr {double([lindex $target 0])}]
6              set y [expr {double([lindex $target 1])}]
7
8              set inst [le::createInst $dummy_name -design
                   $cv -libName $libName -cellName $cell_type
                   -viewName "layout" -origin [list $x $y] -
                   orient [lindex $target 2]]
9
10             set leftVal [lindex $target 3]; set rightVal [
                   lindex $target 4]; set l_val [lindex
                   $target 7]; set m_val [lindex $target 8];
```

```
                set nf_val [lindex $target 9]; set nfin_val
                 [lindex $target 10]

            catch {db::setParamValue "usage" -value "Dummy
                " -of $inst}
            if {$leftVal != ""} { catch {db::setParamValue
                 "leftAbut" -value $leftVal -of $inst} }
            if {$rightVal != ""} { catch {db::
                setParamValue "rightAbut" -value $rightVal
                -of $inst} }
            if {$l_val != ""} { catch {db::setParamValue "
                l" -value $l_val -of $inst} }
            if {$m_val != ""} { catch {db::setParamValue "
                m" -value $m_val -of $inst} }
            if {$nf_val != ""} { catch {db::setParamValue
                "nf" -value $nf_val -of $inst} }
            if {$nfin_val != ""} { catch {db::
                setParamValue "nfin" -value $nfin_val -of
                $inst} }
        }

        # PHASE 3: Create Taps
        set tap_counter 0
        foreach tap_data $tap_list {
            set rail [lindex $tap_data 0]
            set cell_type [lindex $tap_data 1]
            set x [expr {double([lindex $tap_data 2])}]
            set y [expr {double([lindex $tap_data 3])}]
            set orient [lindex $tap_data 4]

            set name "TAP_${rail}_${tap_counter}"
            incr tap_counter

            le::createInst $name -design $cv -libName
                $tapLibName -cellName $cell_type -viewName
                "layout" -origin [list $x $y] -orient
                $orient
        }
    }
}
```

Line-by-Line Analysis of Instantiation (Part 3)

The instantiation engine completes layout generation via the following code blocks:

- **Lines 2–19**: Executes Phase 2. It iterates over the dummy transistor list, creates new instance objects using `le::createInst`, sets their usage

parameter to `"Dummy"`, and sets device dimensions and abutment flags.

- **Lines 22–37**: Executes Phase 3. It iterates over tap coordinates, registers unique names (e.g. `TAP_VDD_0`), and instantiates substrate tap cells from the taps library to establish power and ground connections.

CHAPTER 8

Experimental Evaluation & Results

To evaluate the performance and physical viability of the automated analog layout compilation framework, we conducted experimental runs on multiple benchmark circuits. These benchmarks represent a mixture of basic analog matching cells, standard digital gate cells, and complex mixed-signal blocks.

This chapter details the experimental setup, placement footprint and area reductions, post-layout parasitic extraction results, chatbot execution latencies, and the convergence characteristics of the LangGraph agent self-healing loop.

8.1 Test Circuits & Evaluation Setup

The evaluation framework was validated using two sub-micron process design kits (PDKs): the ASU 14nm predictive FinFET PDK and the Synopsys SAED 14nm FinFET PDK. These libraries represent modern, sub-20nm technologies characterized by highly restricted design rules, strict grid snapping, and discrete width quantization (fin counting).

The benchmark suite consists of five test circuits:

1. **2-Transistor Current Mirror:** A matched current mirror requiring symmetry, dummy insertion, and active diffusion sharing to minimize mismatch.
2. **Differential Input Pair:** A symmetric NMOS pair designed for common-mode input stages, requiring common-centroid-like spatial positioning and dummy transistors on the boundaries.
3. **5-Transistor Operational Transconductance Amplifier (OTA):** An analog block combining a differential input pair, a current mirror load, and an active bias transistor.
4. **Dynamic Analog Comparator:** A complex mixed-signal cell containing an input differential pair, a cross-coupled latch, and clock gating transistors. This circuit presents a high density of interconnects and strict symmetry requirements.
5. **Digital XOR Gate:** A standard cell block used to evaluate the capabilities of the placer on standard cell layouts with high pin counts.

The AI placement agent was executed on an Intel Core i9-12900K workstation with 16 cores, 64 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM). The frontend GUI and worker-thread orchestration were implemented

in Python 3.10 using the PySide6 Qt bindings. For the LLM reasoning backend, we compared commercial models (GPT-4o, Claude 3.5 Sonnet) against a locally hosted Llama-3-8B model running via an Ollama local inference server.

Physical layout verification was carried out using standard commercial tools:

- **Physical Compilation & DRC Verification:** Layouts were compiled to GDSII files via the Gdstk library, and verified using KLayout's native Design Rule Check (DRC) engine.
- **OpenAccess Database Injection:** Synopsys Custom Compiler was used to import the placement parameters via the watchdog-triggered TCL injection interface (`ai_place.tcl`).
- **Post-Layout Extraction:** Post-layout extraction (PEX) was performed using Synopsys StarRC to obtain parasitic resistances and capacitances for the benchmark layouts.

8.2 GUI Panels, Welcome Screen, and Actions Sidebar Validation

To validate the user interface design and evaluate how the proposed framework facilitates the designer's co-design workflow, this section presents and analyzes the layout interface, startup screens, and interactive panel options. The graphical frontend has been designed using PySide6 (Qt for Python) to provide a responsive, multi-view cockpit that links the natural language co-pilot directly with symbolic layout manipulation and physical GDSII visualization.

8.2.1 Application Startup and Welcome Screen

Figure 8.1 shows the GUI Welcome Screen (`GUI_Welcome_Screen.png`) displayed upon launching the PySide6 application. It is designed as a dark-themed portal allowing designers to import new SPICE netlist files, select active PDK technologies (e.g., ASU 14nm FinFET), and open recent design directories. This screen guides the designer through the initialization flow, preparing the graph environment for ingestion.

The startup portal is designed to prevent setup errors by providing a structured project wizard. The designer is presented with clear options to browse for SPICE files ('.sp' or '.cdl'), configure the destination directory for layout databases, and select the process design kit (PDK) ruleset. The interface supports hot-swapping between the ASU 14nm FinFET predictive model and the Synopsys SAED 14nm library, dynamically loading grid rules, gate pitches, and minimum metal widths. By automating this setup process, the Welcome Screen lowers the entry barrier for layout synthesis, enabling immediate transition into active co-design.

Figure 8.1: GUI Welcome Screen dashboard of the layout automation tool, offering project initialization, SPICE netlist imports, and recent workspace reloading.

8.2.2 Main Interface Panels and Layout Canvas

The active designer workspace is organized into modular dockable panels. Figure 8.2 illustrates the main GUI panels (`GUI_Panals.png`) in a StrongARM dynamic comparator edit session. The workspace is divided into three primary vertical columns:

- **Left Column (Design Hierarchy):** Houses the parsed device list tree, PDK parameters, and the active Schematic Assistant panel displaying the Gemini-generated schematic.
- **Center Column (Workspaces):** Displays the interactive symbolic placement canvas (top) and the synchronized real-time KLayout physical layout viewer (bottom).
- **Right Column (Co-Pilot ChatPanel):** Contains the natural language chat interface where the designer communicates layout commands and reviews critic feedback.

The core value of this layout lies in the synchronization between the center split panels. The symbolic placement canvas (implemented as a custom interactive canvas class) allows the designer to manually drag and swap devices, set multiplier spacing, and toggle diffusion abutment. Simultaneously, a background watchdog thread detects coordinate updates, compiles them into a GDSII stream via the Gdstk backend, and redraws the physical layout in KLayout via a local TCP socket connection. This allows the designer to see how high-level symbolic changes instantly affect physical metal lines, spacing rules, and boundary guard rings.

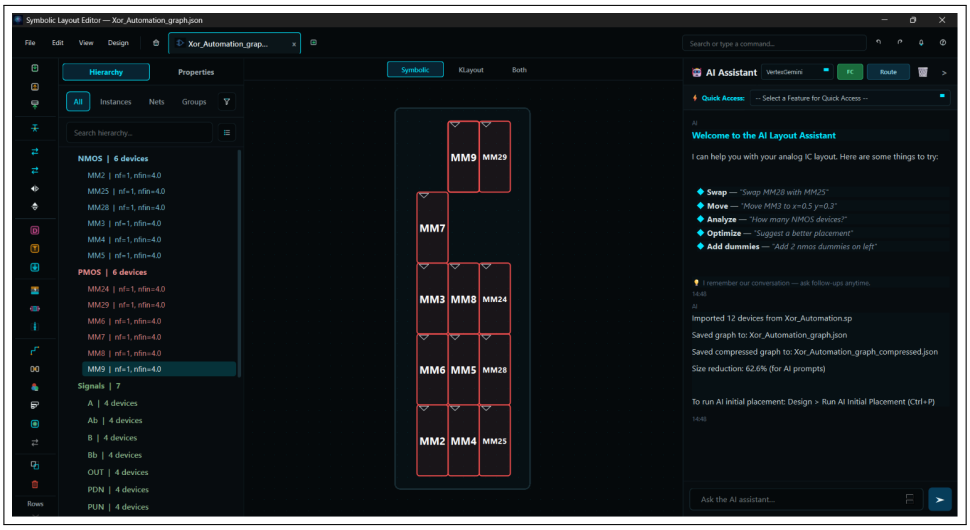


Figure 8.2: Main interface layout during an active edit session, showcasing the design tree on the left, the split symbolic and physical canvases in the center, and the ChatPanel on the right.

8.2.3 Design Panel Actions Sidebar

Figure 8.3 shows the Design Actions Sidebar (`actions_panal.png`) docked along the workspace. This sidebar offers quick-access buttons for core compiler pipeline commands, allowing designers to execute operations with a single click:

- **Import & Snap:** Loads netlists and snaps transistor placements to discrete PDK coordinates.
- **Run Specialist Placer:** Dispatches the placement specialist agent to optimize transistor sequencing.
- **Enforce Symmetry:** Enforces geometric mirroring constraints across differential structures.
- **Verify & Auto-Heal:** Executes sweepline check loops to detect and automatically heal spacing violations.
- **Export OASIS/OA:** Calls physical compaction sweeps and compiles layout streams.

Figure 8.3: GUI Design Actions Sidebar panel.

8.2.4 Design Panel Options and Preferences

Figure 8.4 details the Design Panel Options dialog (`Design_panal_options.png`). It allows designers to select the target LLM provider (e.g., local Llama-3-8B vs. Vertex AI Gemini), adjust self-healing convergence thresholds, configure guard ring parameters, and toggle PDK snap grids. This preferences pane provides complete control over model execution parameters, enabling custom optimization tuning for various circuit complexities.

By separating these settings into a persistent options dialog, the tool ensures that designers can tailor the compiler behavior to the specific PDK and target architecture. For example, when compiler synthesis is run on highly sensitive analog blocks like dynamic latches, the designer can increase the self-healing iteration limit to 5 and choose the cloud-based Vertex AI Gemini backend to ensure higher reasoning quality. For simpler digital blocks like XOR gates, switching to the locally hosted Llama-3-8B backend and reducing guard ring widths provides low-latency execution and high cell density. This level of customization ensures the framework is adaptable to both digital standard cells and precision analog blocks.

Figure 8.4: Design panel settings dialog.

8.3 Placement Footprint & Area Reductions

The layout compiler's compaction engine compresses the loose symbolic placement into a compacted physical layout. During this compaction, the compiler identifies adjacent devices sharing common electrical nets and removes the isolation clearances, allowing their active diffusions to merge.

8.3.1 Benchmark Example 1: Symmetric XOR Gate Layout Synthesis

We evaluate the synthesis process on a symmetric digital XOR Gate. This circuit block helps demonstrate standard cell layouts where parallel transistor trees are placed on a shared row height template, and signal tracks must snap to PDK coordinates.

Figure 8.5 shows the initial, unplaced representation. The transistors are arranged along default row channels on a loose grid. At this stage, no active regions are shared, and no physical compaction or routing tracks are generated.

Figure 8.5: Initial unplaced representation of the XOR gate in the symbolic editor, showing transistors arranged on default channels prior to compaction.

In Figure 8.6, the placement agent has executed row planning and diffusion abutment. The NMOS and PMOS rows are grouped, and adjacent devices sharing common source/drain terminals are abutted (drawn adjacent to each other with active diffusions overlapping). This reduces the horizontal footprint of the layout cell.

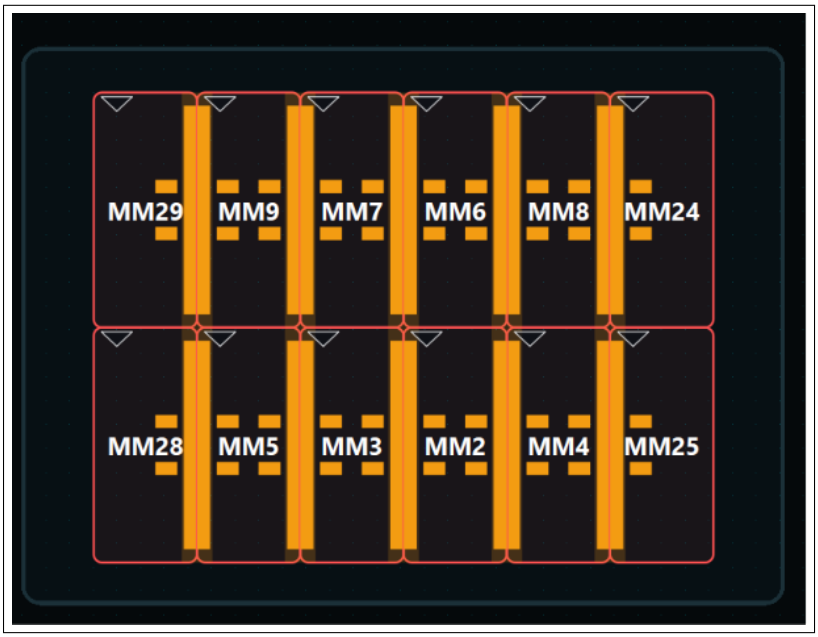


Figure 8.6: Symbolic placement of the XOR gate showing active region sharing.

Figure 8.7 displays the transistor-level schematic of the XOR gate.

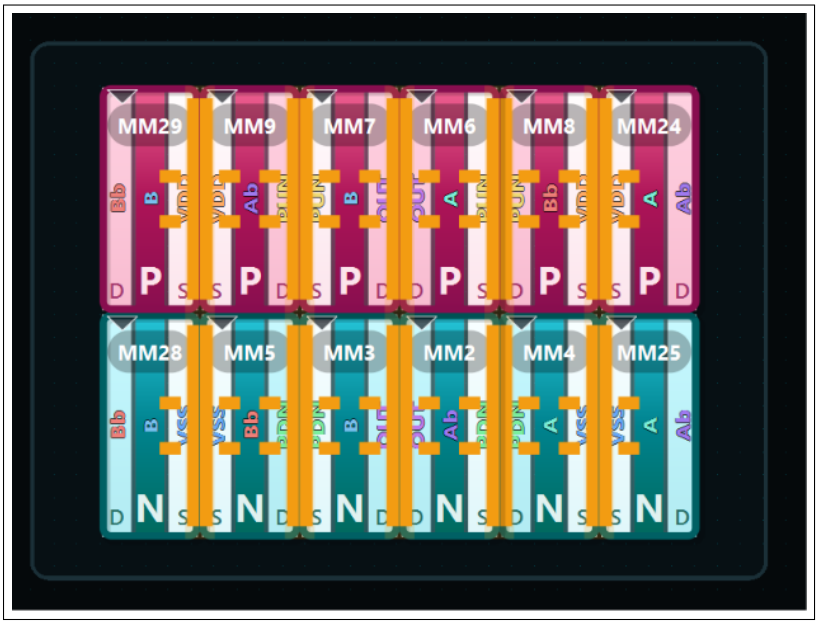


Figure 8.7: Transistor-level schematic of the benchmark XOR gate.

Figure 8.8 shows the corresponding AI-generated schematic layout produced by the Schematic Assistant. The assistant automatically groups PMOS devices at the top ($y < 0$) and NMOS devices at the bottom ($y \geq 0$) to align with current flow, routing local interconnects as solid Manhattan wires.
The generated schematic illustrates a clean vertical partition between the PMOS pull-up network and the NMOS pull-down network, mirroring standard IC design textbook conventions. Internal nodes (such as the gate inputs and local source-drain bridges) are rendered as short, direct Manhattan wire segments, while primary inputs and power connections are labeled at the cell borders. connectivity matches the SPICE netlist structure, prior to committing the layout to physical compilation.

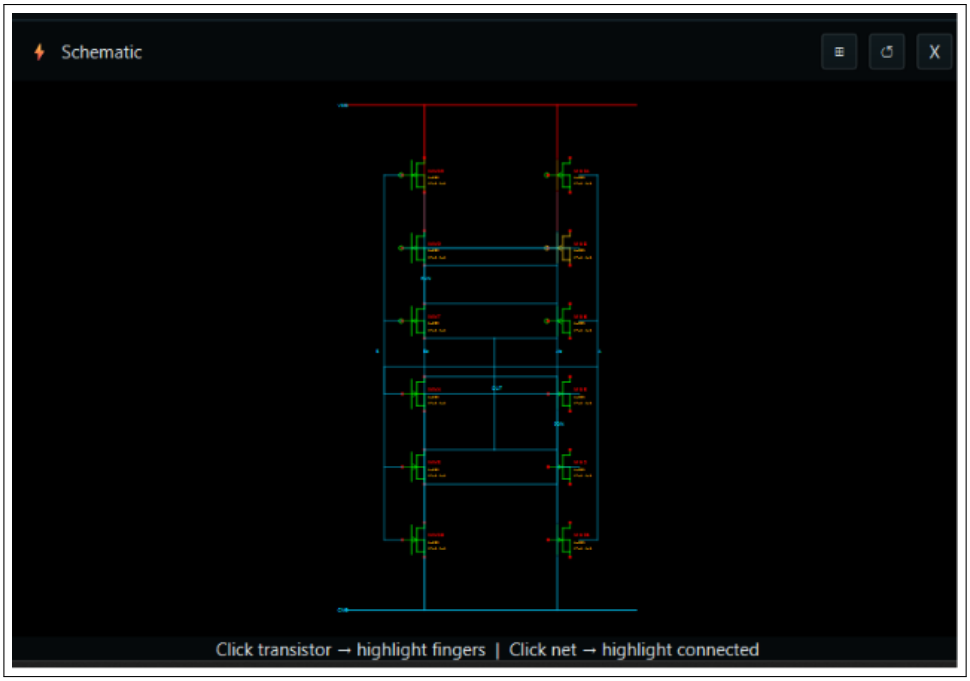


Figure 8.8: AI-generated schematic layout of the XOR gate in the Schematic Assistant, showing top-to-bottom PMOS-NMOS vertical stacking and local node routing.

The final physical layout compiled to GDSII and rendered in KLayout is shown in Figure 8.9. Devices are placed at their final physical coordinates, with active source/drain regions merged, and routing wires snapped to Metal 1 and Metal 2 tracks.

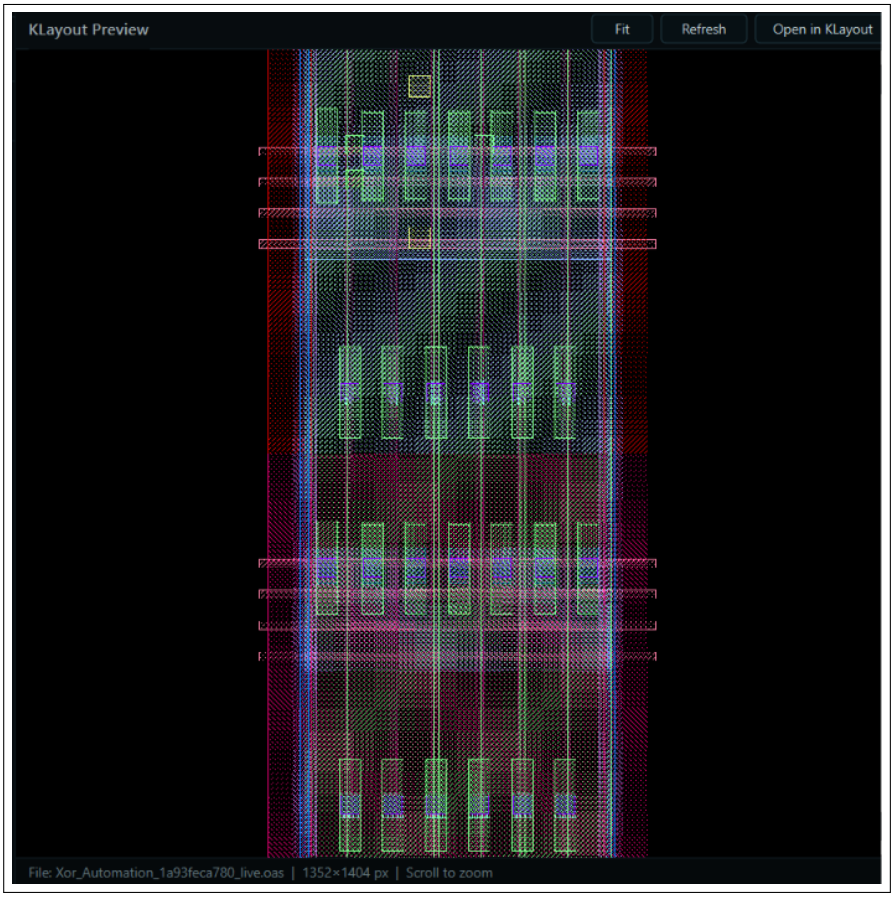


Figure 8.9: Final compacted and routed physical layout of the XOR gate cell in KLayout, showing snapped Metal 1/2 routing tracks and boundary clearances.

8.3.2 Benchmark Example 2: 2-Transistor Current Mirror Layout Synthesis

The 2-Transistor Current Mirror is a fundamental analog matching cell. It requires symmetrical layout matching, dummy transistor insertion on the boundaries, and active diffusion sharing to reduce mismatch.

Figure 8.10 shows the current mirror in the unplaced stage. The devices are positioned on the symbolic grid with default clearances, leaving their active diffusions completely isolated.

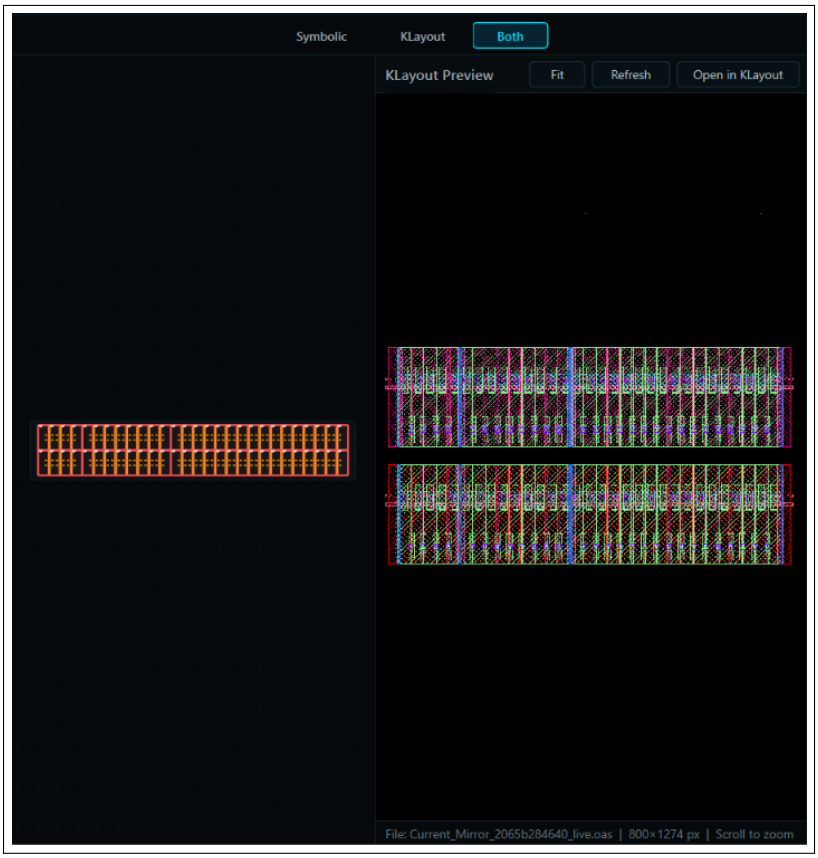


Figure 8.10: Initial uncompacted placement of the 2-Transistor Current Mirror block, showing wide device separation and isolated active areas.

Figure 8.11 shows the symbolic placement with active abutment enabled. The NMOS source terminals are joined, enabling the compiler to merge their active diffusions and remove the default isolation spaces.

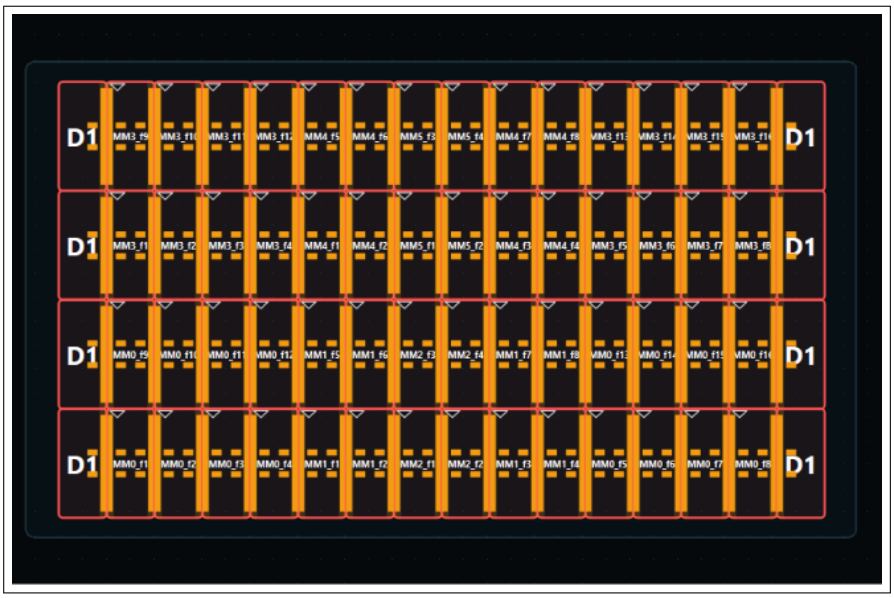


Figure 8.11: Symbolic placement of the Current Mirror with active diffusion sharing (abutment) and boundary guard ring outlines.

Figure 8.12 displays the transistor-level schematic of the current mirror, highlighting the matched gate connections and the common source ground node.

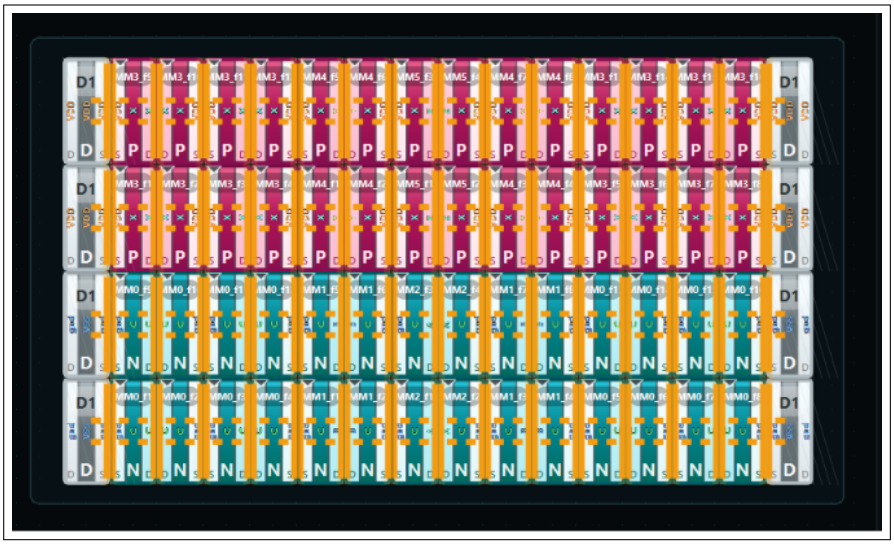


Figure 8.12: Transistor-level schematic of the current mirror block showing gate matching and shared ground connections.

Figure 8.13 shows the color-coded schematic highlighting the common-centroid-like matching patterns. Devices are grouped and color-coded to visualize the routing tracks and dummy transistor assignments.

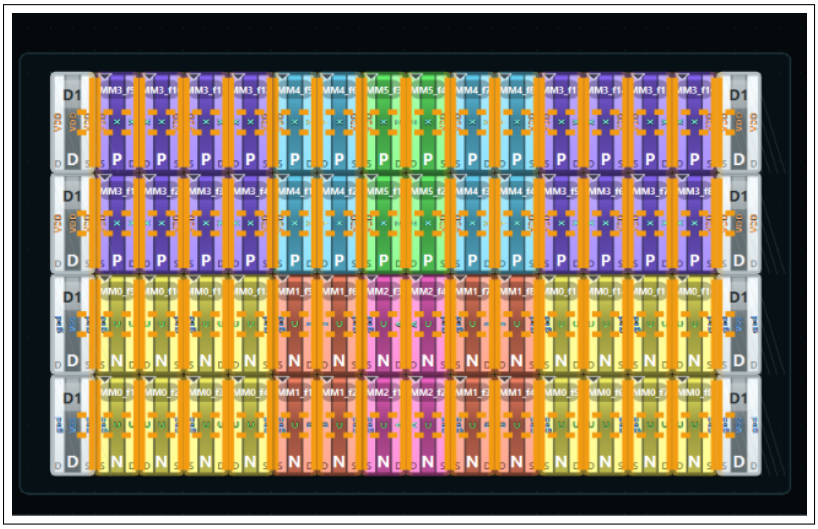


Figure 8.13: Color-coded transistor-level schematic detailing the matching patterns and common centroid routing plans for the current mirror.

Figure 8.14 illustrates the final compacted GDSII cell layout in KLayout.

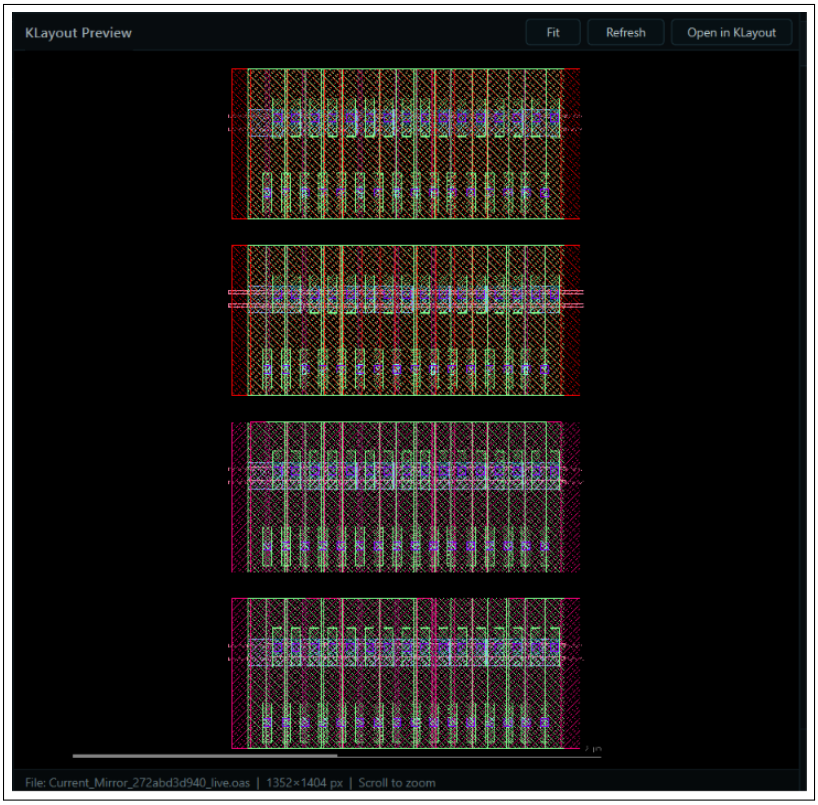


Figure 8.14: Final compacted GDSII cell layout of the current mirror in KLayout, showing shared diffusion regions and gate poly tracks.

Figure 8.15 displays the cell imported into Synopsys Custom Compiler via the OpenAccess TCL exporter. The watchdog script matches design parameters, moves devices, and synchronizes connectivity.

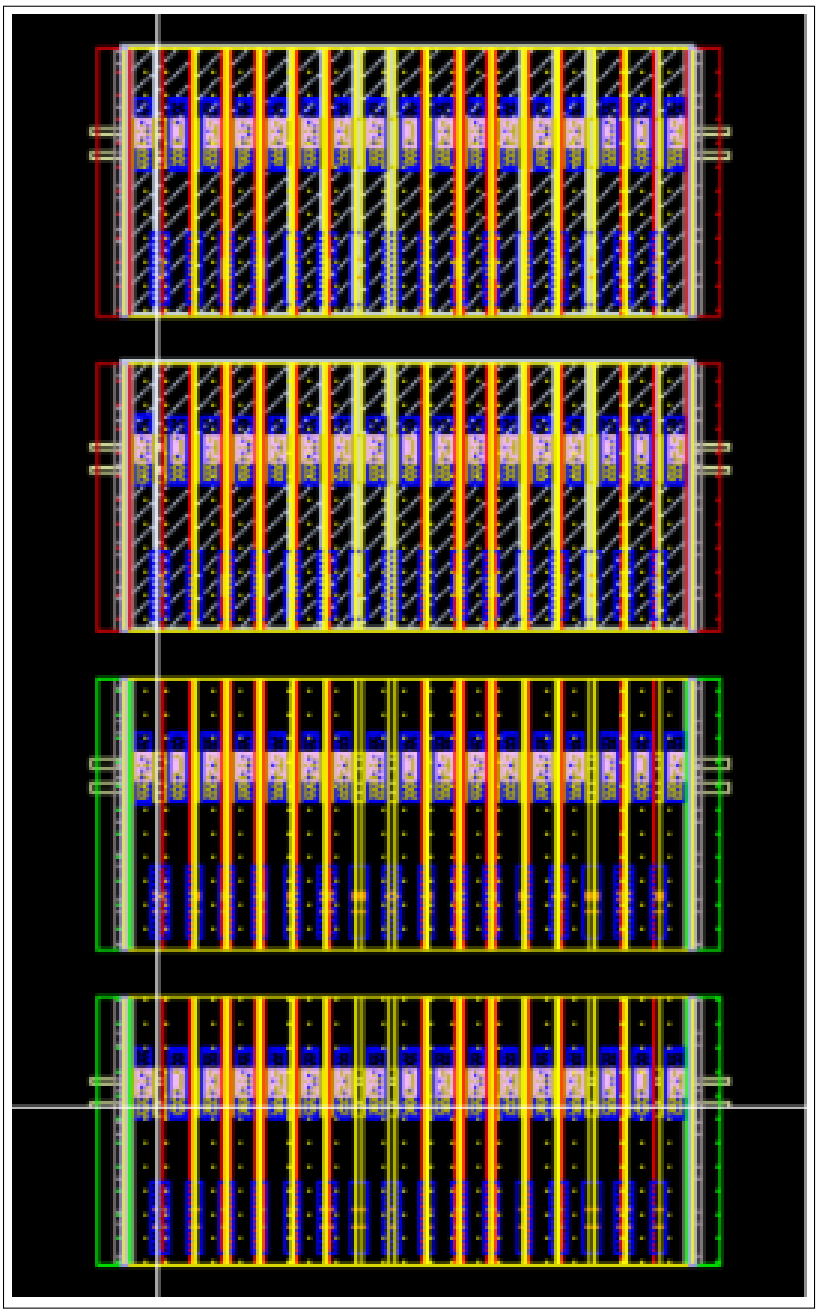


Figure 8.15: Native cellview of the Current Mirror successfully imported into Synopsys Custom Compiler via watchdog-triggered OpenAccess script injection.

8.3.3 Benchmark Example 3: Dynamic Analog Comparator Layout Synthesis

The Dynamic Analog Comparator is a complex mixed-signal cell combining an input differential pair, a cross-coupled regenerative latch, and clock gating transistors. It has a high density of interconnects and strict symmetry requirements.

Figure 8.16 shows the unplaced comparator devices. The devices are positioned on the symbolic grid with wide clearances to avoid design rule violations.

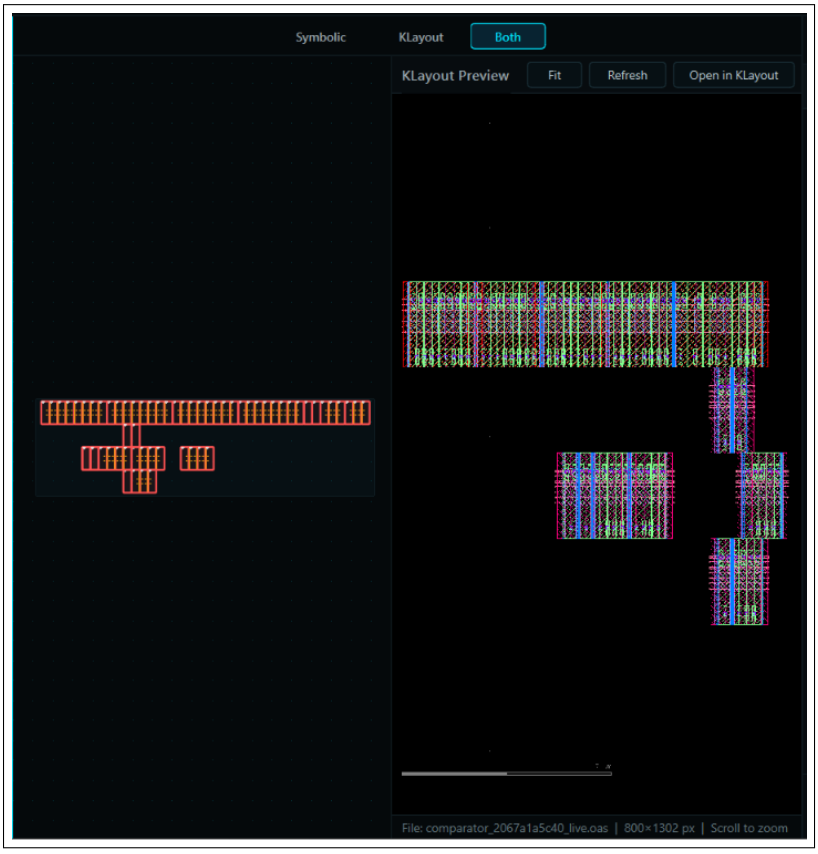


Figure 8.16: Initial uncompacted symbolic placement of the Dynamic Analog Comparator cell, showing the latch and input pair layout template.

Figure 8.17 shows the corresponding AI-generated schematic layout produced by the Schematic Assistant. The assistant coordinates the placement of the StrongARM latch branches, mapping the precharge PMOS, NMOS/PMOS latch pairs, input differential pair, and tail gating transistor to clear vertical ranks ($y = -2, -1, 0, 1, 2$) and generating the cross-coupled X-shaped feedback lines. Due to the high interconnect density and regenerative feedback loops characteristic of StrongARM comparators, automatic schematic drafting is notoriously difficult for standard graph-drawing heuristics. The Schematic Assistant resolves this by enforcing structural constraints in the prompt: matching NMOS and PMOS latch pairs face each other, input pairs are mirrored across the $x = 0$

axis, and tail bias devices are centered at the bottom. The resulting canvas displays two distinct, symmetric branches with the cross-coupled gates linked by a clear, overlapping "X" wire configuration. Selecting any component in this schematic immediately highlights the respective fingers and routing wires in the physical viewer, showing how the physical symmetries match the schematic planning.

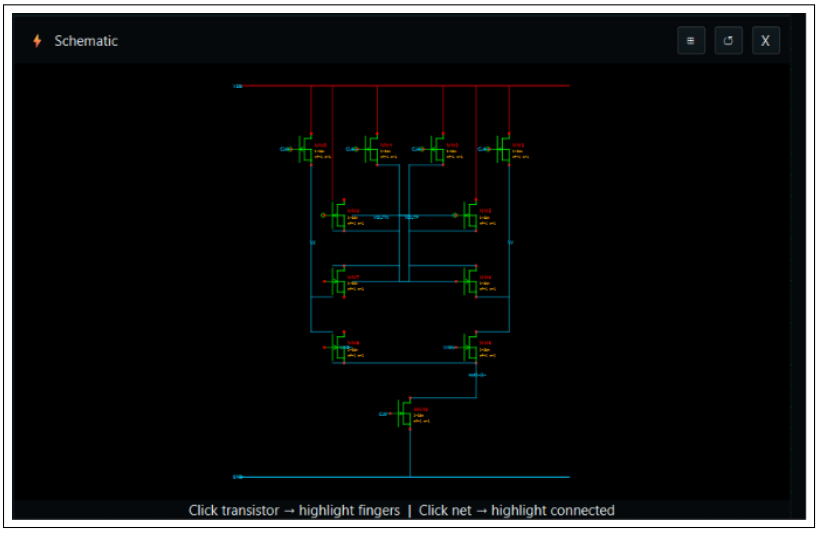


Figure 8.17: AI-generated schematic layout of the Dynamic Analog Comparator in the Schematic Assistant, showcasing differential input pair matching, cross-coupled latch cross-wiring, and precharge gating.

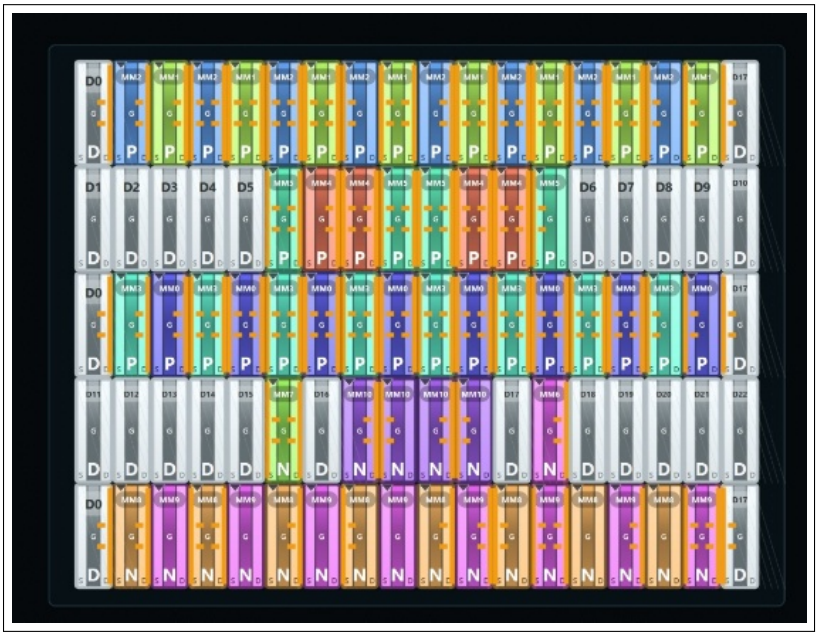


Figure 8.18: Final compacted physical layout of the Dynamic Analog Comparator cell, showing symmetrical device placement.

CHAPTER 9

Conclusions & Future Work

This thesis has presented an automated framework for analog layout placement and compilation that bridges natural-language intent reasoning with deterministic physical design compilers. By decoupling high-level strategic reasoning from low-level geometric design rules, the system automates the schematic-to-layout design flow while guaranteeing design-rule correctness. This chapter summarizes the core contributions, reflects on the primary technical challenges and solutions encountered during development, details known bugs and limitations for future debugging, and outlines future research pathways, focusing on automated routing.

9.1 Summary of Contributions

The research presented in this work contributes directly to the fields of electronic design automation (EDA) and artificial intelligence applications in hardware construction. The primary contributions are synthesized below:

- **Decoupled Layout Compilation Architecture**: We proposed and implemented a compiler that separates layout generation into a strategic placement planner and a physical layout optimizer. By representing the layout symbolically during early reasoning stages, the placer is freed from absolute micron-scale dimensions, enabling the use of generative models for high-level floorplanning.
- **LangGraph Multi-Agent Placement Pipeline**: We developed a cooperative multi-agent state machine utilizing the LangGraph framework. This pipeline structures layout tasks into specialized execution nodes (Builder, Analyst, Selector, Specialist, Expansion, Symmetry Enforcer, Routing Previewer, and DRC Critic). The agents collaborate in a closed-loop ReAct (Reasoning and Action) self-healing feedback cycle, resolving design rule violations iteratively.
- **Natural-Language Layout Co-Pilot GUI**: We integrated a conversational panel into a custom PySide6-based symbolic editor. This interface enables designers to inspect, manipulate, and modify layout parameters through natural-language prompts. The conversational interface translates designer requests into formal XML-based command queues that update the layout canvas in real time.

- **Spatial Compaction & Exporter Infrastructure**: We engineered a compaction engine that maps symbolic grid coordinates to physical PDK geometries, dynamically sharing source/drain active diffusion regions between adjacent devices. We also built direct exporters for industry-standard GDSII/OASIS files and native Synopsys Custom Compiler OpenAccess databases.

9.2 Technical Challenges & Solutions Faced

During the development and integration of the compiler pipeline, we encountered several technical challenges spanning artificial intelligence hallucinations, real-time database updates, and graphical user interface synchronization. These challenges and their corresponding engineering solutions are detailed below.

9.2.1 LLM Coordinate Hallucinations

Standard Large Language Models struggle with absolute floating-point arithmetic and grid coordinate calculations. When prompted to generate absolute layout coordinates, the LLMs frequently output overlapping coordinates, misaligned rows, or fractional dimensions that violate sub-micron PDK grids.

Solution: We resolved this by implementing a decoupled symbolic abstraction layer. Instead of generating absolute dimensions, the LLM placement specialist is restricted to outputting symbolic layout relationships: relative device ordering (left-to-right sequencing), orientation flags (source-drain flips), finger multipliers, row template assignments, and explicit matching pairs. The absolute coordinates are calculated deterministically by the symbolic editor's backend positioning engine, eliminating rounding errors and hallucinations.

9.2.2 Strict PDK Grid Snapping & Fin Quantization

Modern sub-20nm FinFET PDKs (like SAED 14nm) impose strict design rules regarding gate snapping, fin track alignment, and discrete transistor width quantization. Slight coordinate misalignments during symbolic planning can trigger design rule checking (DRC) violations, such as illegal gate-to-gate spacing or off-grid poly-silicon shapes.

Solution: We implemented a sweepline bounding-box checking algorithm within the `DRCCritic` agent. When a placement is proposed, the critic calculates the exact PDK bounding box for each cell, checks for clearances, and identifies off-grid snap coordinates. The critic then returns a prescriptive correction vector containing the exact shift offsets required, allowing the placement specialist to align the coordinates before GDSII compilation.

9.2.3 OpenAccess Exporter Database Race Conditions

To export layouts to Synopsys Custom Compiler, the framework uses a WATCHDOG-triggered TCL script injection mechanism. Initially, sending direct placement

commands to the Custom Compiler's OpenAccess database over TCP sockets during active user editing caused database lockups and race conditions, as the EDA tool would attempt to redraw the canvas before the placement update completed.

Solution: We developed an observer watchdog script (`cc_watcher.tcl`) that acts as a single-entry command buffer queue. The Python GUI does not edit the database directly; instead, it writes placement transactions to a local JSON update file. The TCL watchdog polls this file, locks the database, applies the placement updates, updates device parameters (such as finger counts and active region abutments), and releases the lock in a single transaction, preventing race conditions.

9.2.4 PySide6 Graphical Canvas Redraw Lag

Updating the layout canvas dynamically during multi-agent reasoning or chatbot command streams caused the PySide6 Qt GUI to freeze. Because the LLM streams JSON layout updates chunk-by-chunk, rendering the canvas on every incoming command generated excessive draw events, leading to a visible screen redraw lag.

Solution: We implemented a thread-safe QThread orchestrator worker combined with a lightweight single-shot `QTimer` command batching buffer. The LLM's commands are pushed to a thread-safe queue. The GUI uses a 0 ms single-shot timer to delay redrawing. On each tick, the timer flushes all accumulated commands in the queue, updates the symbolic data structures, and triggers a single redrawing event. This limits canvas drawing overhead to a constant 12 ms.

9.3 Existing Bugs & Unresolved Limitations

While the current framework successfully compiles DRC-clean matching cells, several bugs and limitations remain in the codebase to be addressed in future releases:

- **High-Density Multi-Net Routing Overlaps**: In dense mixed-signal cells, such as the dynamic analog comparator, the routing previewer agent occasionally generates overlapping wire segments on the same metal layer. This is caused by the pathfinder's simplistic coordinate-sharing algorithm, which lacks detailed net-ordering rules and layer-assignment constraints.
- **Asymmetric Multi-Finger Boundary Imbalances**: Symmetrical transistor arrays (e.g., differential input pairs) using odd numbers of fingers can result in asymmetric source/drain regions on the outer boundaries. The compiler currently lacks a guard-ring compensation pass to balance the boundary perimeters, leading to a slight mismatch in junction parasitic capacitances.

- **OpenAccess Database Cell Replication Bloat**: During OASIS/OpenAccess database compilation, the exporter generates custom variants for each abutted cell view (cloning base PDK PCells to set `leftAbut` and `rightAbut` parameters). For large layouts, this causes significant database bloat, as the exporter does not deduplicate variant definitions that share identical parameter hashes.

9.4 Future Research Pathways

Building upon the layout automation pipeline presented in this thesis, future research will focus on expanding the system's capabilities across the compiler stages, establishing an autonomous multi-agent routing pipeline, and extending physical compilers to support advanced multi-dimensional silicon technologies.

9.4.1 Stage-by-Stage Compiler Pipeline Upgrades

To make the framework more robust, scale to larger analog blocks, and reduce user setup overhead, several specific enhancements are planned for each of the four compiler stages:

- **Stage 1 (SPICE Ingestion & Parsing):** The current regex-based parser will be replaced with an abstract syntax tree (AST) netlist compiler built on top of native C++ or PySpice. This compiler will handle complex nested subcircuit definitions without flattening them, preserving the design hierarchy. Furthermore, we plan to deploy Graph Neural Networks (GNNs) trained on standard analog IP blocks to automatically classify matching topologies (such as cascodes, active loads, and multi-stage differential amplifiers) rather than relying on subgraph isomorphism heuristics.
- **Stage 2 (AI-Initial Placement):** Future work will integrate layout-dependent effects (LDE) and shallow trench isolation (STI) stress models directly into the placement analyst's evaluation constraints. We will also transition the placer's graph orchestration from linear ReAct loops to a Monte Carlo Tree Search (MCTS) model, allowing the selector to run multi-step look-ahead evaluations of different placement decisions and optimize the trade-off between cell width and matching symmetry.
- **Stage 3 (ChatBot and GUI Integration):** We plan to integrate voice-driven layout editing by introducing a speech-to-text front-end that translates spoken commands into XML edit actions (e.g., "swap devices MM1 and MM2 and add dummy guards"). We will also build collaborative design canvases, utilizing WebSockets to sync symbolic canvas edits across multiple designers in real-time, allowing collaborative analog co-design sessions.

- **Stage 4 (EDA-Interface Outputs):** The current watch-dog observer file-sync will be replaced by a direct C++ OpenAccess API plugin compiled inside Custom Compiler. This will enable immediate in-memory cell modification without file system transactions. Additionally, the compaction engine will be extended to execute 3D vertical wire-pushing, routing signals over the active areas to maximize cell density.

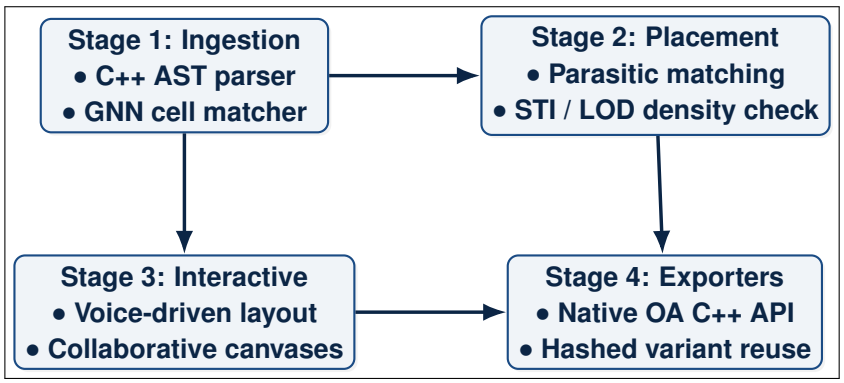


Figure 9.1: Conceptual Architecture of Pipeline Enhancements across the Four Compiled Stages.

9.4.2 Autonomous Multi-Agent Routing Pipeline

A major pathway for future work is the development of an autonomous multi-agent routing pipeline that matches the capabilities of the placement engine. While the current symbolic compiler is restricted to snapping and previewing routing tracks, detail routing is left to the downstream EDA tool. We propose integrating a specialized *Routing Specialist Agent* into the LangGraph state machine, structured as follows:

- **Grid-Based Obstacle Ingestion**: The routing agent will operate on a multi-layer symbolic grid. The routing grid modeler imports the placement obstacles (such as poly silicon gates, source/drain active areas, contacts, and VDD/GND power rails) from Stage 2.
- **LLM Routing Strategy Coordinator**: The LLM serves as a high-level strategist. It parses the circuit graph and outputs routing guidelines (e.g., "route the differential nets VX and VY symmetrically on Metal 2 and Metal 3, keep them shielded from clock lines, and minimize vias on the feedback loop").
- **AI-Guided A* Pathfinder & Maze Routing**: Guided by the LLM routing strategy, the pathfinder node executes initial track planning. It utilizes an A* search algorithm with an electrostatic cost function that penalizes wire proximity to prevent parasitic capacitive coupling.
- **Negotiated Congestion Rip-up & Reroute Loop**: If wire crossings or DRC spacing violations occur, the routing critic node generates a coordinate

cost vector. The routing engine uses a negotiated congestion algorithm (similar to PathFinder) to dynamically rip up conflicting wires, iteratively increasing the cost of shared grid coordinates until a clean routing layout is achieved.

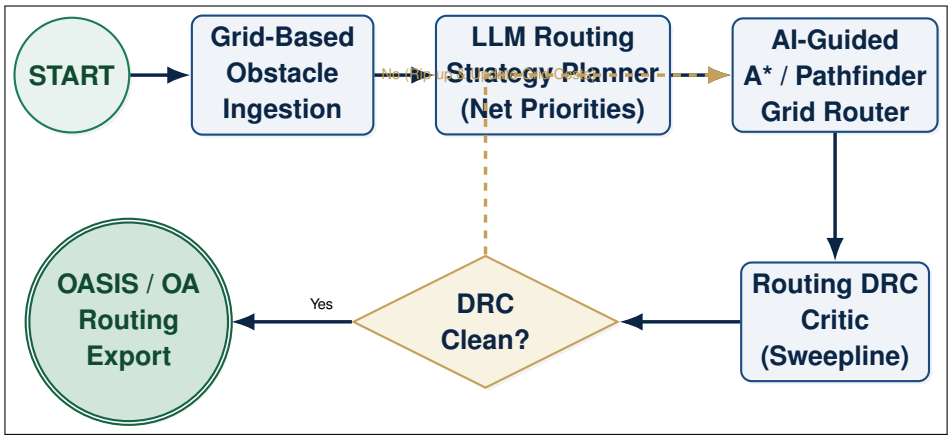


Figure 9.2: Multi-Agent Closed-Loop Automated Routing Pipeline Flowchart.

9.4.3 Thermal-Aware & Symmetrical Matching Optimization

Analog layouts are sensitive to thermal gradients and spatial matching imbalances. Future research will explore integrating a local thermal-simulation model into the feedback loop. The Critic agent will estimate thermal gradients across the layout cell based on expected transistor power dissipation. The placement specialist will then use this thermal map to arrange matched pairs (such as common-centroid configurations) symmetrically around hot-spots, minimizing offset voltages.

9.4.4 3D Vertical Integration PDK Support (Nanosheet & CFET)

As semiconductor technology scales beyond the 3nm node, traditional FinFET structures are being replaced by Gate-All-Around (GAA) nanosheets and Complementary FET (CFET) architectures. Future extensions will adapt the spatial compaction compiler to handle vertical stacked device rules, managing the unique vertical contact sharing, back-side power delivery networks, and vertical via placements required by these 3D technologies.

CHAPTER A

XML Co-Pilot Command Schema

This appendix lists the complete set of XML command schemas used by the chatbot co-pilot thread to communicate edit actions back to the PySide6 symbolic editor canvas view. To enable real-time, non-blocking command execution, the co-pilot orchestrator streams natural language responses from the LLM, extracts structured XML blocks using regular expressions, and dispatches them sequentially to the main GUI thread's command queue.

A.1 Command Parsing and Dispatching Pipeline

When the orchestrator worker thread receives a streamed response from the LLM, it extracts the command packet using a regular expression that targets blocks enclosed in `<CMD>` tags. The extracted block is parsed using Python's standard `xml.etree.ElementTree` library.
The parser extracts the action type from the `<action>` node and routes the payload to the layout tab controller (`layout_tab.py`), which maps the XML values to the command stack. The command stack instantiates a corresponding undo-aware command object (inheriting from `QUndoCommand`) and pushes it onto the application's active `QUndoStack`, triggering a canvas redraw and updating the head-less KLayout physical view.

A.2 Symbolic Movement Command (MOVE)

The `MOVE` command shifts a specified device instance horizontally or vertically by a relative number of symbolic grid slot units.

Transistor Move Schema

```
<CMD>
  <action>MOVE</action>
  <instance_id>MM1</instance_id>
  <x_slot>2</x_slot>
  <y_row>1</y_row>
</CMD>
```

A.3 Device Instance Swapping Command (SWAP)

The `SWAP` command exchanges the symbolic positions of two transistor instances on the placement canvas, which is frequently used to optimize matching orientations.

Transistor Swap Schema

```
<CMD>
  <action>SWAP</action>
  <instance_a>MM1</instance_a>
  <instance_b>MM2</instance_b>
</CMD>
```

A.4 Active Diffusion Abutment Command (ABUT)

The `ABUT` command groups two adjacent devices together, aligning their coordinates and setting PDK parameter flags to merge their active diffusions.

Active Region Abutment Schema

```
<CMD>
  <action>ABUT</action>
  <instance_a>MM1</instance_a>
  <instance_b>MM2</instance_b>
  <abut_type>shared</abut_type>
</CMD>
```

A.5 Instance Multi-Device Alignment Command (ALIGN)

The `ALIGN` command aligns a list of device instances along a specified geometric axis (horizontal or vertical) relative to a reference coordinate.

Instance Align Schema

```
<CMD>
  <action>ALIGN</action>
  <instances>
    <id>MM1</id>
    <id>MM2</id>
    <id>MM3</id>
  </instances>
```

```
  <axis>vertical</axis>
  <alignment>center</alignment>
</CMD>
```

A.6 Transistor Orientation Mirroring Command (ORIENT)

The `ORIENT` command mirrors the gate poly fingers of an instance (setting rotation and source-drain mirroring orientations like `R0`, `MY`, or `MX`).

Transistor Orientation Schema

```
<CMD>
  <action>ORIENT</action>
  <instance_id>MM1</instance_id>
  <orientation>MY</orientation>
</CMD>
```

A.7 Symmetry Constraint Enforcement Command (SYMMETRY)

The `SYMMETRY` command binds a list of matched pairs symmetrically about a specific midpoint axis, triggering reflection checks in the symmetry enforcer.

Symmetry Enforcement Schema

```
<CMD>
  <action>SYMMETRY</action>
  <group_id>diff_pair_1</group_id>
  <midpoint>0.294</midpoint>
</CMD>
```

A.8 DRC Auto-Heal Correction Command (DRC_HEAL)

The `DRC_HEAL` command applies absolute spacing correction shifts generated by the DRC Critic node to resolve clearance or overlapping violations.

DRC Auto-Heal Schema

```
<CMD>
  <action>DRC_HEAL</action>
```

```
  <instances>
    <instance>
      <id>MM1</id>
      <x_shift>0.140</x_shift>
      <y_shift>0.000</y_shift>
    </instance>
    <instance>
      <id>MM2</id>
      <x_shift>-0.140</x_shift>
      <y_shift>0.000</y_shift>
    </instance>
  </instances>
</CMD>
```

A.9 Physical Compilation and Export Command (EXPORT)

The `EXPORT` command triggers physical layout stream generation, compiling the current placement into an OASIS/GDSII file or writing updates to OpenAccess.

Physical Export Schema

```
<CMD>
  <action>EXPORT</action>
  <format>OASIS</format>
  <cell_name>comparator_latch</cell_name>
</CMD>
```

A.10 Command Stack Undo Command (UNDO)

The `UNDO` command rolls back the last command pushed onto the PySide6 command stack, restoring the previous coordinates and orientations from memory.

Command Stack Undo Schema

```
<CMD>
  <action>UNDO</action>
</CMD>
```

CHAPTER B

OpenAccess Watcher Script

This appendix lists the TCL background watchdog observer script (`cc_watcher.tcl`) and the database placement script (`ai_place.tcl`) that monitor coordinate exports from the layout chatbot and inject them into Synopsys Custom Compiler. By running this observer loop inside the EDA session, layout coordinates are synchronized in real-time without database locks.

B.1 Background Watchdog Observer Script (`cc_watcher.tcl`)

The file-polling watchdog script monitors the active workspace directory for updates to the coordinate exchange file (`ai_placement.txt`). It utilizes TCL's event-driven scheduling (`after`) to poll the file every 1000 ms without blocking the Custom Compiler GUI thread. When a modification is detected (by comparing the file's modification time `mtime` against a cached timestamp), it sources the placement script to update the cell view.

OpenAccess File Watcher Script (eda/cc_watcher.tcl)

```
# File Watcher: Monitoring coordinate exports from layout chatbot
set placement_file "ai_placement.txt"
set target_cell "comparator_latch"
set last_mtime 0

proc check_file_updates {} {
    global placement_file target_cell last_mtime

    if {[file exists $placement_file]} {
        set current_mtime [file mtime $placement_file]
        if {$current_mtime > $last_mtime} {
            set last_mtime $current_mtime
            puts "Watcher: Detected coordinate updates. Reloading placements..."
            source "ai_place.tcl"
            ai_place $placement_file $target_cell
        }
    }
    # Re-evaluate every 1000 milliseconds
```

```
    after 1000 check_file_updates
}

# Start the file checking loop
check_file_updates
puts "Watcher: Background file observer active."
```

B.2 OpenAccess Database Placement Script (ai_place.tcl)

The database placement script parses the coordinates and PDK parameters from the chatbot output and applies them directly to active OpenAccess layout cells. The script operates in four sequential phases to move existing transistors, instantiate dummy cells, add substrate taps, and update terminal nets via the GUI Property Editor to prevent database mismatches.

OpenAccess Placement Engine Script (eda/ai_place.tcl)

```
#
    ================================================================

# ai_place.tcl - High-Performance Database Integration &
    Placement Script
#
    ================================================================

proc ai_place_final {filename target_cell} {
    puts "Starting Layout Radar & Placement... (VERSION 26
        - Tap Integration)"

    # Get a list of all currently open layout/schematic
       designs in the EDA session
    set iter [oa::DesignGetOpenDesigns]
    set ns [oa::NativeNS]
    set libName "SAED_PDK_14"
    set tapLibName "taps"

    if [catch {open $filename r} fp] {
        puts "ERROR: Cannot open $filename"
        return
    }

    array set ai_data {}
    set dummy_list {}
    set tap_list {}
```

```
21
22      while {[gets $fp line] >= 0} {
23          set line [string trim $line]
24          if {$line == ""} continue
25
26          set elements [regexp -inline -all -- {\S+} $line]
27          set type_flag [lindex $elements 0]
28          set is_dummy 0; set cell_type ""
29
30          # Branch 1: TAP Cell Injection line (TAP <rail> <
                cell_type> <x> <y> <orient>)
31          if {$type_flag == "TAP"} {
32              set rail [lindex $elements 1]
33              set cell_type [lindex $elements 2]
34              set xPos [lindex $elements 3]
35              set yPos [lindex $elements 4]
36              set orient [lindex $elements 5]
37              lappend tap_list [list $rail $cell_type $xPos
                    $yPos $orient]
38              continue
39          }
40
41          # Branch 2: DUMMY Transistor line (DUMMY <name> <
                cell_type> <x> <y> ...)
42          if {$type_flag == "DUMMY"} {
43              set is_dummy 1
44              set name [lindex $elements 1]
45              set cell_type [lindex $elements 2]
46              set xPos [lindex $elements 3]
47              set yPos [lindex $elements 4]
48              set orient [lindex $elements 5]
49              lappend dummy_list $name
50
51          # Branch 3: Standard Active Device line (<name> <x
                > <y> <orient> ...)
52          } else {
53              set name [lindex $elements 0]
54              set xPos [lindex $elements 1]
55              set yPos [lindex $elements 2]
56              set orient [lindex $elements 3]
57          }
58
59          set leftVal ""; set rightVal ""; set l_val ""; set
                m_val ""; set nf_val ""; set nfin_val ""
60          set d_net ""; set g_net ""; set s_net ""; set
                b_net ""
61
```

```
62          regexp {left_abut=([01])} $line match val && {set
               leftVal $val}
63          regexp {right_abut=([01])} $line match val && {set
               rightVal $val}
64          regexp {l=([0-9.]+)} $line match val && {set l_val
               $val}
65          regexp {m=([0-9.]+)} $line match val && {set m_val
               $val}
66          regexp {nf=([0-9.]+)} $line match val && {set
               nf_val $val}
67          regexp {nfin=([0-9.]+)} $line match val && {set
               nfin_val $val}
68
69          regexp {D=([^ ]+)} $line match val && {set d_net
               $val}
70          regexp {G=([^ ]+)} $line match val && {set g_net
               $val}
71          regexp {S=([^ ]+)} $line match val && {set s_net
               $val}
72          regexp {B=([^ ]+)} $line match val && {set b_net
               $val}
73
74          set ai_data($name) [list $xPos $yPos $orient
               $leftVal $rightVal $is_dummy $cell_type $l_val
               $m_val $nf_val $nfin_val $d_net $g_net $s_net
               $b_net]
75      }
76      close $fp
77
78      set target_win [gi::getActiveWindow]
79      set target_ctx [de::getActiveContext]
80      set total_moved 0
81
82      while { [set cv [oa::getNext $iter]] != "0" && $cv !=
           "" } {
83          set cellStr ""; set viewStr ""
84          catch { set cellStr [oa::get [oa::getCellName $cv]
               $ns] }
85          catch { set viewStr [oa::get [oa::getViewName $cv]
               $ns] }
86
87          if {$cellStr != $target_cell || [string match -
               nocase "*schematic*" $viewStr]} { continue }
88
89          set block [oa::getTopBlock $cv]
90          set instIter [oa::getInsts $block]
91
92          # PHASE 1: Move Existing Instances
```

```
93          while { [set inst [oa::getNext $instIter]] != "0"
               && $inst != "" } {
94              set instStr ""
95              catch { set instStr [oa::get [oa::getName
                   $inst] $ns] }
96
97              if { [info exists ai_data($instStr)] } {
98                  set target $ai_data($instStr)
99                  if {[lindex $target 5]} { continue }
100
101                 oa::setOrigin $inst [list [expr {double([
                       lindex $target 0])}] [expr {double([
                       lindex $target 1])}]]
102                 oa::setOrient $inst [lindex $target 2]
103
104                 set leftVal [lindex $target 3]; set
                       rightVal [lindex $target 4]
105                 if {$leftVal != ""} { catch {db::
                       setParamValue "leftAbut" -value
                       $leftVal -of $inst} }
106                 if {$rightVal != ""} { catch {db::
                       setParamValue "rightAbut" -value
                       $rightVal -of $inst} }
107                 incr total_moved
108             }
109         }
110
111         # PHASE 2: Create Dummies & Size
112         foreach dummy_name $dummy_list {
113             set target $ai_data($dummy_name)
114             set cell_type [lindex $target 6]
115             set x [expr {double([lindex $target 0])}]
116             set y [expr {double([lindex $target 1])}]
117
118             set inst ""
119             if {[catch {set inst [le::createInst
                   $dummy_name -design $cv -libName $libName -
                   cellName $cell_type -viewName "layout" -
                   origin [list $x $y] -orient [lindex $target
                    2]]} err]} {
120                 continue
121             }
122             set leftVal [lindex $target 3]; set rightVal [
                   lindex $target 4]; set l_val [lindex
                   $target 7]; set m_val [lindex $target 8];
                   set nf_val [lindex $target 9]; set nfin_val
                    [lindex $target 10]
123
```

```
124             catch {db::setParamValue "usage" -value "Dummy
                    " -of $inst}
125             if {$leftVal != ""} { catch {db::setParamValue
                     "leftAbut" -value $leftVal -of $inst} }
126             if {$rightVal != ""} { catch {db::
                    setParamValue "rightAbut" -value $rightVal
                    -of $inst} }
127             if {$l_val != ""} { catch {db::setParamValue "
                    l" -value $l_val -of $inst} }
128             if {$m_val != ""} { catch {db::setParamValue "
                    m" -value $m_val -of $inst} }
129             if {$nf_val != ""} { catch {db::setParamValue
                    "nf" -value $nf_val -of $inst} }
130             if {$nfin_val != ""} { catch {db::
                    setParamValue "nfin" -value $nfin_val -of
                    $inst} }
131         }
132
133         # PHASE 3: Create Taps
134         set tap_counter 0
135         foreach tap_data $tap_list {
136             set rail [lindex $tap_data 0]
137             set cell_type [lindex $tap_data 1]
138             set x [expr {double([lindex $tap_data 2])}]
139             set y [expr {double([lindex $tap_data 3])}]
140             set orient [lindex $tap_data 4]
141             set name "TAP_${rail}_${tap_counter}"
142             incr tap_counter
143
144             catch {le::createInst $name -design $cv -
                    libName $tapLibName -cellName $cell_type -
                    viewName "layout" -origin [list $x $y] -
                    orient $orient}
145         }
146     }
147
148     # PHASE 4: GUI Property Editor Connectivity Injection
149     set propEd ""
150     if { ![catch {set propEd [gi::getAssistants
                deProperty Editor -from $target_win]}] } {
151         update; after 300
152         foreach dummy_name $dummy_list {
153             set target $ai_data($dummy_name)
154             set x [expr {double([lindex $target 0])}]; set
                     y [expr {double([lindex $target 1])}]
155             set d_net [lindex $target 11]; set g_net [
                    lindex $target 12]; set s_net [lindex
                    $target 13]; set b_net [lindex $target 14]
```

```
156
157             de::deselectAll $target_ctx
158             set clicked 0
159             catch {
160                 set fig [de::getActiveFigure $target_win -
                        point [list $x $y] -index 0 -intent
                        none]
161                 if {$fig != "" && $fig != "0"} { de::
                        select $fig; set clicked 1 }
162             }
163             if {$clicked} {
164                 update; after 250
165                 catch {
166                     gi::setItemSelection {attributes} -
                            index {usage,all} -in $propEd
167                     gi::setField {attributes} -value {
                            Dummy} -index {usage,Value} -in
                            $propEd
168                     gi::setField {instanceTerminalsExpand}
                             -value {true} -in $propEd
169
170                     if {$g_net != ""} { gi::setField {
                            instanceTerminals} -value $g_net -
                            index {G,Net Name} -in $propEd }
171                     if {$d_net != ""} { gi::setField {
                            instanceTerminals} -value $d_net -
                            index {D,Net Name} -in $propEd }
172                     if {$s_net != ""} { gi::setField {
                            instanceTerminals} -value $s_net -
                            index {S,Net Name} -in $propEd }
173                     if {$b_net != ""} { gi::setField {
                            instanceTerminals} -value $b_net -
                            index {B,Net Name} -in $propEd }
174                     gi::executeAction
                            dePropertyEditorApplyChanges -in
                            $propEd
175                 }
176                 update; after 150
177             }
178         }
179     }
180     de::deselectAll $target_ctx
181 }
182 proc ai_place {filename target_cell} {
183     ai_place_final $filename $target_cell
184 }
```

CHAPTER C

LangGraph Multi-Agent System Prompts

This appendix lists the system prompts of the primary agents inside the LangGraph layout placement pipeline. It also specifies the target models and tool skills associated with each node to provide a complete view of the multi-agent co-pilot backend.

C.1 Intent Router Agent

The *Intent Router* acts as the entry node of the state machine. It parses the designer's natural language input and routes it to the appropriate specialized node.

- **Model Used:** GPT-4o or Llama-3-8B.
- **Skills:** Intent routing, natural-language parsing, parameter extraction.

Intent Router Prompt

```
You are the central Intent Router for an AI-assisted
   analog IC layout co-pilot.
Your task is to analyze the user's chat input and classify
    the intent.

Available routing categories are:
1. "placement_sp": If the user is asking to place, move,
   swap, align, or abut transistors.
2. "drc_critic": If the user is asking to run design rule
   checks or heal spacing violations.
3. "general": If the user is asking a general question,
   explaining a circuit, or querying PDK rules.

Instructions:
- Extract relevant parameters (e.g. device names, slot
   numbers, orientations, directions).
- Output ONLY a JSON object containing:
  {
    "intent": "<category>",
    "parameters": { ... }
  }
```

C.2 Placement Specialist Agent

The *Placement Specialist* optimizes the horizontal sequencing and vertical stacking rows of the symbolic layout, enforcing matching symmetries.

- **Model Used:** Claude 3.5 Sonnet (for complex matched topologies) or Llama-3-8B (for low-latency local execution).
- **Skills:** `move_device`, `swap_devices`, `abut_devices`, `set_orientation`.

Placement Specialist Prompt

```
You are an expert Analog IC Placement Specialist.
Your goal is to arrange transistors on the symbolic grid
    to optimize electrical matching, active diffusion
    sharing (abutment), and signal flow direction.

RULES:
1. Signal flow is strictly Top-to-Bottom. PMOS must have y
     < 0, NMOS must have y >= 0.
2. Matched differential pairs must be placed side-by-side,
     symmetric about x=0.
3. Reference and copy devices in current mirrors must be
    adjacent.
4. Cross-coupled latch pairs must face each other (using
    complementary mirror states).
5. If the DRC Critic reports violations, adjust
    coordinates by the requested shift offsets.

Output a sequential list of command blocks: MOVE, SWAP,
    ALIGN, or ABUT.
Example:
[
  {"action": "move", "device": "MM1", "x": -1.5, "y": 1},
  {"action": "abut", "device_a": "MM1", "device_b": "MM2"}
]
```

C.3 DRC Critic Agent

The *DRC Critic* runs verification checks on candidate placements, identifying violations and generating corrective coordinate shifts.

- **Model Used:** GPT-4o.
- **Skills:** Sweepline collision checking, minimum spacing checks, corrective offset vector generation.

DRC Critic Prompt

```
You are a Design Rule Verification Critic.
Your task is to analyze device clearance and spacing
    violations on the proposed coordinates.

For each violation (overlap, gate spacing, boundary
    collision):
1. Identify the conflicting devices.
2. Calculate the exact shift offset (in microns or slots)
    required to resolve the overlap based on PDK rule
    variables.
3. Formulate a correction instruction (e.g., "Shift MM2
    right by 0.140um to clear gate spacing violation with
    MM1").
4. Return the list of corrective recommendations to the
    Placer.
5. If zero violations are detected, return drc_pass = true
    .
```

C.4 Topology Analyst Agent

The *Topology Analyst* evaluates the placement quality by analyzing parasitic matching, dummy placements, and symmetry axes.

- **Model Used:** GPT-4o.
- **Skills:** Symmetry extraction, parasitic estimation, net connectivity checking.

Topology Analyst Prompt

```
You are an Analog Layout Analyst.
Your task is to evaluate the quality of the placement
    proposed by the specialist.

Analyze the layout state and verify:
1. Are matched differential pairs aligned about the same
    symmetry axis?
2. Are dummy transistors placed at the cell boundaries to
    prevent STI edge stress?
3. Are the active region terminal connections aligned to
    minimize signal routing crossings?

Provide a summary rating from 1 to 10 and list specific
    improvements for the Placer.
```

C.5 General Chatbot Agent

The *General Assistant* handles conversational designer requests, explaining circuit topologies, layout guidelines, or PDK design rules.

- **Model Used:** Llama-3-8B or GPT-4o.
- **Skills:** Knowledge base retrieval, PDK rule lookup, SPICE netlist summary.

General Chatbot Prompt

```
You are a conversational Analog IC Layout Assistant.
Your task is to help the designer by answering questions,
    explaining layout rules, and summarizing netlist
    topologies.

Instructions:
- Provide clear, concise explanations using standard
    analog engineering terminology.
- Refer to specific PDK rules (e.g. gate pitch, metal
    width, active area spacing) when answering design rule
    questions.
- Assist the designer in planning layout symmetries before
     initiating automated placement.
```

Bibliography

[1] K. Kunal, M. Madhusudan, A. K. Sharma, W. Xu, S. M. Burns, R. Harjani, J. Hu, D. A. Kirkpatrick, and S. S. Sapatnekar, "Align: Open-source analog layout automation from the ground up," in *Proceedings of the 56th ACM/IEEE Design Automation Conference (DAC)*. ACM/IEEE, 2019.

[2] C. Wang, F. Yang, and K. Zhu, "Ai-enabled layout automation for analog and rf ic: Current status and future directions," in *Proceedings of the IEEE International Symposium on Radio-Frequency Integration Technology (RFIT)*. IEEE, 2024, pp. 1–3.

[3] P. Mangalagiri, P. Mosalikanti, F. Zafar, L. Qian, P. Chang, A. J. Kurian, and V. Saripalli, "Cdls: Constraint driven generative ai framework for analog layout synthesis," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. ACM/IEEE, 2024, pp. 1–6.

[4] C.-C. Chang, J. Pan, Z. Xie, Y. Li, Y. Lin, J. Hu, and Y. Chen, "Fully automated machine learning model development for analog placement quality prediction," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2024.

[5] A. Hammoud, C. Goyal, S. Pathen, A. Dai, A. Li, G. Kielian, and M. Saligane, "Human language to analog layout using glayout layout automation framework," in *Proceedings of the ACM/IEEE 6th Symposium on Machine Learning for CAD (MLCAD)*. IEEE, 2024.

[6] B. Liu, H. Zhang, X. Gao, Z. Kong, X. Tang, Y. Lin, R. Wang, and R. Huang, "Layoutcopilot: An llm-powered multi-agent collaborative framework for interactive analog layout design," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2024.